

React.js - Développeur Web

◆ Chapitre 1 : Introduction à React

◆ Présentation de React : Pourquoi l'utiliser ?

★ Qu'est-ce que React ?

React est une **bibliothèque JavaScript** développée par **Facebook (Meta)** et utilisée pour la création d'interfaces utilisateur interactives et réactives. Il est principalement utilisé pour construire des **applications web monopages (SPA - Single Page Applications)** et des interfaces dynamiques.

◆ Présentation de React : Pourquoi l'utiliser ?

★ Qu'est-ce que React ?

React est une **bibliothèque JavaScript** développée par **Facebook (Meta)** et utilisée pour la création d'interfaces utilisateur interactives et réactives. Il est principalement utilisé pour construire des **applications web monopages (SPA - Single Page Applications)** et des interfaces dynamiques.

✂ Pourquoi utiliser React ?

✓ Composants réutilisables

- React est basé sur une architecture **modulaire** où l'interface est découpée en **composants** indépendants et réutilisables.
- Facilite la maintenance et l'évolutivité des projets.

✓ Performance optimisée avec le Virtual DOM

- React utilise un **Virtual DOM** qui permet de mettre à jour l'interface de manière **efficace et rapide**, sans recharger la page entière.
- Seules les parties de l'UI qui ont changé sont mises à jour, améliorant ainsi les performances.

✓ Unidirectional Data Flow (Flux de données unidirectionnel)

- Contrairement à d'autres frameworks, React adopte un **flux de données unidirectionnel**, ce qui facilite la gestion de l'état et réduit les bugs.

✓ Grande communauté et écosystème riche

- Une communauté active avec des milliers de **bibliothèques et outils** disponibles (Redux, React Router, Tailwind, Material UI, etc.).

✓ Facilité d'apprentissage

- Syntaxe **JSX** intuitive qui mélange JavaScript et HTML.
- Approche déclarative qui simplifie le développement des interfaces utilisateur.

✓ Support mobile avec React Native

- React permet aussi de développer des **applications mobiles natives** avec **React Native**, en réutilisant une grande partie du code écrit pour le web.

◆ Installation et configuration de l'environnement (Node.js, npm, Vite/Create React App)

Avant de commencer à coder avec **React**, il faut configurer un environnement de développement adapté. Voici les étapes essentielles pour l'installation et la configuration.

1. Installation de Node.js et npm

Pourquoi ?

React utilise **Node.js** pour exécuter des outils comme **npm (Node Package Manager)** et gérer les dépendances.

◆ Vérifier si Node.js est installé

Ouvre un terminal et tape la commande suivante :

```
node -v
```

Si Node.js est installé, la version s'affiche. Si ce n'est pas le cas, installe-le en suivant les étapes ci-dessous.

◆ Télécharger et installer Node.js

- Va sur <https://nodejs.org/>
- Télécharge la version **LTS (Long-Term Support)**
- Installe Node.js (npm est inclus avec Node.js)

◆ Vérifier npm

Après l'installation, vérifie que **npm** est bien installé avec :

```
npm -v
```

2. Créer un projet React avec Vite (Recommandé)

Pourquoi Vite ?

Vite est plus rapide que Create React App (CRA) car il optimise le développement avec un serveur de build ultra-rapide.

◆ **Installer Vite et créer un projet React**

Dans le terminal, exécute :

```
npm create vite@latest nom-du-projet --template react
```

Remplace **nom-du-projet** par le nom de ton projet.

◆ **Aller dans le dossier du projet**

```
cd nom-du-projet
```

◆ **Installer les dépendances**

```
npm install
```

◆ **Démarrer le projet**

```
npm run dev
```

Cela lancera un serveur local (par défaut sur `http://localhost:5173`).

3. Alternative : Créer un projet React avec Create React App (CRA)

⚠ **Create React App est plus lent** et moins optimisé que Vite, mais reste une option classique.

◆ **Créer un projet avec CRA**

```
npx create-react-app nom-du-projet
```

(Npx est inclus avec npm, il permet d'exécuter des paquets sans les installer globalement.)

◆ **Aller dans le dossier du projet et lancer le serveur**

```
cd nom-du-projet  
npm start
```

L'application s'ouvre sur `http://localhost:3000`.

◆ Concepts clés : Composants, JSX, Virtual DOM

Avant de commencer à coder en React, il est essentiel de comprendre ses concepts fondamentaux.

1. Composants (Components)

◆ Définition

- Un **composant** est un **bloc réutilisable** qui représente une partie de l'interface utilisateur (bouton, formulaire, carte, etc.).
- En React, **tout est basé sur des composants**.

◆ Types de composants

☞ Composants fonctionnels (recommandés)

- Ce sont de simples **fonctions JavaScript** qui retournent du JSX.
- Plus faciles à lire, tester et optimiser.

Exemple :

```
jsx

function Bonjour(props) {
  return <h1>Salut, {props.nom} !</h1>;
}
```

☞ Composants de classe (ancienne méthode)

- Définis avec une **classe ES6** et un `render()`.
- Utilisés avant l'introduction des **Hooks** (React 16.8).

Exemple :

```
jsx

class Bonjour extends React.Component {
  render() {
    return <h1>Salut, {this.props.nom} !</h1>;
  }
}
```

★ Bonnes pratiques

- ✓ Un composant **doit être indépendant et réutilisable**
- ✓ Toujours nommer les composants en **PascalCase** (`MonComposant.js`)

2. JSX (JavaScript XML)

◆ Définition

JSX est une **extension syntaxique** qui permet d'écrire du HTML directement dans du JavaScript.

◆ Pourquoi JSX ?

- ✓ Plus lisible et intuitif
- ✓ Permet de combiner logique et UI dans un seul fichier
- ✓ Sécurisé et optimisé après compilation

◆ Exemple de JSX

```
jsx  
  
const titre = <h1>Bienvenue</h1>;
```

◆ JSX vs JavaScript

Sans JSX, on écrirait ceci :

```
jsx  
  
const titre = React.createElement('h1', {}, 'Bienvenue');
```

Avec JSX, c'est **plus propre et lisible** ✓

✦ Bonnes pratiques JSX

- ✓ Un composant **doit retourner un seul élément parent**

✗ Mauvais :

```
jsx  
  
function App() {  
  return (  
    <h1>Salut</h1>  
    <p>Bienvenue</p>  
  );  
}
```

✓ Bon :

```
jsx  
  
function App() {  
  return (  
    <>  
      <h1>Salut</h1>  
      <p>Bienvenue</p>  
    </>  
  );  
}
```

```
);  
}
```

🔗 **Astuce :** Utiliser `<></>` (**Fragments**) pour éviter des `<div>` inutiles.

3. Virtual DOM 📄

🔗 Qu'est-ce que le DOM ?

Le **DOM (Document Object Model)** est la structure HTML interprétée par le navigateur.

🔗 Problème avec le DOM classique

- Modifier directement le DOM est **lent** ⚠️
- Chaque mise à jour **rafraîchit toute la page**, ce qui **ralentit les performances**

🔗 Solution : Virtual DOM

- **React crée une copie virtuelle du DOM en mémoire**
- Lorsqu'un changement est détecté, **React met à jour uniquement les parties modifiées**, au lieu de recharger toute la page
- Cela **améliore considérablement les performances**

🔗 Comment ça marche ?

1. React garde un **Virtual DOM** en mémoire
2. Lorsqu'un état change, React compare l'ancien et le nouveau Virtual DOM
3. Il met à jour **seulement les parties modifiées** du **vrai DOM**
4. 🔗 **Exemple illustré**

Action utilisateur	Virtual DOM met à jour	DOM réel est modifié
L'utilisateur clique sur un bouton	Virtual DOM met à jour le bouton	React met à jour uniquement ce bouton dans le DOM

★ Avantages du Virtual DOM

- ✓ **Optimisation des performances**
- ✓ **Moins de manipulations du DOM réel**
- ✓ **Expérience utilisateur fluide**

Conclusion

- ✓ **Les composants** rendent le code modulaire et réutilisable.
- ✓ **JSX** permet d'écrire du HTML directement dans JavaScript, rendant le code plus lisible.
- ✓ **Le Virtual DOM** améliore les performances en mettant à jour uniquement les parties nécessaires.

◆ Premier projet React : structure d'un projet

1. Structure d'un projet React

Après avoir créé un projet avec **Vite** (ou **Create React App**), voici la structure typique :

mon-projet-react/

```
| — node_modules/      # Dépendances installées
| — public/             # Fichiers publics (favicon, index.html...)
| — src/                # Code source de l'application
|   | — App.jsx         # Composant principal
|   | — main.jsx        # Point d'entrée de l'application
|   | — components/     # Dossier pour les composants React
|   | — assets/          # Images, styles et ressources
| — .gitignore          # Fichiers à ignorer par Git
| — package.json        # Liste des dépendances et scripts
| — vite.config.js      # Configuration de Vite
| — README.md           # Documentation du projet
```

Fichiers importants :

- `App.jsx` : Composant principal
- `main.jsx` : Monte l'application dans le DOM
- `package.json` : Contient les dépendances et scripts

2. Création d'une première application React

Nous allons créer une **application simple** qui affiche un message de bienvenue et un compteur interactif.

Étape 1 : Créer un projet React avec Vite

Dans ton terminal, exécute :

```
npm create vite@latest mon-premier-react --template react
```

```
cd mon-premier-react
```

```
npm install
```

```
npm run dev
```

Ouvre <http://localhost:5173> dans ton navigateur

```
D:\CFITECH\React\cours react 2025\exercices>npm create vite@latest test --template react
```

```
> npx
> create-vite test react

? Select a framework: » - Use arrow-keys. Return to submit.
  Vanilla
  Vue
>  React✓
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Angular
  Others
```

```
> npx
> create-vite test react

✓ Select a framework: » React
? Select a variant: » - Use arrow-keys. Return to submit.
  TypeScript
  TypeScript + SWC
>  JavaScript✓
  JavaScript + SWC
  React Router v7 ↗
```

```
> npx
> create-vite test react

✓ Select a framework: » React
✓ Select a variant: » JavaScript

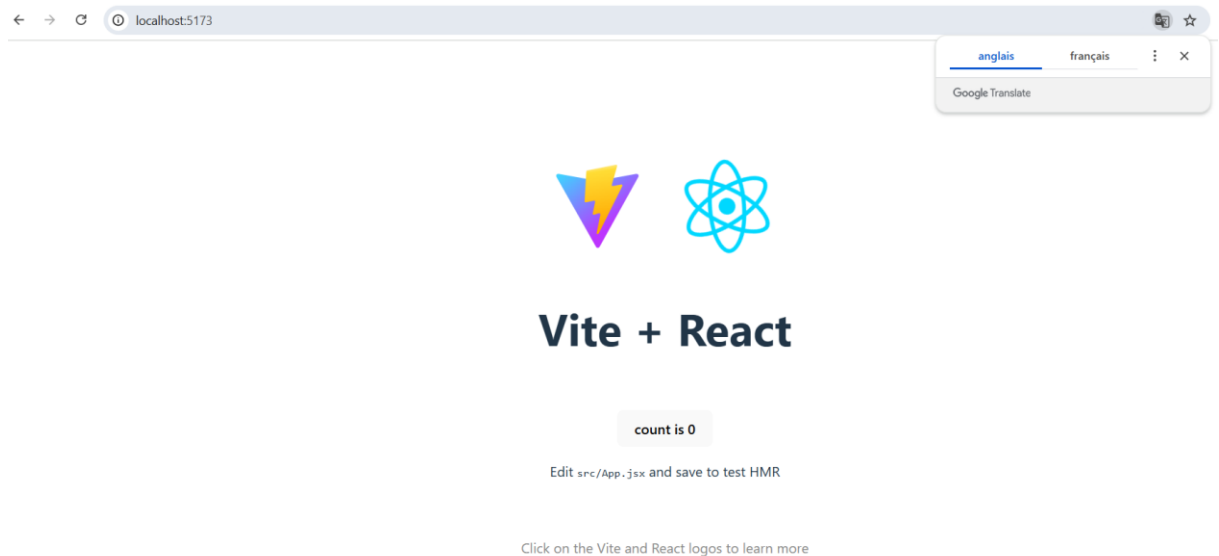
Scaffolding project in D:\CFITECH\React\cours react 2025\exercices\test...

Done. Now run:

cd test ✓
npm install ✓
npm run dev ✓
```

```
C:\Windows\system32\cmd.e: X + v

VITE v6.0.11 ready in 270 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```



Étape 2 : Modifier `App.jsx` pour afficher un message de bienvenue

Dans `src/App.jsx`, remplace le code par ceci :

`jsx`

```
import { useState } from 'react';

import './App.css';

function App() {

  const [count, setCount] = useState(0);


  return (

    <div className="container">

      <h1>Bienvenue sur mon premier projet React !</h1>

      <p>Ceci est une application React simple.</p>


      <h2>Compteur : {count}</h2>

      <button onClick={() => setCount(count + 1)}>➕ Augmenter</button>

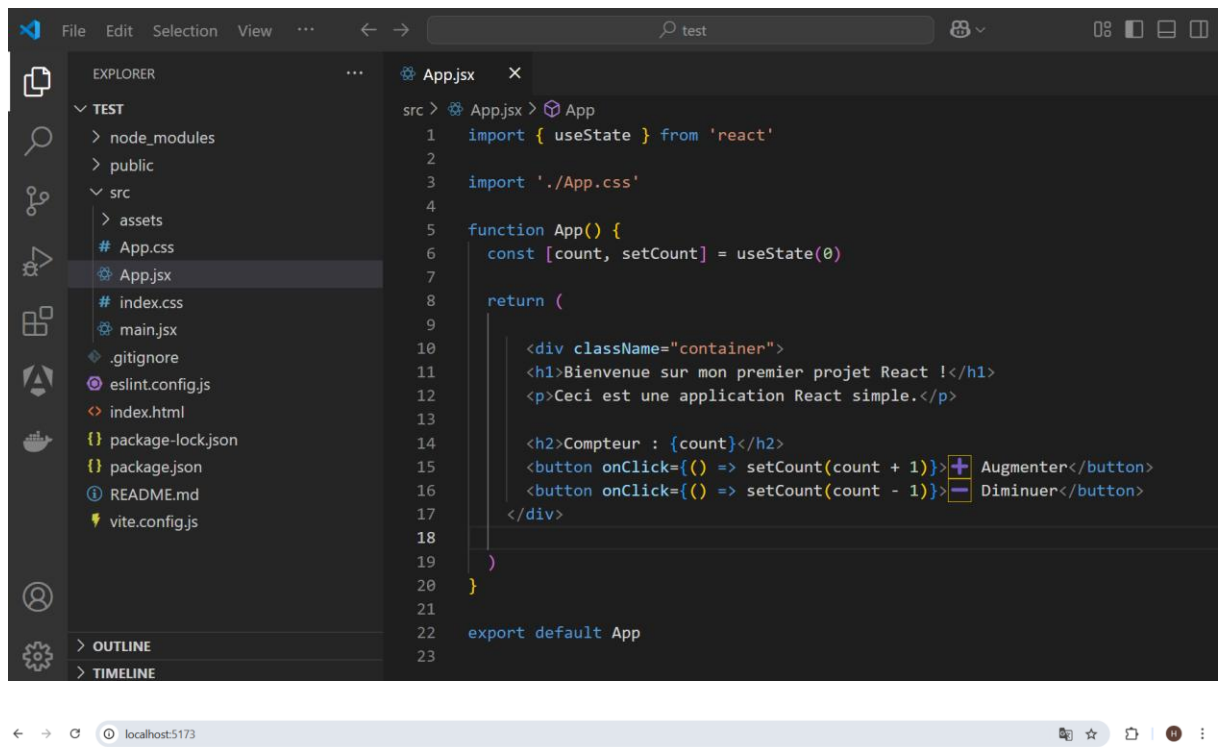
      <button onClick={() => setCount(count - 1)}>➖ Diminuer</button>

    </div>

  );

}

export default App;
```



Bienvenue sur mon premier projet React !

Ceci est une application React simple.

Compteur : -4

+ Augmenter - Diminuer

Étape 3 : Ajouter un peu de style

Dans `src/App.css`, remplace le contenu par :

```
.container {  
  
  text-align: center;  
  
  font-family: Arial, sans-serif;  
  
  margin-top: 50px;  
  
}
```

```
h1 {  
  color: #2c3e50;  
}
```

```
button {  
  margin: 10px;  
  padding: 10px 15px;  
  font-size: 16px;  
  cursor: pointer;  
  border: none;  
  border-radius: 5px;  
}
```

```
button:hover {  
  opacity: 0.8;  
}
```

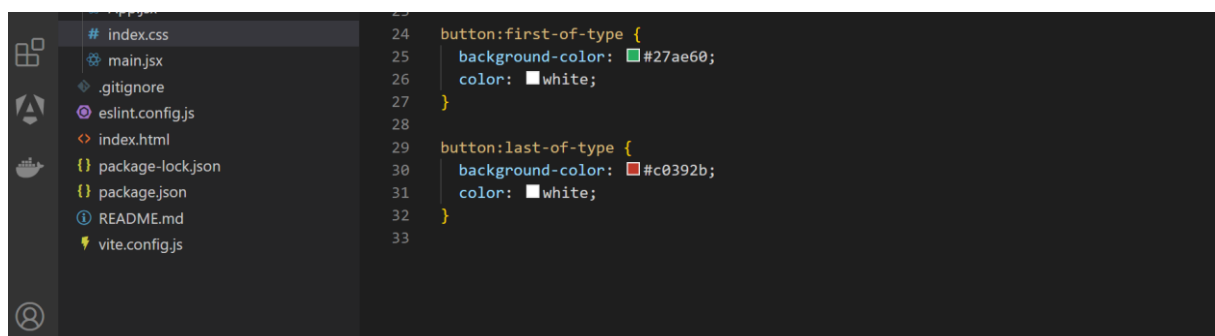
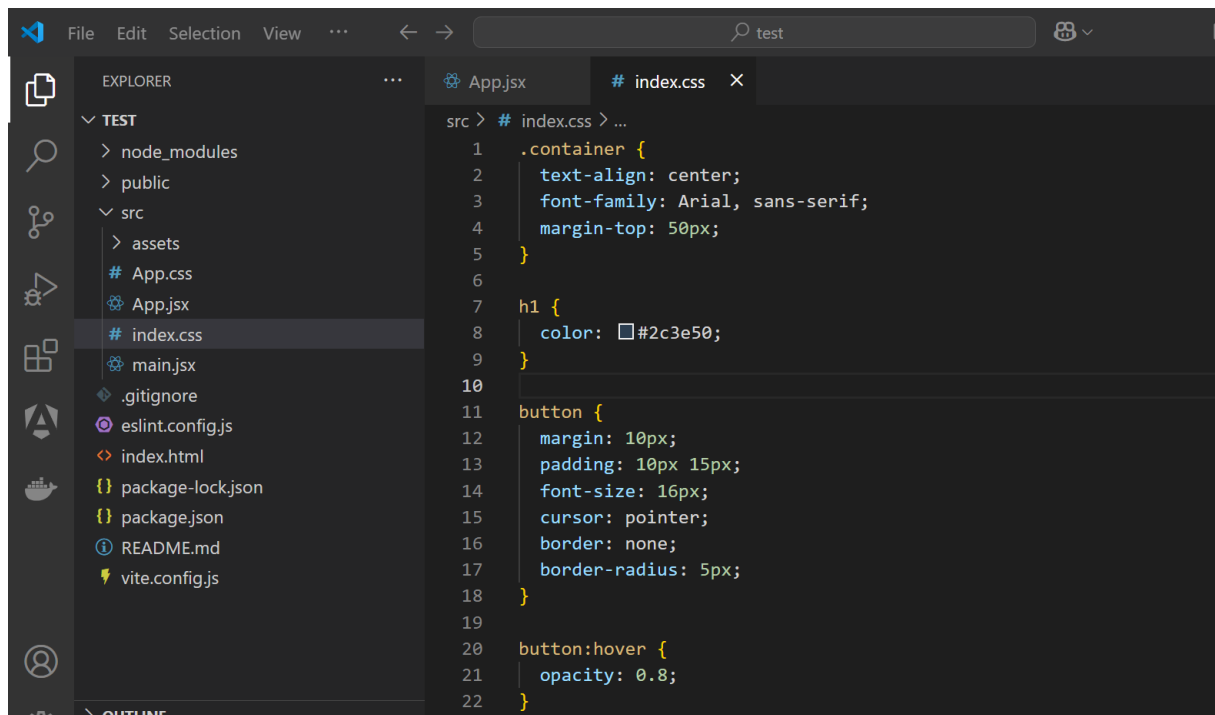
```
button:first-of-type {  
  background-color: #27ae60;  
  color: white;  
}
```

```
button:last-of-type {  
  background-color: #c0392b;
```



```
color: white;

}
```



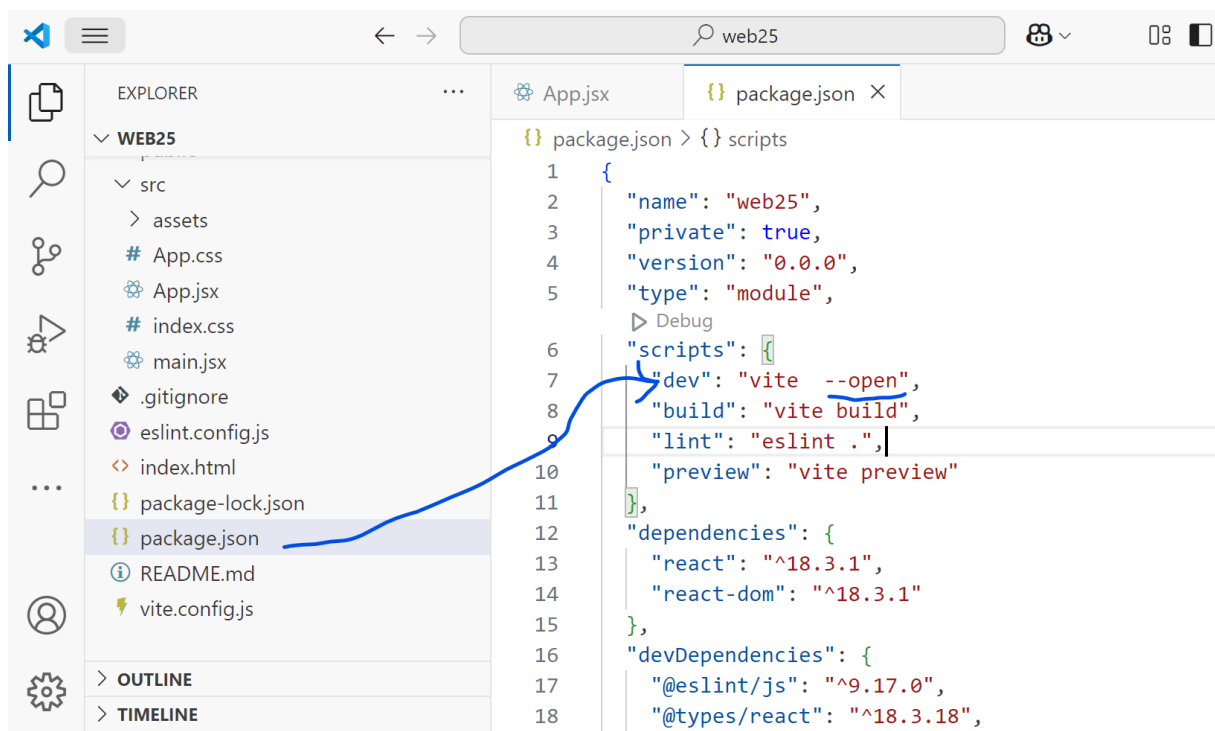
Étape 4 : Lancer l'application

Dans le terminal, tape :

```
npm run dev
```

```
D:\CFITECH\React\cours react 2025\exercices\test>npm run dev
```

Vous pouvez aussi modifier ton `package.json` pour que le script `dev` ouvre automatiquement le navigateur :



`npm run dev`

Bienvenue sur mon premier projet React !

Ceci est une application React simple.

Compteur : 0



Résumé

- ✓ On a **créé un projet React avec Vite**
- ✓ On a **compris la structure d'un projet**
- ✓ On a **ajouté un premier composant avec un état (useState)**
- ✓ On a **appliqué du CSS pour améliorer l'interface**

◆ Chapitre 2 : Composants et Props

Les **composants** sont la base de React. Ils permettent de **réutiliser du code** et de **structurer une application** de manière modulaire.

1. Qu'est-ce qu'un composant ?

Un **composant** en React est une **fonction** ou une **classe** qui retourne du JSX.

Il peut représenter une **petite partie de l'interface** (ex. un bouton) ou un **gros bloc** (ex. une page entière).

Il existe **deux types de composants** :

1. **Les composants fonctionnels** (recommandés)
2. **Les composants de classe** (moins utilisés depuis les Hooks)

2. Création d'un composant fonctionnel

◆ Exemple d'un composant simple

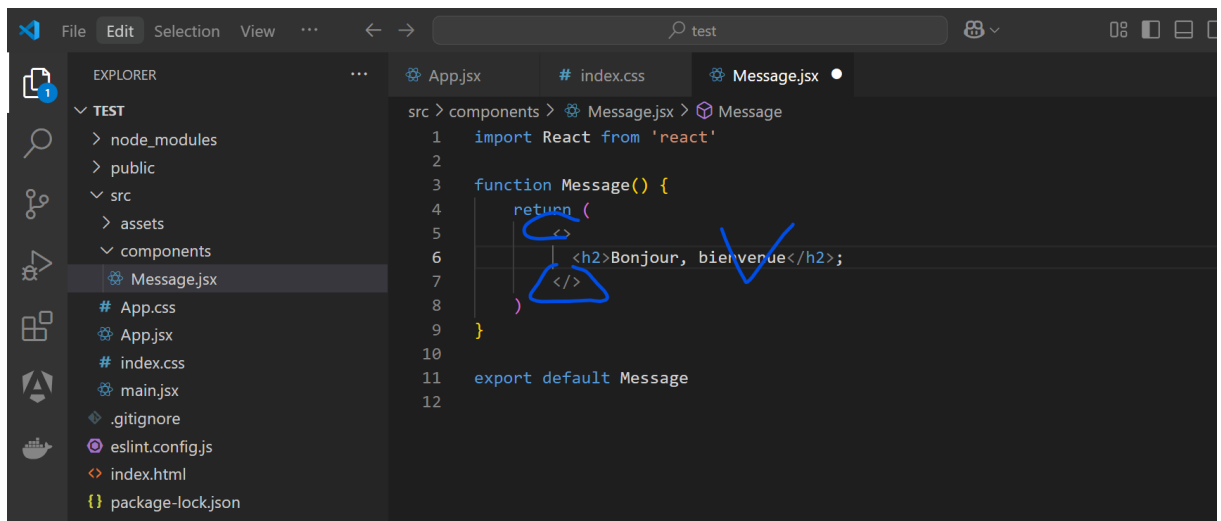
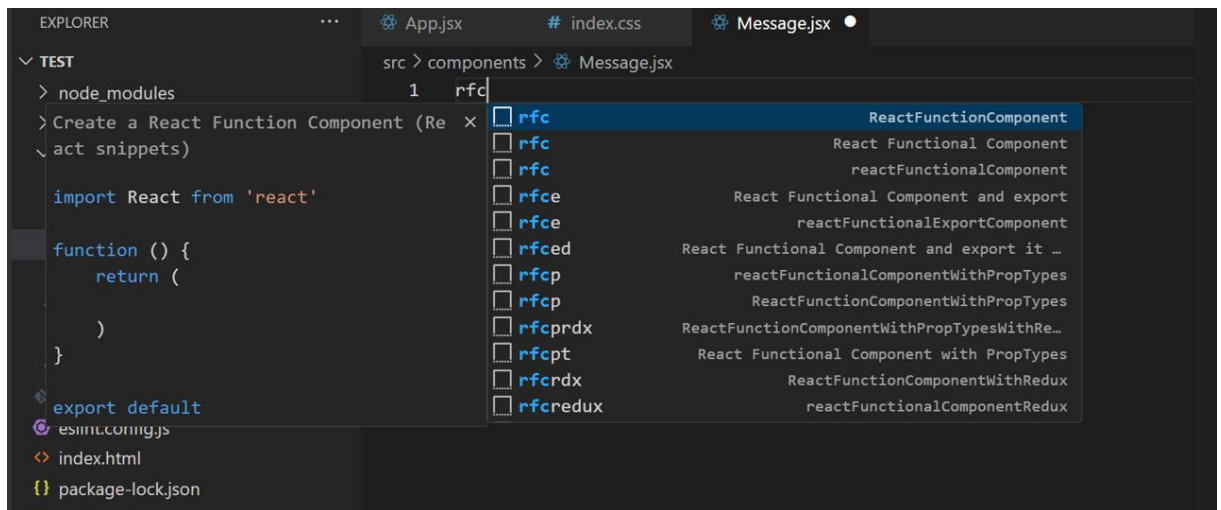
Dans le dossier `src/components/`, crée un fichier `Message.jsx` et ajoute ce code :

```
function Message() {  
  
  return <h2> Bonjour, bienvenue</h2>;  
  
}
```

```
export default Message;
```

Si tu utilises **React avec l'extension ES7+ React/Redux/React-Native snippets** sur VS Code, tu peux taper des raccourcis comme pour créer un composant:

- `rafce` → **React Arrow Function Component with Export**
- `rfce` → **React Function Component with Export**
- `rfc` → **React Function Component**
- `rafc` → **React Arrow Function Component**



◆ Utilisation du composant dans App . jsx

Dans App . jsx, importe et utilise le composant :

```
import Message from "./components/Message";
```

```
function App() {

  return (

    <div>

      <h1>Mon Application</h1>
```

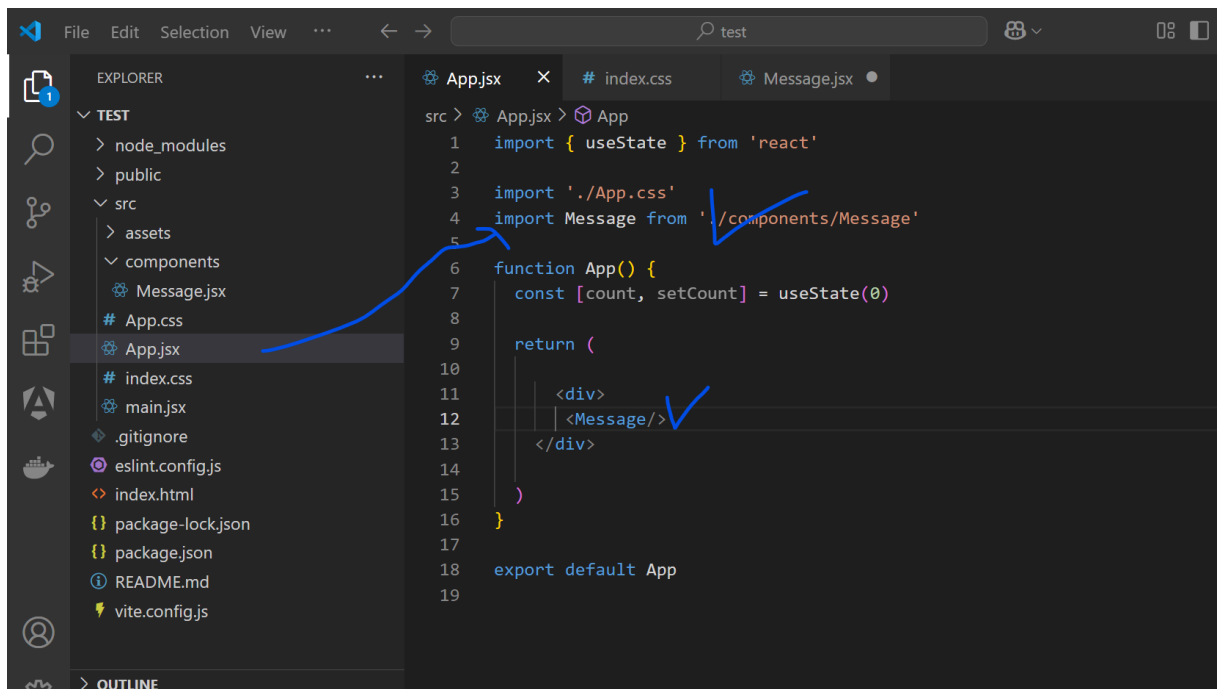
```
<Message />

</div>

);

}
```

export default App;



✓ **Résultat** : L'application affiche un message grâce à un composant réutilisable.



Bonjour, bienvenue

;

3. Passage de données avec les Props

Qu'est-ce qu'une prop ?

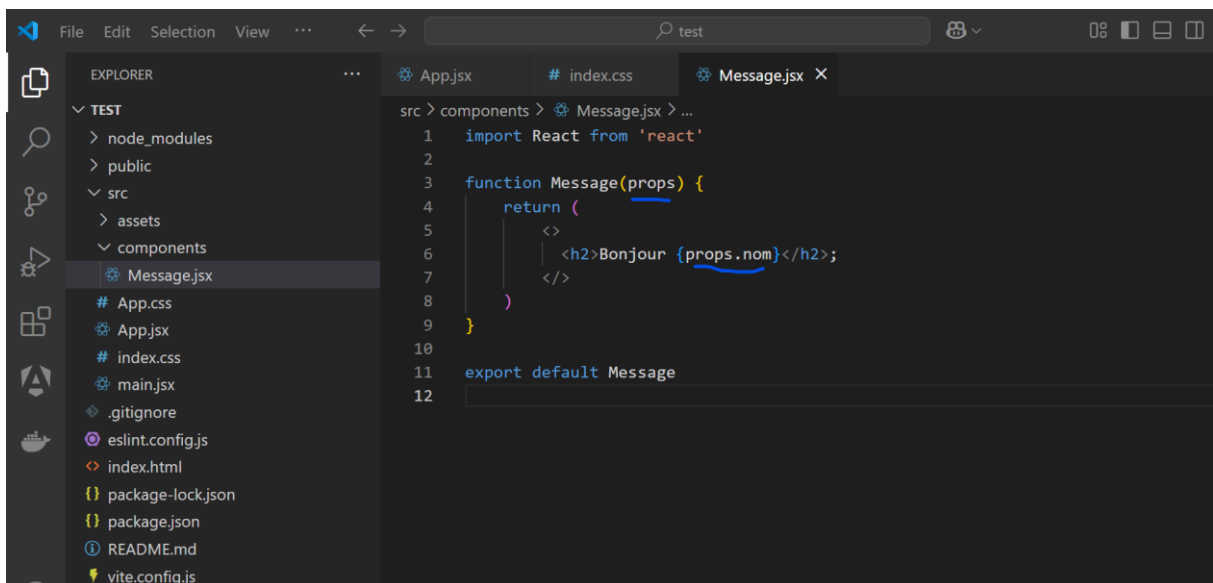
Les **props** (propriétés) permettent de **transmettre des données** d'un parent à un enfant.

◆ Exemple d'un composant avec des props

Modifions `Message.jsx` pour afficher un message personnalisé :

```
function Message(props) {  
  
  return <h2> Bonjour, {props.nom} !</h2>;  
  
}
```

```
export default Message;
```



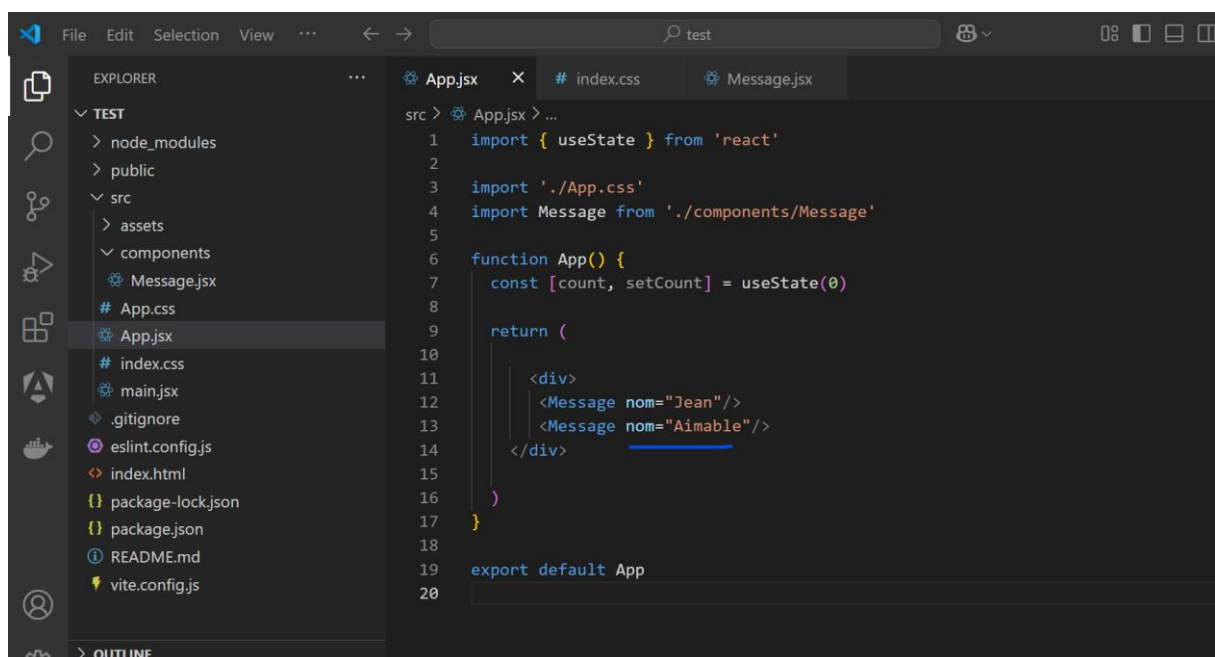
◆ Utilisation avec des valeurs dynamiques

Dans `App.jsx`, passe un nom en **prop** :

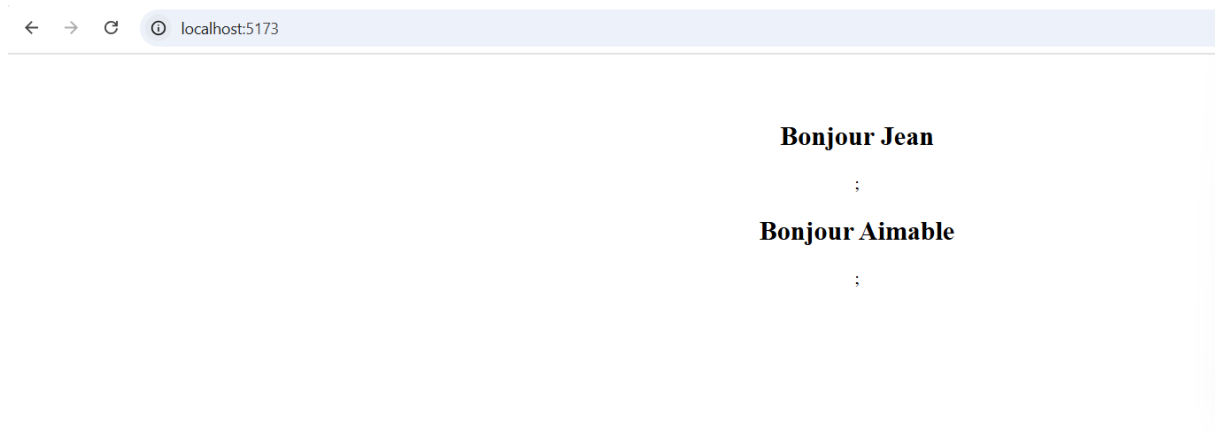
```
import Message from "../components/Message";
```

```
function App() {  
  
  return (  
  
    <div>  
  
      <h1>Mon Application</h1>  
  
      <Message nom="Jean" />  
  
      <Message nom="Aimable" />  
  
    </div>  
  
  );  
}
```

export default App;



✓ Résultat :



4. Composants avec plusieurs props

Un composant peut recevoir **plusieurs props**.

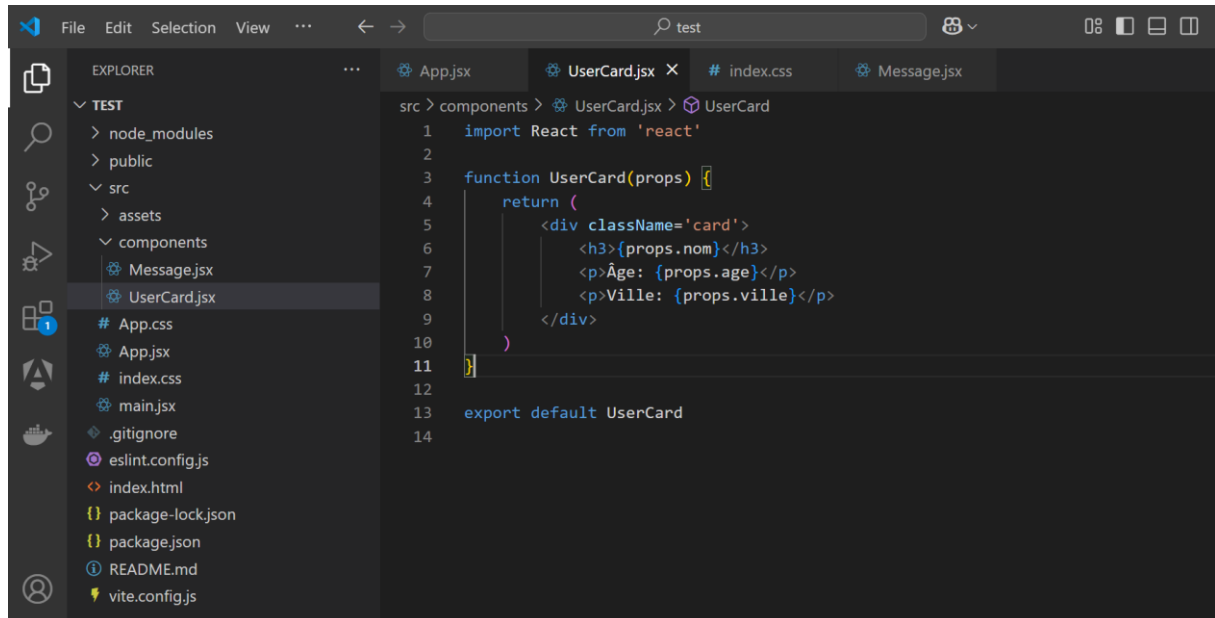
◆ Exemple : Une carte utilisateur

Dans `src/components/UserCard.jsx` :

```
function UserCard(props) {  
  return (  
    <div className="card">  
      <h3>{props.nom}</h3>  
      <p>Âge : {props.age}</p>  
      <p>Ville : {props.ville}</p>  
    </div>  
  );  
}
```

```
}
```

```
export default UserCard;
```



✦ Utilisation dans App.js

```
import UserCard from "../components/UserCard";
```

```
function App() {
```

```
  return (
```

```
    <div>
```

```
      <h1>Cfitech</h1>
```

```
      <UserCard nom="Michel" age={20} ville="Bruxelles" />
```

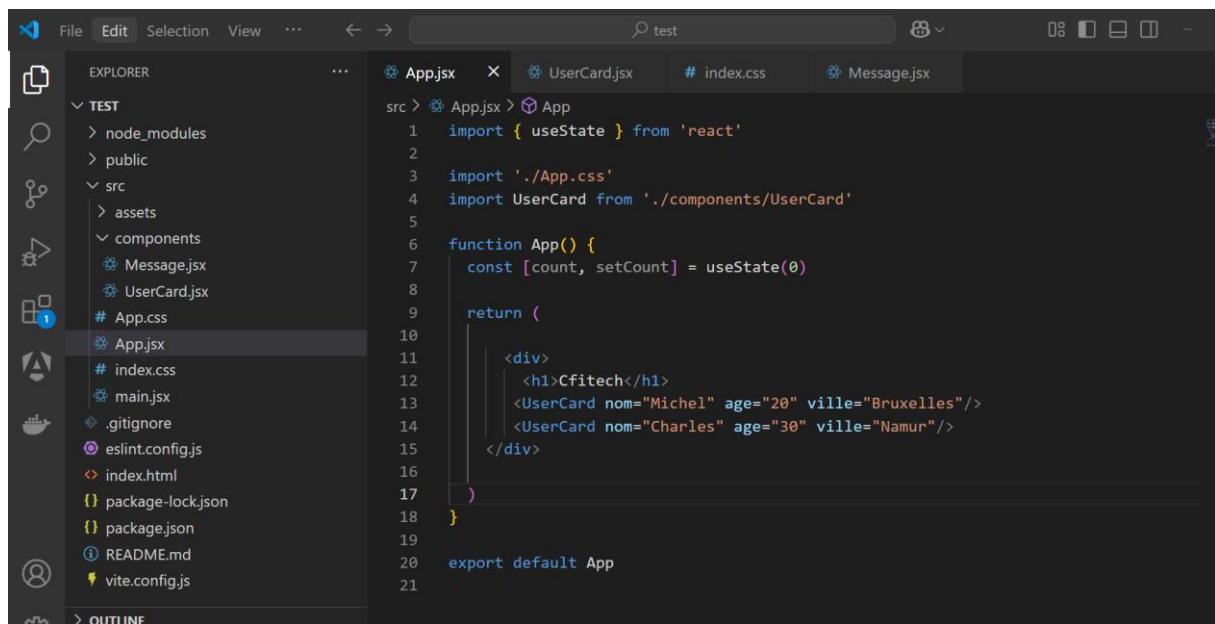
```
      <UserCard nom="Charles" age={30} ville="Namur" />
```

```
    </div>
```

```
  );
```

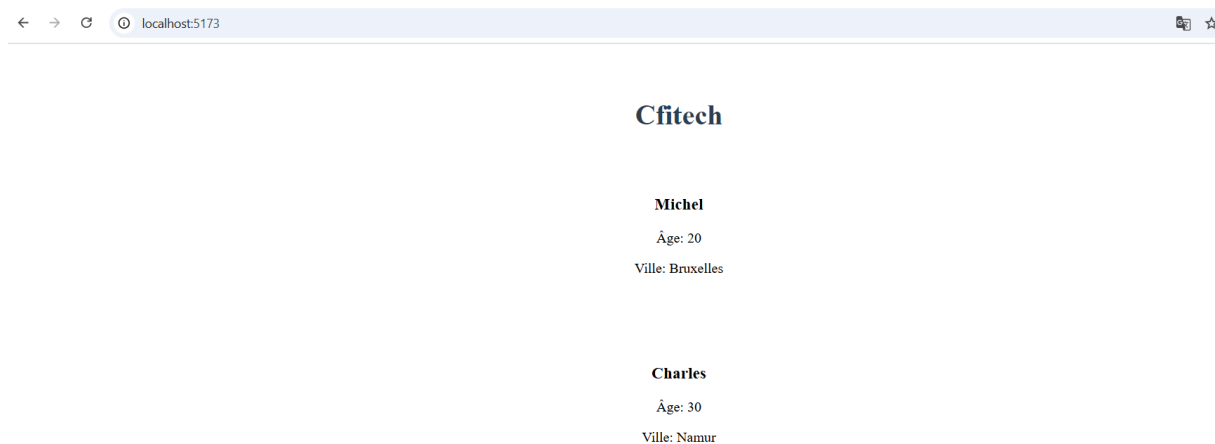
}

export default App;



```
src > App.jsx > App
1  import { useState } from 'react'
2
3  import './App.css'
4  import UserCard from './components/UserCard'
5
6  function App() {
7    const [count, setCount] = useState(0)
8
9    return (
10      <div>
11        <h1>Cfitech</h1>
12        <UserCard nom="Michel" age="20" ville="Bruxelles"/>
13        <UserCard nom="Charles" age="30" ville="Namur"/>
14      </div>
15    )
16  }
17
18  export default App
19
20
21
```

✓ Résultat :



Résumé

- ✓ **Les composants** permettent de découper l'UI en morceaux réutilisables.
- ✓ **Les props** permettent de transmettre des données entre les composants.
- ✓ **Les composants fonctionnels** sont la méthode recommandée en React.

◆ Composants Fonctionnels vs Class Components en React

Dans React, il existe **deux types de composants** :

1. **Les composants fonctionnels** (modernes, plus simples ✓)
2. **Les composants de classe** (ancienne méthode, avant les Hooks ⚠)

Depuis l'introduction des **Hooks** (React 16.8), **les composants fonctionnels sont privilégiés**

1. Composants Fonctionnels (Recommandé ✓)

Un **composant fonctionnel** est une **fonction JavaScript** qui retourne du JSX.
Il est **plus simple, plus lisible et plus performant** que les composants de classe.

Exemple : Composant Fonctionnel

```
function Message(props) {  
  return <h2> Bonjour, {props.nom} !</h2>;  
}
```

```
export default Message;
```

✓ Avantages des composants fonctionnels

- ✓ Syntaxe plus **simple et concise**
- ✓ Meilleure **lisibilité et maintenabilité**
- ✓ **Performance améliorée** (moins de code)
- ✓ Supporte les **Hooks** (`useState`, `useEffect`...)

2. Composants de Classe (Ancienne Méthode [⚠](#))

Avant les Hooks, on utilisait des **classes** pour créer des composants avec un état (*state*). Ils sont **plus lourds et plus complexes** que les composants fonctionnels.

Exemple : Composant de Classe

```
import React, { Component } from "react";

class Message extends Component {

  render() {

    return <h2>Bonjour, {this.props.nom} !</h2>;

  }

}

export default Message;
```

⚠ Inconvénients des composants de classe

- ✗ **Syntaxe plus lourde** (besoin d'utiliser `this.props`, `this.state`, etc.)
- ✗ **Plus difficile à lire et à comprendre**
- ✗ **Moins performant** que les composants fonctionnels

3. Gestion de l'État : `useState` vs `this.state`

Avec un composant fonctionnel (`useState`) ✓

```
import { useState } from "react";
```

```
function Counter() {
```

```
const [count, setCount] = useState(0);

return (
  <div>
    <h2>Compteur : {count}</h2>
    <button onClick={() => setCount(count + 1)}>➕ Augmenter</button>
  </div>
);
}

export default Counter;
```

Avec un composant de classe (this.state) ⚠

```
import React, { Component } from "react";

class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  increment = () => {
```

```

    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <div>
        <h2>Compteur : {this.state.count}</h2>
        <button onClick={this.increment}>+ Augmenter</button>
      </div>
    );
  }
}

export default Counter;

```

✓ **Avec** useState, le code est plus simple et plus court

Conclusion : Quel type de composant utiliser ?

Critère	Composant Fonctionnel 	Composant de Classe 
Simplicité	✓ Très simple	✗ Complexe
Performance	✓ Optimisé	✗ Moins performant
Lisibilité	✓ Facile à lire	✗ Difficile à comprendre
Utilisation des Hooks	✓ Oui (<code>useState</code> , <code>useEffect</code>)	✗ Non
État et Lifecycle	✓ Plus intuitif avec Hooks	✗ <code>this.state</code> et <code>componentDidMount</code> ...

NB : En React, `props` et `useState` sont deux concepts fondamentaux qui servent des objectifs différents :

1. `props` (Propriétés)

- Les `props` sont utilisées pour **transmettre des données** d'un composant parent à un composant enfant.
- Elles sont **immuables** dans le composant enfant (le composant qui les reçoit ne peut pas les modifier directement).
- Elles permettent de rendre les composants **réutilisables** et **dynamiques**.

◆ Exemple :

```
function Enfant({ message }) {  
  
  return <h1>{message}</h1>;  
  
}
```

```
function Parent() {  
  
  return <Enfant message="Bonjour !" />;  
  
}
```

Ici, `message` est une prop passée du Parent au Enfant.

★ 2. `useState` (État local)

- `useState` est un **hook** qui permet à un composant fonctionnel de gérer son propre état interne.
- Contrairement aux `props`, l'état **peut être modifié** par le composant lui-même.
- Les changements d'état provoquent un **re-render** du composant.

◆ Exemple

```
import { useState } from "react";
```

```
function Compteur() {  
  
  const [compte, setCompte] = useState(0);  
  
  
  return (  

```



```

<div>

  <p>Valeur : {compte}</p>

  <button onClick={() => setCompte(compte + 1)}>+1</button>

</div>

);
}

```

Ici, `compte` est un état local qui change lorsqu'on clique sur le bouton.

vs Différences Clés :

Critère	props	useState
Définition	Valeurs passées d'un parent à un enfant	Valeur locale propre au composant
Modifiable ?	❌ Non (immuable)	✅ Oui (avec <code>setState</code>)
Responsabilité	Dépend du parent	Dépend du composant lui-même
Provoque un Re-render ?	❌ Non	✅ Oui, quand mis à jour
Usage principal	Partage de données entre composants	Gestion des données internes du composant

En React, le **rendering** (rendu) est le processus qui permet d'afficher ou de mettre à jour l'interface utilisateur d'un composant.

💡 Comment fonctionne le rendu en React ?

1. **Initial Render (Premier rendu)**
 - Lorsqu'un composant est monté pour la première fois, React exécute sa fonction et retourne un arbre de **JSX** (ou d'éléments React) qui sera converti en HTML et affiché dans le DOM.
2. **Re-render (Mise à jour du rendu)**
 - Un composant se re-render lorsqu'il **reçoit de nouvelles props**, qu'il **met à jour son state**, ou qu'un **parent est mis à jour**.
 - React compare l'ancien et le nouveau rendu et applique uniquement les changements nécessaires (grâce au **Virtual DOM** et au **Diffing Algorithm**).

Quand utiliser quoi ?

✓ **Utiliser `props`** lorsque vous devez **transmettre** des données à un composant enfant sans qu'il ait besoin de les modifier.

✓ **Utiliser `useState`** lorsque le composant doit **gérer et modifier** ses propres données dynamiques.

◆ Exemple combiné `props` + `useState`

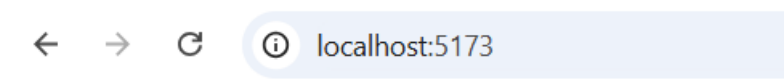
Dans cet exemple, le composant parent transmet une valeur initiale via `props`, et l'enfant peut la modifier avec `useState`.

```
import { useState } from "react";
```

```
function Enfant({ valeurInitiale }) {  
  
  const [compteur, setCompteur] = useState(valeurInitiale);  
  
  return (  
    <div>  
      <p>Compteur : {compteur}</p>  
      <button onClick={() => setCompteur(compteur + 1)}>+1</button>  
    </div>  
  );  
}
```

```
function Parent() {  
  
  return <Enfant valeurInitiale={5} />;  
}
```

```
export default Parent;
```



Compteur : 5



Explication :

1. Le **parent (Parent)** passe `valeurInitiale={5}` en **prop** au composant enfant.
2. L'**enfant (Enfant)** reçoit cette prop et l'utilise comme valeur initiale pour son `useState`.
3. Quand on clique sur le bouton, l'état local `compteur` change sans affecter `valeurInitiale` dans `Parent`

✓ Ainsi, **props** est utilisé pour passer une donnée initiale, et `useState` permet de la modifier localement !

◆ Outils de Développement React (React DevTools)

Lorsqu'on développe une application React, il est essentiel d'avoir les **bons outils** pour **déboguer**, **analyser** et **optimiser** notre code. L'un des meilleurs outils pour ça est

React DevTools.

1.Qu'est-ce que React DevTools ?

React DevTools est une **extension pour Chrome et Firefox** qui permet de :

- ✓ **Explorer la structure des composants**
- ✓ **Voir et modifier les props et le state en direct**
- ✓ **Analyser les performances des composants**
- ✓ **Déboguer plus facilement les applications React**

2.Installation de React DevTools

Option 1 : Installer l'extension navigateur (recommandé)

◆ **Chrome** : React Developer Tools - Chrome Web Store

◆ **Firefox** : [React Developer Tools - Add-ons for Firefox](#)

Une fois installée, l'onglet "**Components**" et "**Profiler**" apparaîtront dans les **Outils de développement** (F12 ou Ctrl + Shift + I).

Pratique : Création d'un Mini-Projet avec des Composants Réutilisables

Objectif du projet : Une Liste de Cartes Utilisateurs

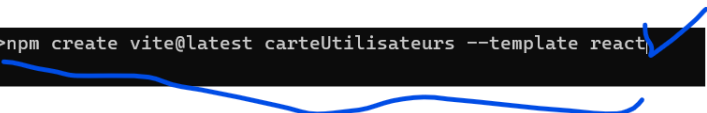
Nous allons créer une **application simple** qui affiche une liste d'utilisateurs sous forme de cartes. Chaque carte affichera le **nom**, **l'âge** et **la ville** d'un utilisateur.

Concepts abordés :

- ✓ Création de **composants réutilisables**
- ✓ Passage de **props**
- ✓ Organisation du projet **modulaire**

Étape 1 : Créer un projet React avec Vite

```
D:\CFITECH\React\cours react 2025\exercices>npm create vite@latest carteUtilisateurs --template react
```



Puis :

```
cd carteUtilisateurs
```

```
npm install
```

```
npm run dev
```

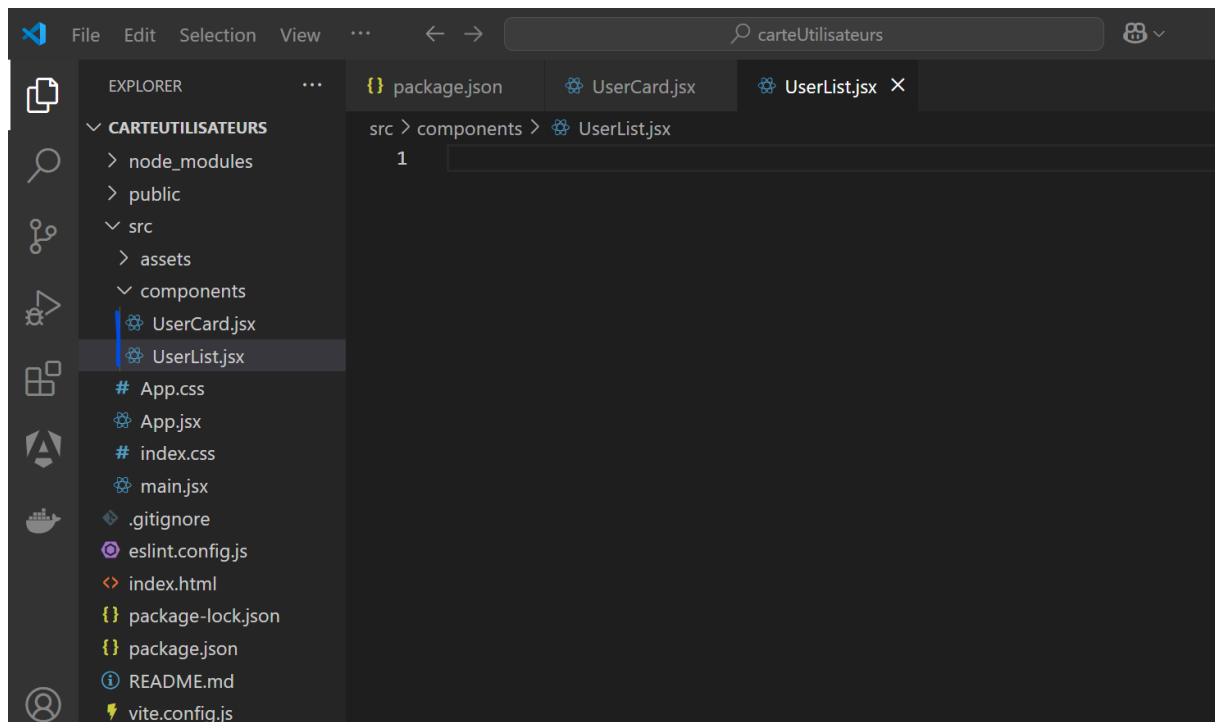
Ouvre <http://localhost:5173> dans ton navigateur.

Étape 2 : Organiser les dossiers

Nous allons organiser notre projet comme suit :

```
carteUtilisateurs/
```

```
| — src/
|   | — components/    # Dossier pour les composants réutilisables
|   |   | — UserCard.jsx # Composant pour afficher un utilisateur
|   |   | — UserList.jsx # Composant pour afficher la liste des utilisateurs
|   | — App.jsx        # Composant principal
|   | — main.jsx       # Point d'entrée de l'application
|   | — App.css        # Styles globaux
```

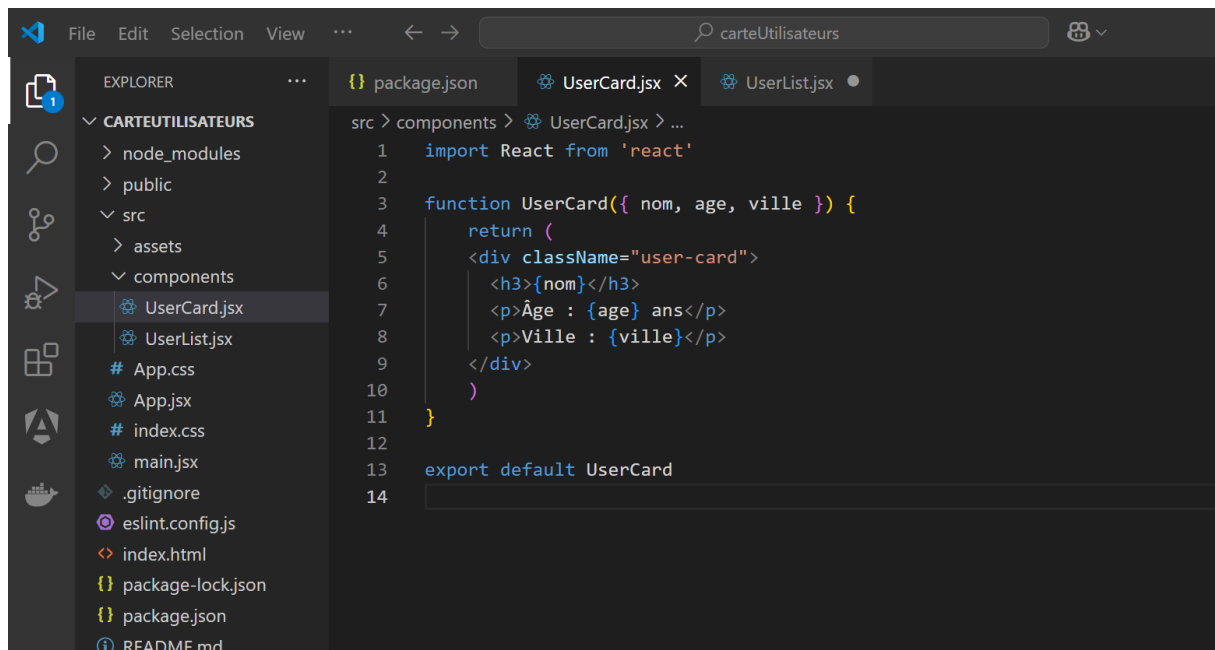


Étape 3 : Création du composant `UserCard.jsx`

Dans `src/components/UserCard.jsx`, ajoute le code suivant :

```
function UserCard({ nom, age, ville }) {  
  
  return (  
  
    <div className="user-card">  
  
      <h3>{nom}</h3>  
  
      <p>Âge : {age} ans</p>  
  
      <p>Ville : {ville}</p>  
  
    </div>  
  
  );  
}
```

```
export default UserCard;
```



✓ Ce composant **UserCard** prend trois props (nom, age, ville) et affiche une carte utilisateur.

Étape 4 : Création du composant `UserList.jsx`

Dans `src/components/UserList.jsx`, ajoute le code suivant :

```
import UserCard from "../UserCard";
```

```
function UserList() {  
  
  const utilisateurs = [  
  
    { id: 1, nom: "Alice", age: 25, ville: "Paris" },  
  
    { id: 2, nom: "Bob", age: 30, ville: "Lyon" },  
  
    { id: 3, nom: "Charlie", age: 22, ville: "Marseille" }  
  
  ];
```

```
  return (  
  
    <div className="user-list">  
  
      <h2>Liste des Utilisateurs</h2>
```

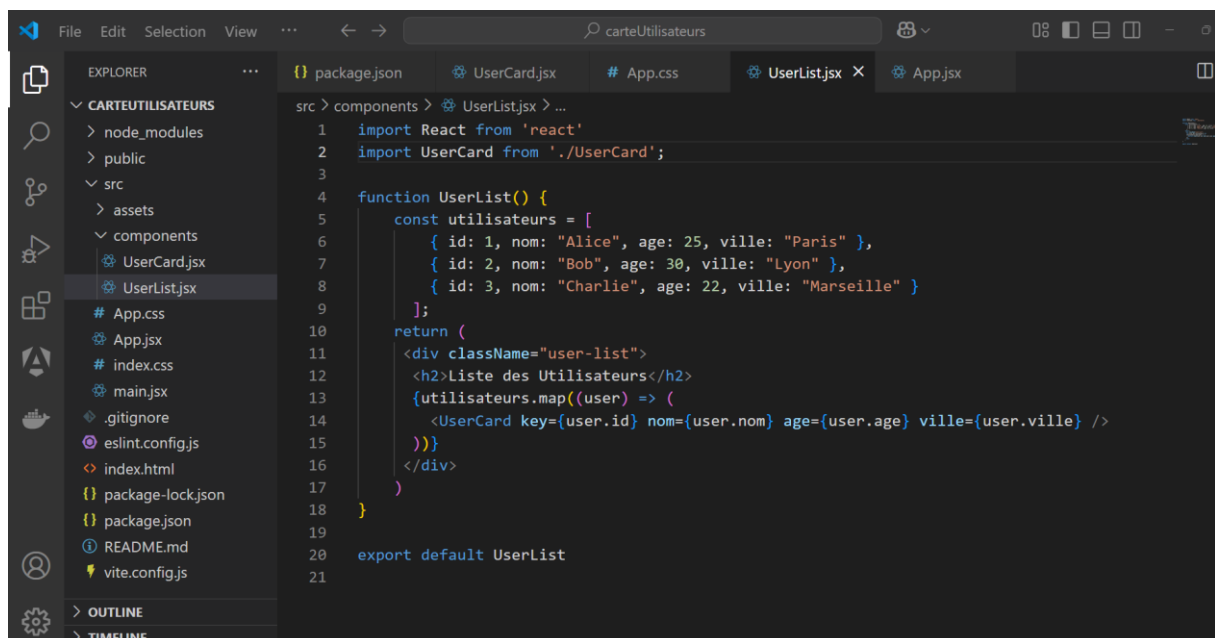
```

    {utilisateurs.map((user) => (
        <UserCard key={user.id} nom={user.nom} age={user.age} ville={user.ville} />
    ))}
</div>

);
}

```

export default UserList;



Étape 5 : Utilisation dans App.jsx

Modifie `src/App.jsx` pour intégrer `UserList` :

```
import UserList from "./components/UserList";
```

```
import "./App.css";
```

```
function App() {
```

```
    return (
```



```
<div className="app">
```

```
<h1> Mon Mini-Projet </h1>
```

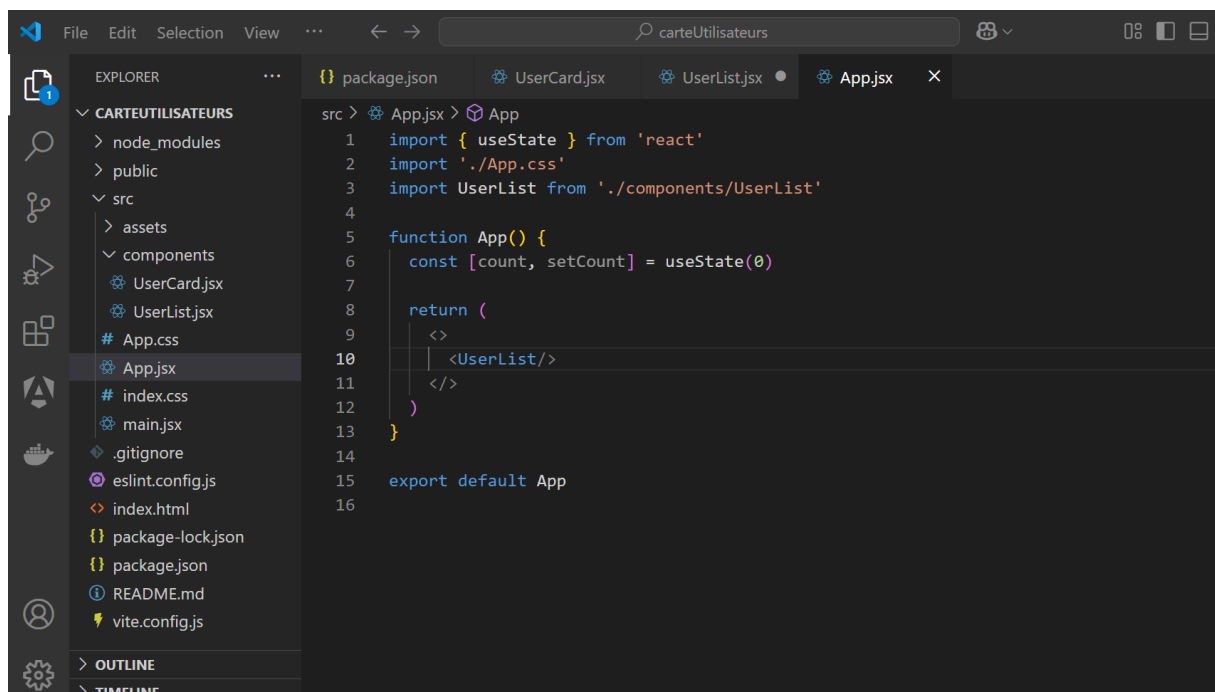
```
<UserList />
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```



Étape 6 : Ajouter du Style avec CSS

Ajoute ces styles dans `src/App.css` :

```
.app {  
  text-align: center;  
  font-family: Arial, sans-serif;  
  margin-top: 20px;  
}  
  
.user-list {  
  display: flex;  
  justify-content: center;  
  gap: 20px;  
  flex-wrap: wrap;  
  margin-top: 20px;  
}  
  
.user-card {  
  background: white;  
  padding: 15px;  
  border-radius: 10px;  
  box-shadow: 2px 2px 10px rgba(0, 0, 0, 0.1);  
  width: 200px;  
  text-align: center;  
}
```

```
h3 {  
  
  color: #2c3e50;  
  
}
```

```
p {  
  
  margin: 5px 0;  
  
  color: #555;  
  
}
```

Résultat Final

Une page affichant plusieurs **cartes utilisateurs** avec un **design propre et réutilisable**.



Liste des Utilisateurs

Alice	Bob	Charlie
Âge : 25 ans Ville : Paris	Âge : 30 ans Ville : Lyon	Âge : 22 ans Ville : Marseille

◆ Chapitre 3 : Gestion d'État avec useState

◆ Introduction au State

Le **state** en React est une **source de données locale** spécifique à un composant, qui permet de gérer les **données dynamiques** et de **réagir aux changements**.
Il est essentiel pour rendre une interface interactive !

1. Qu'est-ce que le State ?

- Le **state** contient des informations qu'un composant peut **modifier au fil du temps**.
- Contrairement aux **props**, le **state est interne** au composant (privé) et ne peut pas être directement modifié par un parent.

2. Exemple concret du State

Imagine un bouton "Like". À chaque clic, le nombre de likes augmente. Cette donnée (le nombre de likes) est gérée via le **state**

3. Gestion du State avec useState

◆ useState : Le Hook pour gérer le State

La fonction **useState** permet de :

1. **Définir une valeur initiale** du state.
2. **Mettre à jour le state** quand un changement est nécessaire.

Syntaxe de useState

```
const [state, setState] = useState(valeurInitiale);
```

- **state** : La valeur actuelle du state.
- **setState** : Une fonction pour **mettre à jour le state**.
- **valeurInitiale** : **La valeur initiale du state**.

4. Exemple Pratique : Compteur

Créons un **bouton compteur** où le nombre affiché augmente à chaque clic.

```
import { useState } from "react";
```

```
function Compteur() {
```

```
  const [count, setCount] = useState(0); // Valeur initiale : 0
```

```
  return (
```

```
    <div>
```

```
      <h2>Compteur : {count}</h2>
```

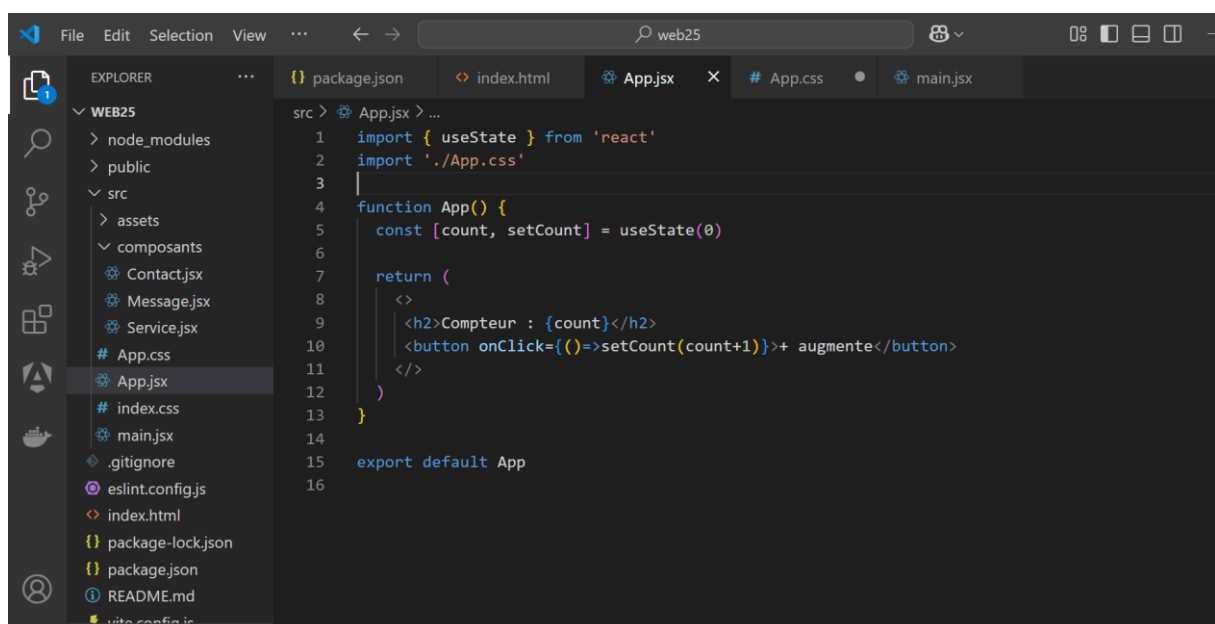
```
      <button onClick={() => setCount(count + 1)}>+ Augmenter</button>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default Compteur;
```



Ce qui se passe :

1. `useState(0)` initialise la variable `count` avec la valeur 0.
2. À chaque clic sur le bouton, la fonction `setCount` met à jour `count`.
3. Le composant se re-render avec la nouvelle valeur.

5. Plusieurs États dans un Composant

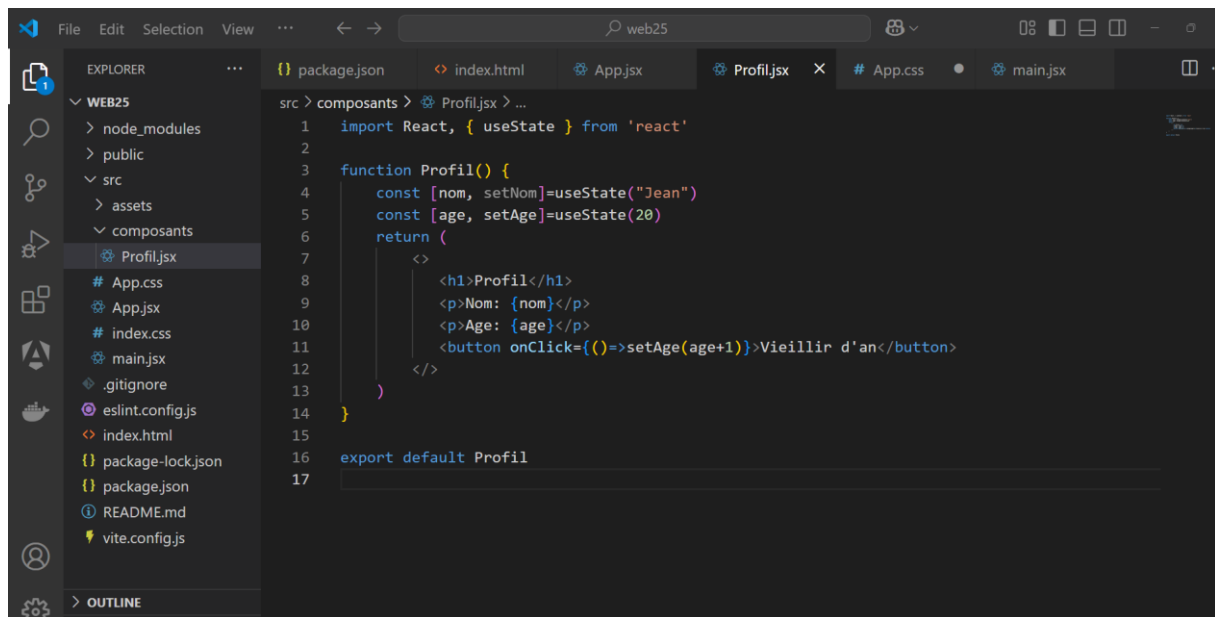
Un composant peut avoir plusieurs états, chacun géré séparément avec `useState`.

Exemple : Gestion de plusieurs champs

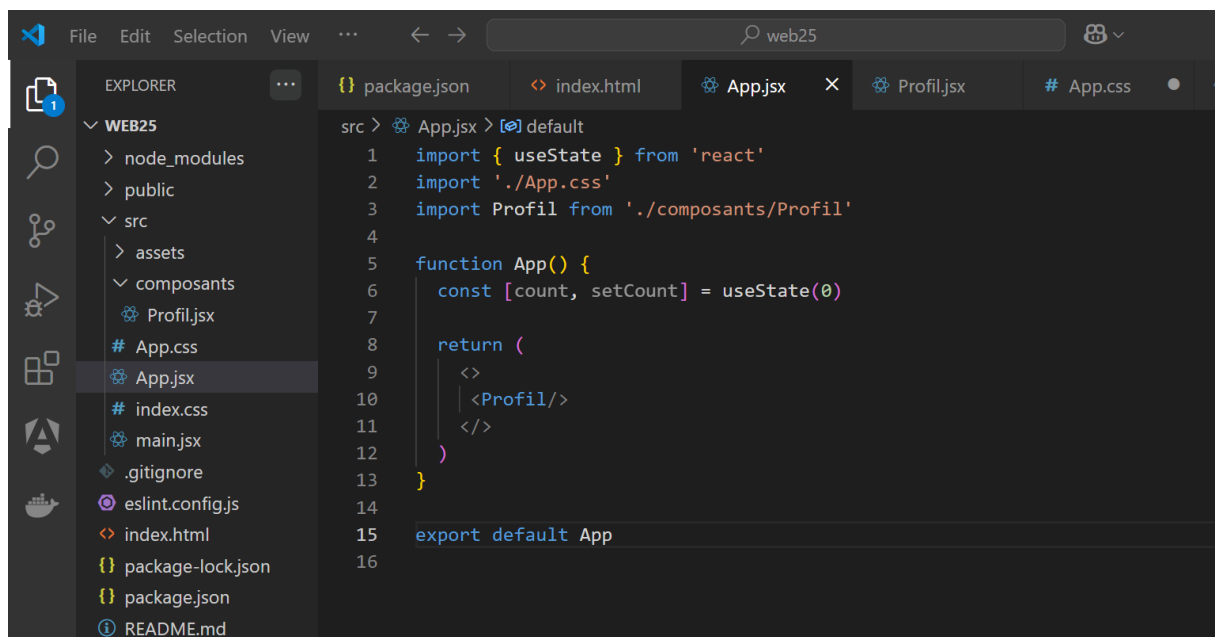
```
import { useState } from "react";
```

```
function Profil() {  
  
  const [nom, setNom] = useState("Alice");  
  
  const [age, setAge] = useState(25);  
  
  return (  
  
    <div>  
  
      <h2>Profil :</h2>  
  
      <p>Nom : {nom}</p>  
  
      <p>Âge : {age}</p>  
  
      <button onClick={() => setAge(age + 1)}> Vieillir d'un an</button>  
  
    </div>  
  
  );  
}
```

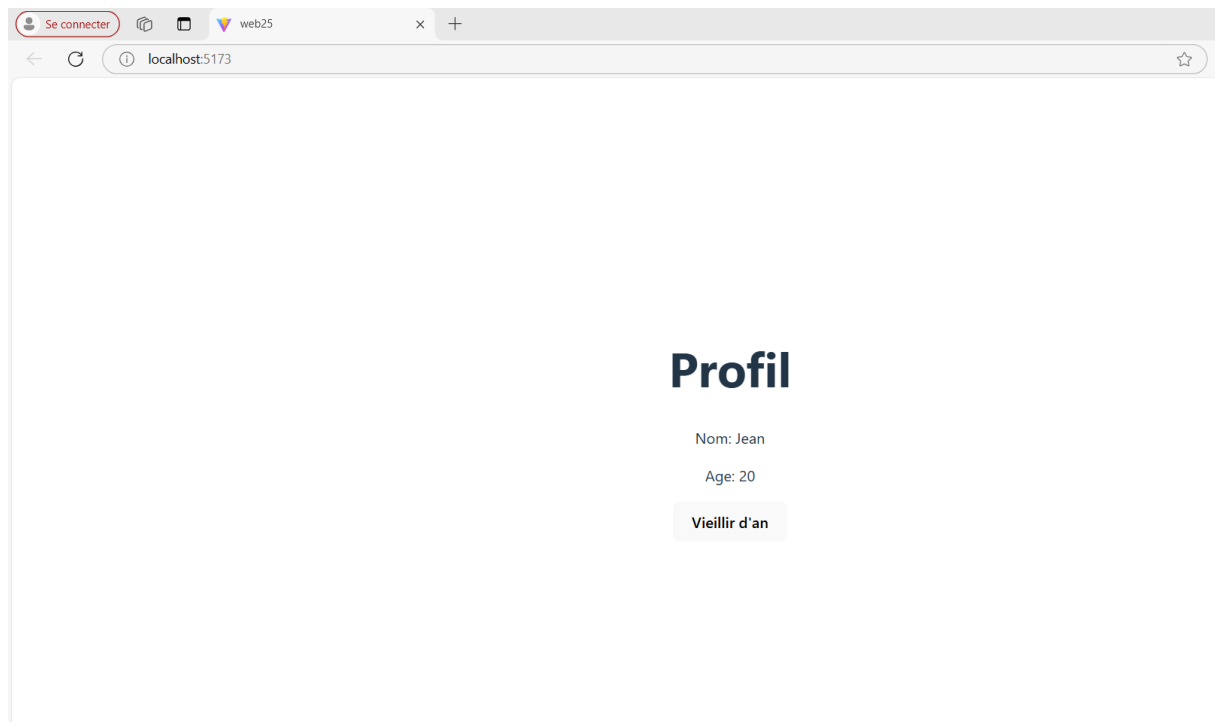
```
export default Profil;
```



```
1 import React, { useState } from 'react'
2
3 function Profil() {
4   const [nom, setNom]=useState("Jean")
5   const [age, setAge]=useState(20)
6   return (
7     <>
8       <h1>Profil</h1>
9       <p>Nom: {nom}</p>
10      <p>Age: {age}</p>
11      <button onClick={()=>setAge(age+1)}>Vieillir d'an</button>
12    </>
13  )
14 }
15
16 export default Profil
17
```



```
1 import { useState } from 'react'
2 import './App.css'
3 import Profil from './composants/Profil'
4
5 function App() {
6   const [count, setCount] = useState(0)
7
8   return (
9     <>
10      <Profil/>
11    </>
12  )
13 }
14
15 export default App
16
```



Ce qui se passe :

1. Deux états sont gérés : `nom` et `age`.
2. La fonction `setAge` permet de modifier uniquement l'âge, sans affecter le nom.

Exemple Avancé : Gérer une Input Dynamique

Ajoutons un champ de texte dont la valeur est contrôlée via le state.

```
import { useState } from "react";
```

```
function SaisieNom() {
```

```
  const [nom, setNom] = useState(""); // Initialisation à une chaîne vide
```

```
  const handleChange = (e) => {
```

```
    setNom(e.target.value); // Mise à jour avec la valeur saisie
```



```
};
```

```
return (
```

```
<div>
```

```
<h2>Bonjour, {nom || "Utilisateur"} !</h2>
```

```
<input
```

```
  type="text"
```

```
  placeholder="Entrez votre nom"
```

```
  value={nom}
```

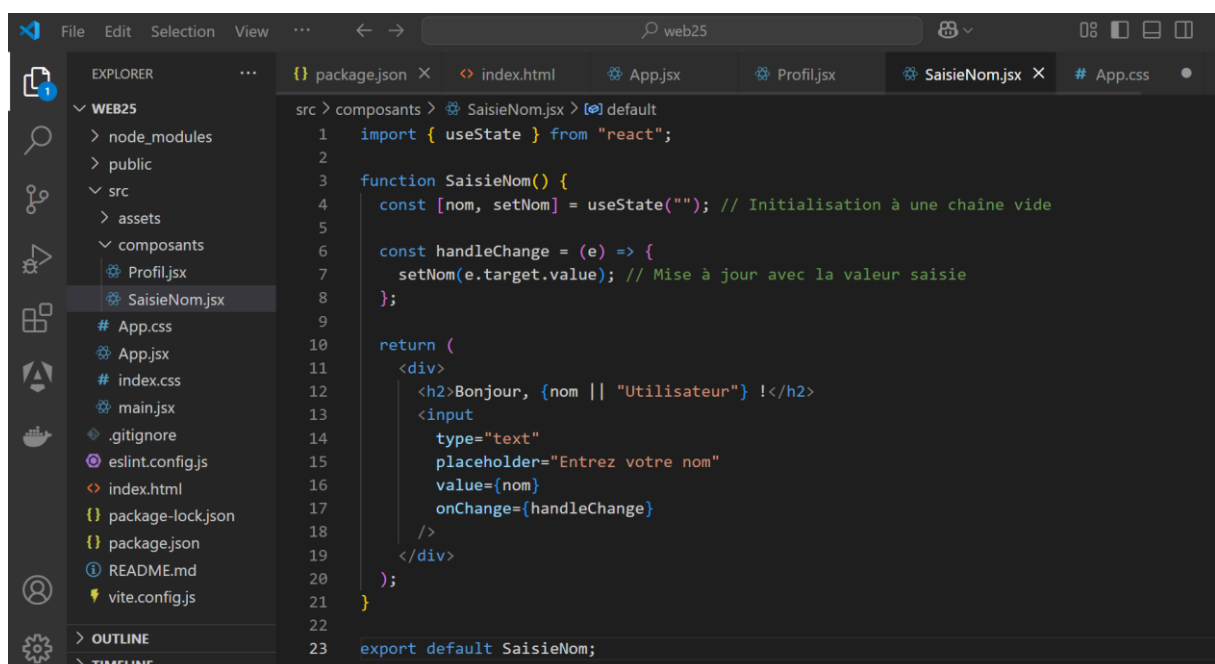
```
  onChange={handleChange}
```

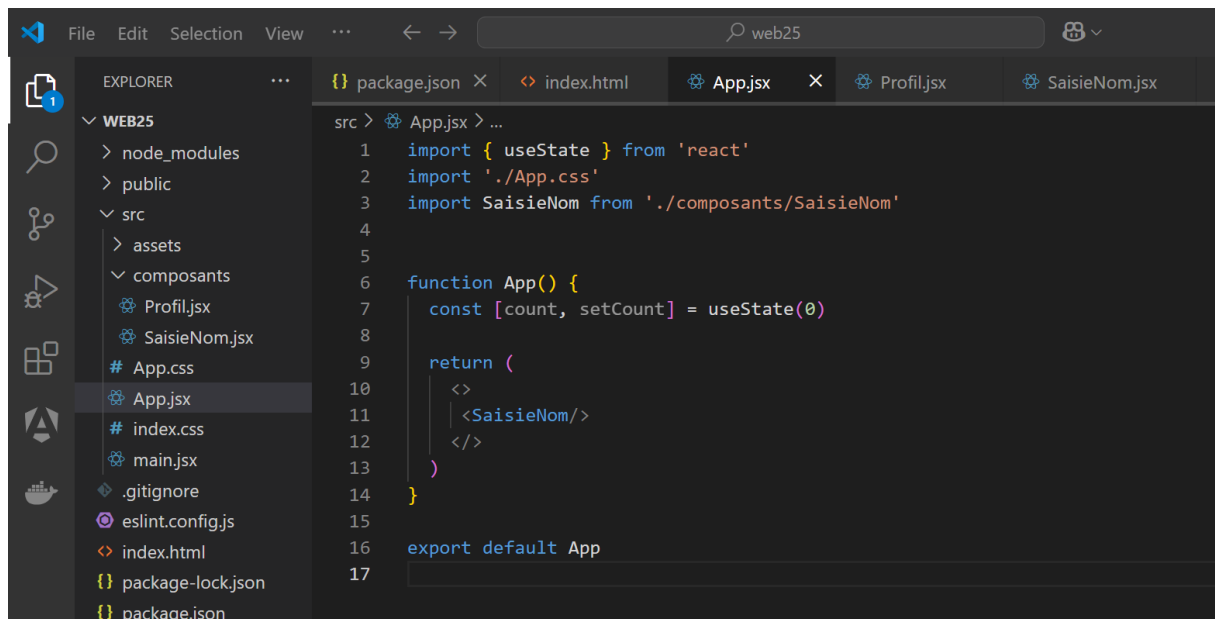
```
</div>
```

```
);
```

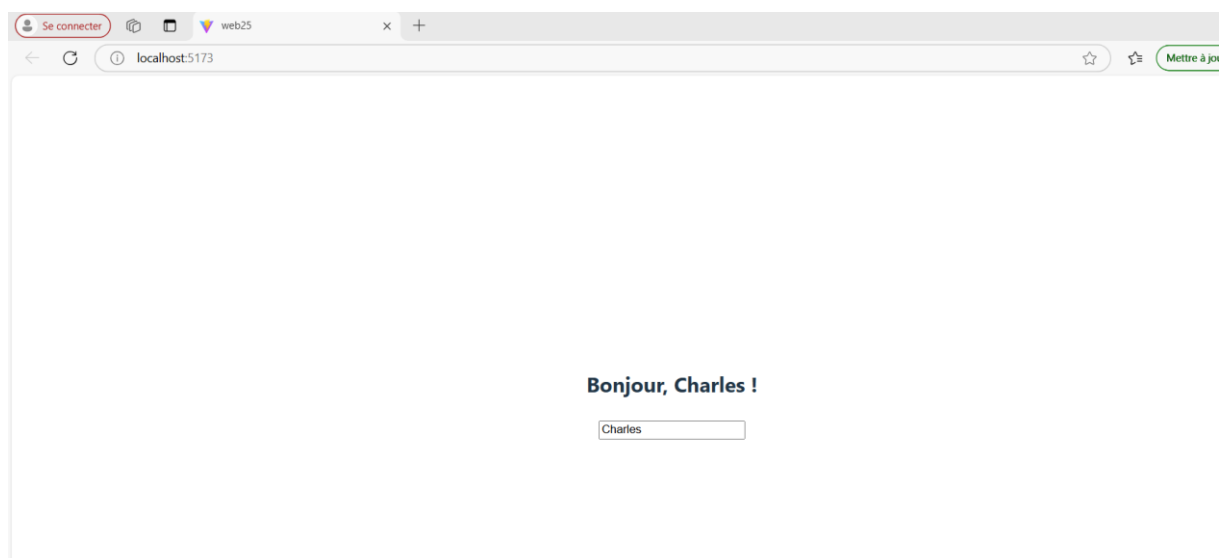
```
}
```

```
export default SaisieNom;
```





```
src > App.jsx > ...
1  import { useState } from 'react'
2  import './App.css'
3  import SaisieNom from './composants/SaisieNom'
4
5
6  function App() {
7    const [count, setCount] = useState(0)
8
9    return (
10     <>
11       <SaisieNom/>
12     </>
13   )
14 }
15
16 export default App
17
```



Explications :

1. **Initialisation** : `nom` est une chaîne vide par défaut.
2. **Événement `onChange`** : La fonction `handleChange` met à jour `nom` à chaque frappe de l'utilisateur.
3. **Affichage dynamique** : Le composant affiche le nom en direct.

Gestion du State avec des Objets ou des Tableaux

Le state peut contenir des **objets** ou des **tableaux**

Exemple : State avec un Objet

```
import { useState } from "react";
```

```
function ProfilUtilisateur() {
```

```
  const [profil, setProfil] = useState({
```

```
    nom: "Alice",
```

```
    age: 25,
```

```
  });
```

```
  const incrementerAge = () => {
```

```
    setProfil({ ...profil, age: profil.age + 1 }); // Copie l'ancien état et met à jour `age`
```

```
  };
```

```
  return (
```

```
    <div>
```

```
      <h2>Nom : {profil.nom}</h2>
```

```
      <p>Âge : {profil.age}</p>
```

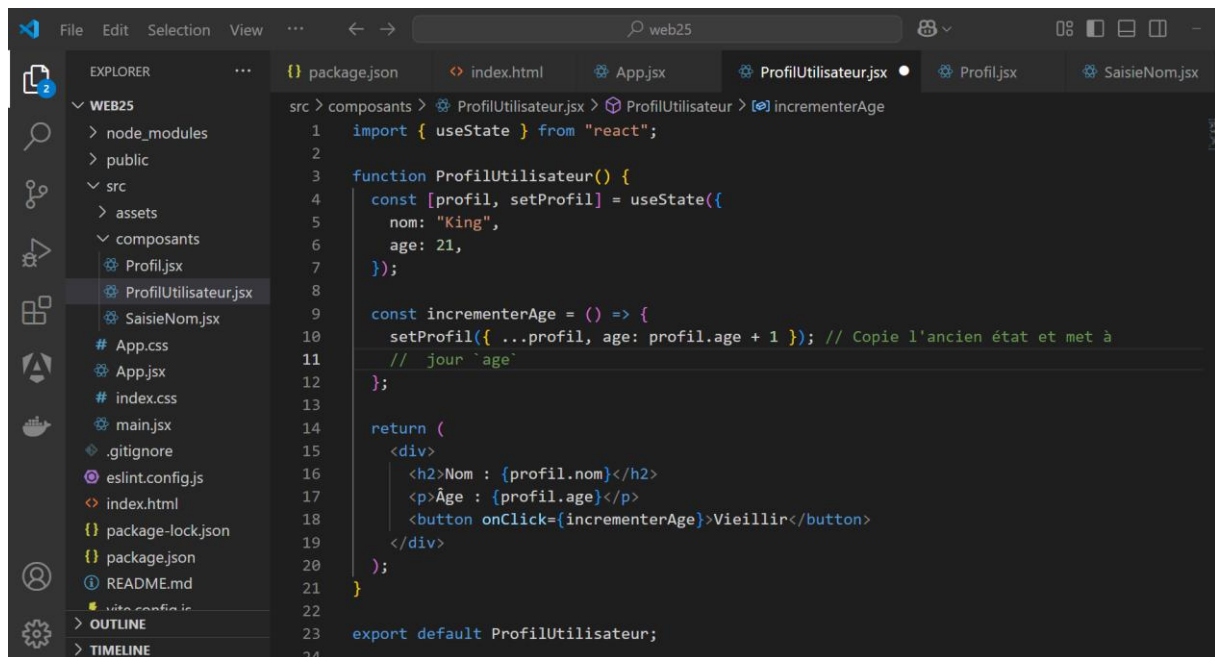
```
      <button onClick={incrementerAge}>Vieillir</button>
```

```
    </div>
```

```
  );
```

```
}
```

export default ProfilUtilisateur;



Explications :

- **Spread operator (...)** : On copie l'état existant (`profil`) pour éviter de le remplacer entièrement.

Exemple : State avec un Tableau

```
import { useState } from "react";
```

```
function ListeTâches() {
```

```
  const [taches, setTaches] = useState(["Acheter du pain", "Lire un livre"]);
```

```
  const ajouterTache = () => {
```

```
    setTaches([...taches, "Nouvelle tâche"]); // Ajout d'une nouvelle tâche
```

```
  };
```

```

return (

  <div>

    <h2>Liste des Tâches :</h2>

    <ul>

      {taches.map((tache, index) => (

        <li key={index}>{tache}</li>

      ))}

    </ul>

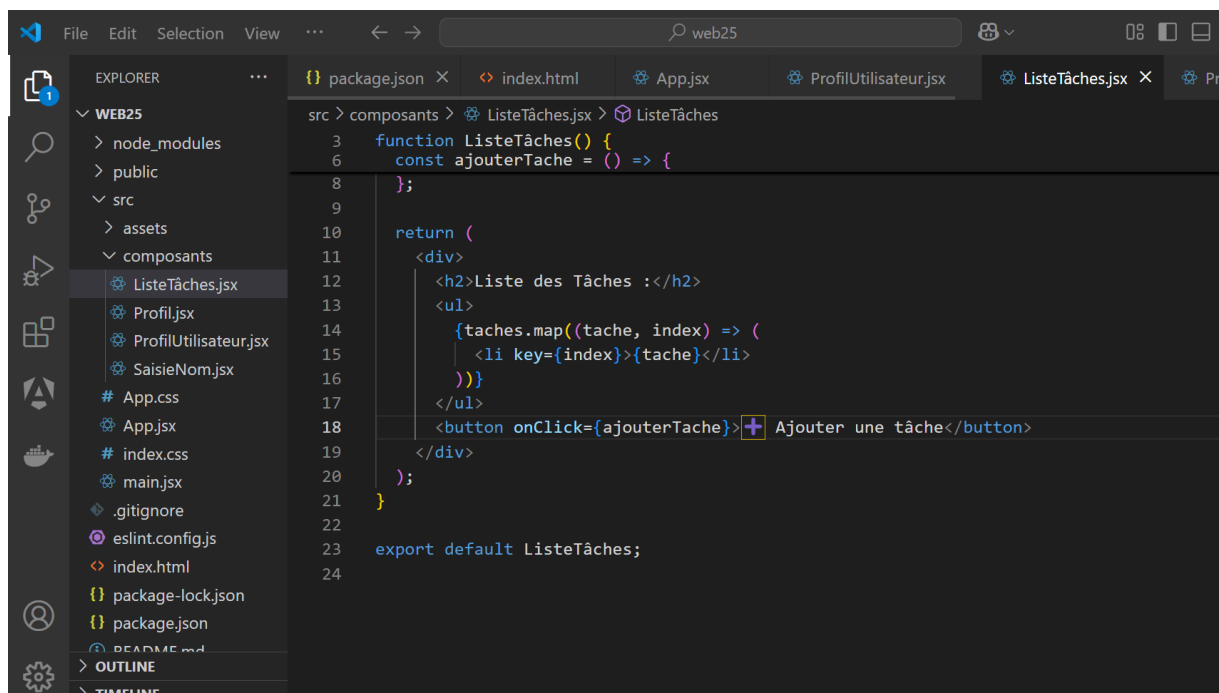
    <button onClick={ajouterTache}>+ Ajouter une tâche</button>

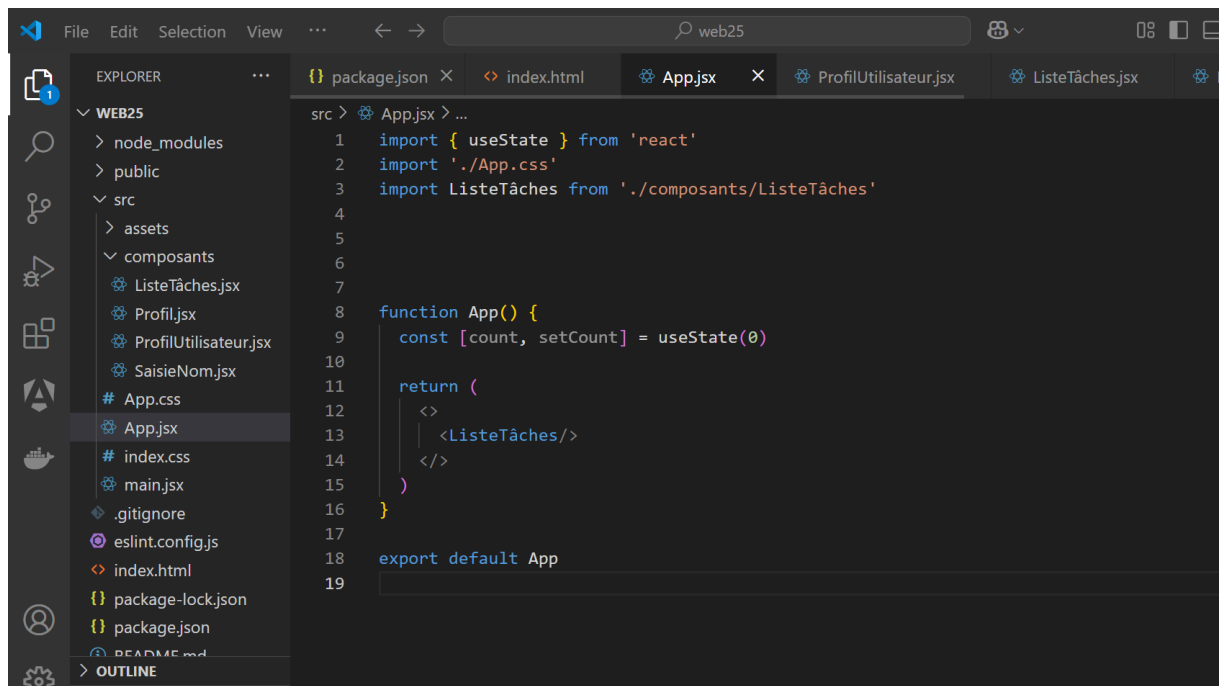
  </div>

);
}

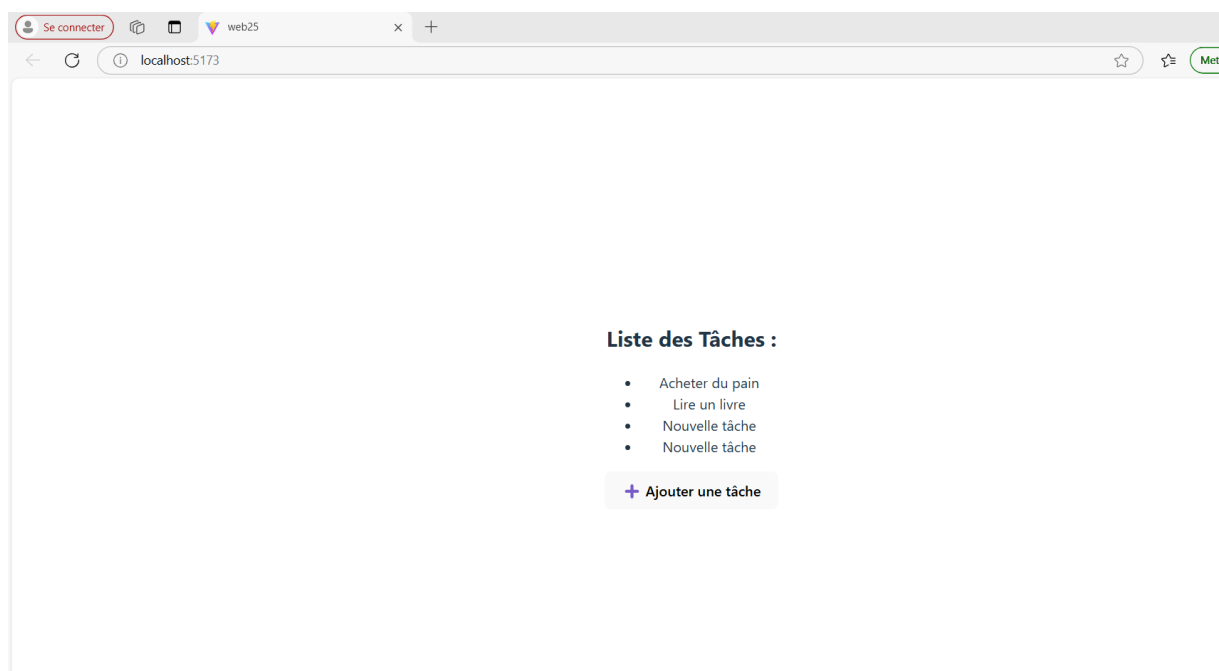
```

export default ListeTâches;





```
1 import { useState } from 'react'
2 import './App.css'
3 import ListeTâches from './composants/ListeTâches'
4
5
6
7
8 function App() {
9   const [count, setCount] = useState(0)
10
11   return (
12     <>
13       <ListeTâches/>
14     </>
15   )
16 }
17
18 export default App
19
```



Explications :

- Immutabilité** : On utilise `[...taches, "Nouvelle tâche"]` pour ajouter une tâche sans modifier directement le tableau existant.

Exercices Pratiques

Exercice 1 : Gestion de compteur

Crée un composant avec deux boutons :

- Un bouton pour **augmenter** le compteur.
- Un bouton pour **diminuer** le compteur.

Rep :

```
import { useState } from "react";

function Compteur() {

  // Initialisation du state avec une valeur de départ de 0

  const [count, setCount] = useState(0);


  // Fonction pour augmenter le compteur

  const augmenter = () => {

    setCount(count + 1);

  };


  // Fonction pour diminuer le compteur

  const diminuer = () => {

    setCount(count - 1);

  };


  return (

    <div style={{ textAlign: "center", marginTop: "20px" }}>
```

```
<h2>Compteur : {count}</h2>
```

```
<button
```

```
  onClick={augmenter}
```

```
  style={{
```

```
    marginRight: "10px",
```

```
    padding: "10px 20px",
```

```
    background: "#4caf50",
```

```
    color: "white",
```

```
    border: "none",
```

```
    borderRadius: "5px",
```

```
    cursor: "pointer",
```

```
  }}  
>
```

```
  + Augmenter
```

```
</button>
```

```
<button
```

```
  onClick={diminuer}
```

```
  style={{
```

```
    padding: "10px 20px",
```

```
    background: "#f44336",
```

```
    color: "white",
```

```
    border: "none",
```

```
    borderRadius: "5px",
```

```
    cursor: "pointer",
```

```
  }}  
>
```


>

— Diminuer

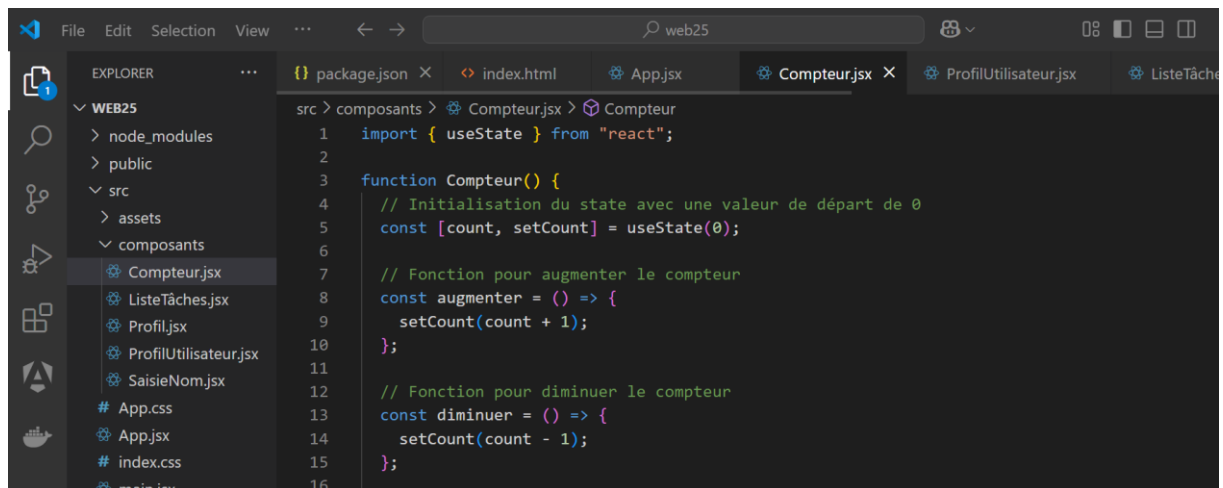
</button>

</div>

);

}

export default Compteur;



The screenshot shows the Visual Studio Code editor with the 'Compteur.js' file open. The Explorer sidebar on the left shows the project structure with 'src > composants > Compteur.js' selected. The main editor area displays the following code:

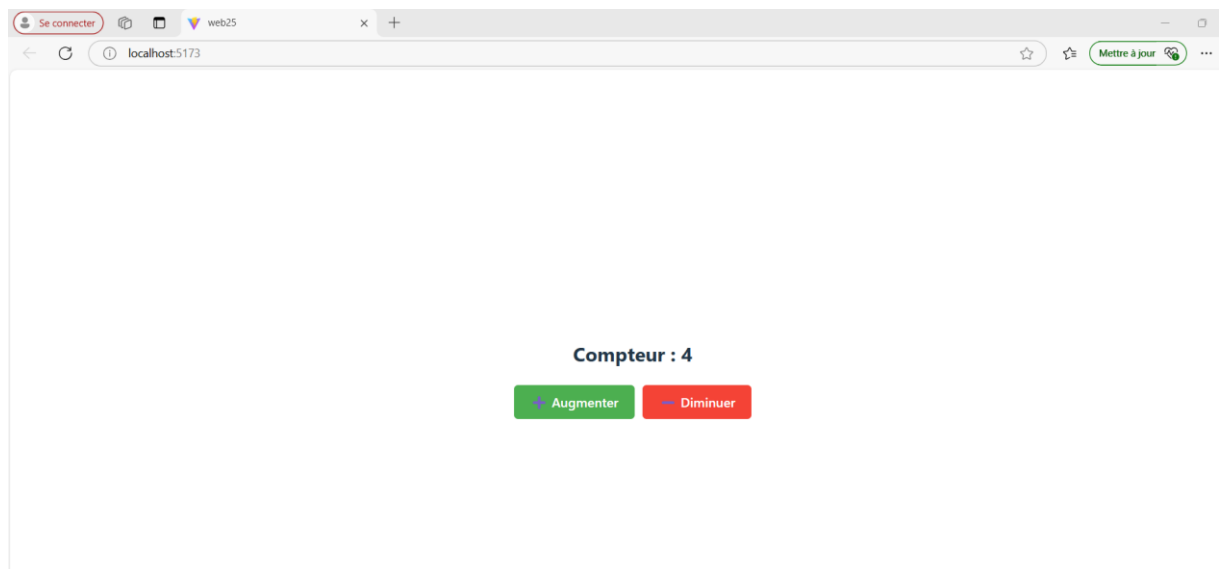
```
1 import { useState } from "react";
2
3 function Compteur() {
4   // Initialisation du state avec une valeur de départ de 0
5   const [count, setCount] = useState(0);
6
7   // Fonction pour augmenter le compteur
8   const augmenter = () => {
9     setCount(count + 1);
10  };
11
12  // Fonction pour diminuer le compteur
13  const diminuer = () => {
14    setCount(count - 1);
15  };
16
```



This screenshot continues the code from the previous one, showing the return statement of the 'Compteur' function. It renders a div with centered text and a button:

```
17
18 return (
19   <div style={{ textAlign: "center", marginTop: "20px" }}>
20     <h2>Compteur : {count}</h2>
21     <button
22       onClick={augmenter}
23       style={{
24         marginRight: "10px",
25         padding: "10px 20px",
26         background: "#4caf50",
27         color: "white",
28         border: "none",
29         borderRadius: "5px",
30         cursor: "pointer",
31       }}
32     />
33   </div>
34 );
```

```
32      + Augmenter
33    </button>
34    <button
35      onClick={diminuer}
36      style={{
37        padding: "10px 20px",
38        background: "#f44336",
39        color: "white",
40        border: "none",
41        borderRadius: "5px",
42        cursor: "pointer",
43      }}
44    >
45      - Diminuer
46    </button>
47  </div>
48  );
49  }
50
51  export default Compteur;
52
```



◆ Chapitre 4 : Effets et Hooks Avancés , Gestion des Événements et Formulaires

6. Les Règles Importantes de useState

- ✓ Le **state** est une donnée locale et dynamique gérée par un composant.
- ✓ Le **Hook useState** est l'outil principal pour créer et modifier le state.
- ✓ Les **misés à jour du state** déclenchent un **nouveau rendu** du composant.

◆ Chapitre 4 : Effets et Hooks Avancés, Gestion des Événements et Formulaires

◆ Introduction aux Hooks (useEffect, useRef, useMemo)

Nous allons explorer des **hooks avancés** pour des cas d'utilisation spécifiques. Ces hooks permettent de gérer les **effets de bord**, d'optimiser les performances et de manipuler des références directement dans le DOM.

1. Hook useEffect : Gestion des Effets de Bord

Quand utiliser useEffect ?

`useEffect` est utilisé pour gérer des **effets de bord** comme :

- Appels API,
- Mise à jour du DOM,
- Gestion des abonnements ou des timers.

Dans la programmation : Les timers sont souvent utilisés pour exécuter une tâche après un certain délai ou à intervalles réguliers.

Syntaxe de base :

```
useEffect(() => {  
  
  // Code à exécuter  
  
  return () => {  
  
    // Code pour nettoyer les effets (facultatif)  
  
  };  
  
}, [dépendances]);
```

Exemple : Affichage dynamique d'un titre de page

```
import { useState, useEffect } from "react";  
  
function TitreDynamique() {  
  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
  
    document.title = `Vous avez cliqué ${count} fois`;  
  
  
    // Nettoyage des effets (facultatif ici)  
  
    return () => {  
  
      console.log("Effet nettoyé");  
  
    };  
  
  }, [count]); // Dépendance : l'effet s'exécute à chaque changement de `count`  
  
  return (  
  
    <div>  
  
      <h2>Compteur : {count}</h2>  
  
      <button onClick={() => setCount(count + 1)}>Cliquez-moi</button>
```

```

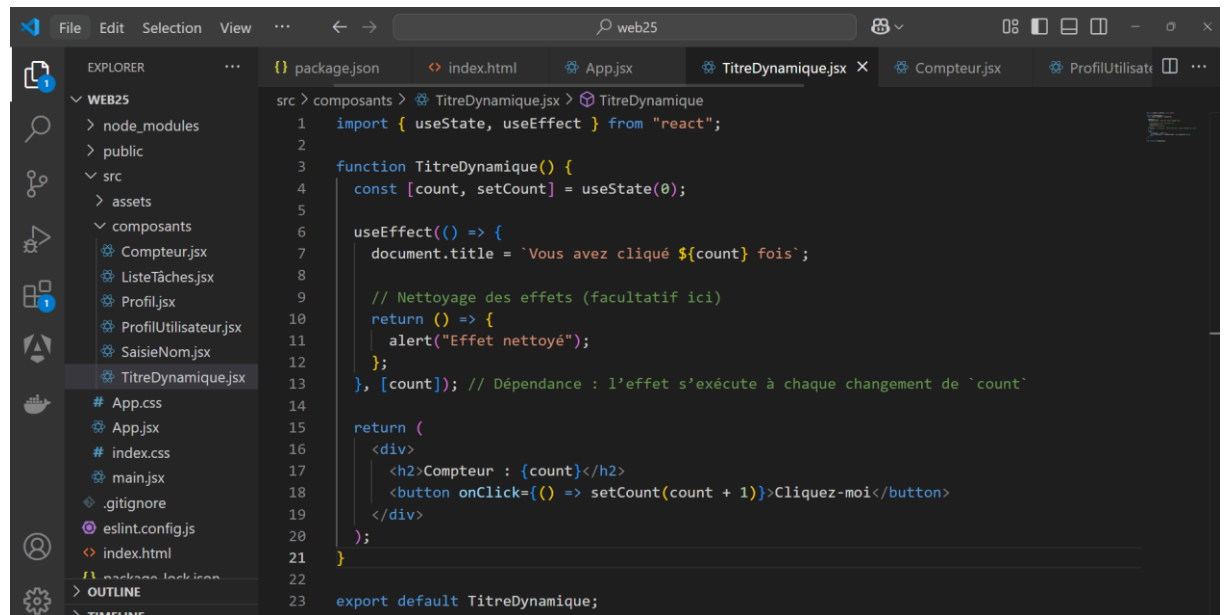
</div>

);

}

```

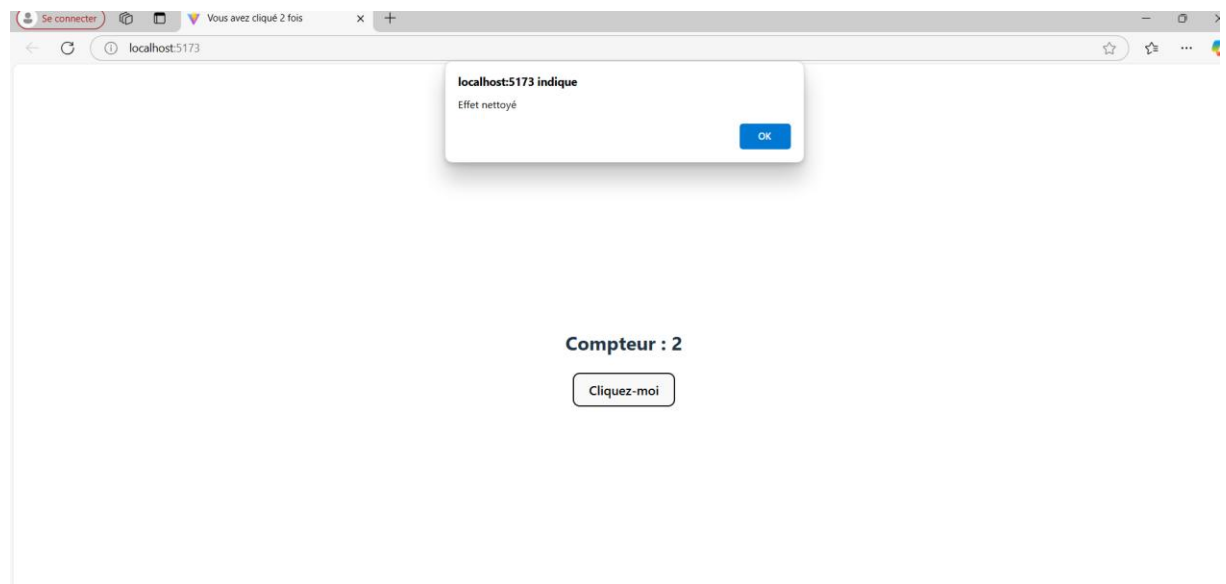
```
export default TitreDynamique;
```



```

src > composants > TitreDynamique.jsx > TitreDynamique
1  import { useState, useEffect } from "react";
2
3  function TitreDynamique() {
4    const [count, setCount] = useState(0);
5
6    useEffect(() => {
7      document.title = `Vous avez cliqué ${count} fois`;
8
9      // Nettoyage des effets (facultatif ici)
10     return () => {
11       alert("Effet nettoyé");
12     };
13   }, [count]); // Dépendance : l'effet s'exécute à chaque changement de `count`
14
15   return (
16     <div>
17       <h2>Compteur : {count}</h2>
18       <button onClick={() => setCount(count + 1)}>Cliquez-moi</button>
19     </div>
20   );
21 }
22
23 export default TitreDynamique;

```



Cas gestion bancaire simple

```
import React, { useState, useEffect } from "react";
```

```
function BankAccount() {

  const [balance, setBalance] = useState(1000); // Solde initial de 1000€

  function deposit() {

    setBalance((prev) => prev + 100); // Déposer 100€

  }

  function withdraw() {

    setBalance((prev) => (prev >= 100 ? prev - 100 : prev)); // Retirer 100€, sans aller sous 0€

  }

  useEffect(() => {

    console.log(`Nouveau solde : ${balance}€`);

  }, [balance]); // S'exécute à chaque changement du solde

  return (

    <div style={{ textAlign: "center", padding: "20px", fontSize: "20px" }}>

      <h2>Solde : {balance}€</h2>

      <button onClick={deposit} style={{ marginRight: "10px" }}>

        Déposer 100€

      </button>

      <button onClick={withdraw} disabled={balance < 100}>

        Retirer 100€

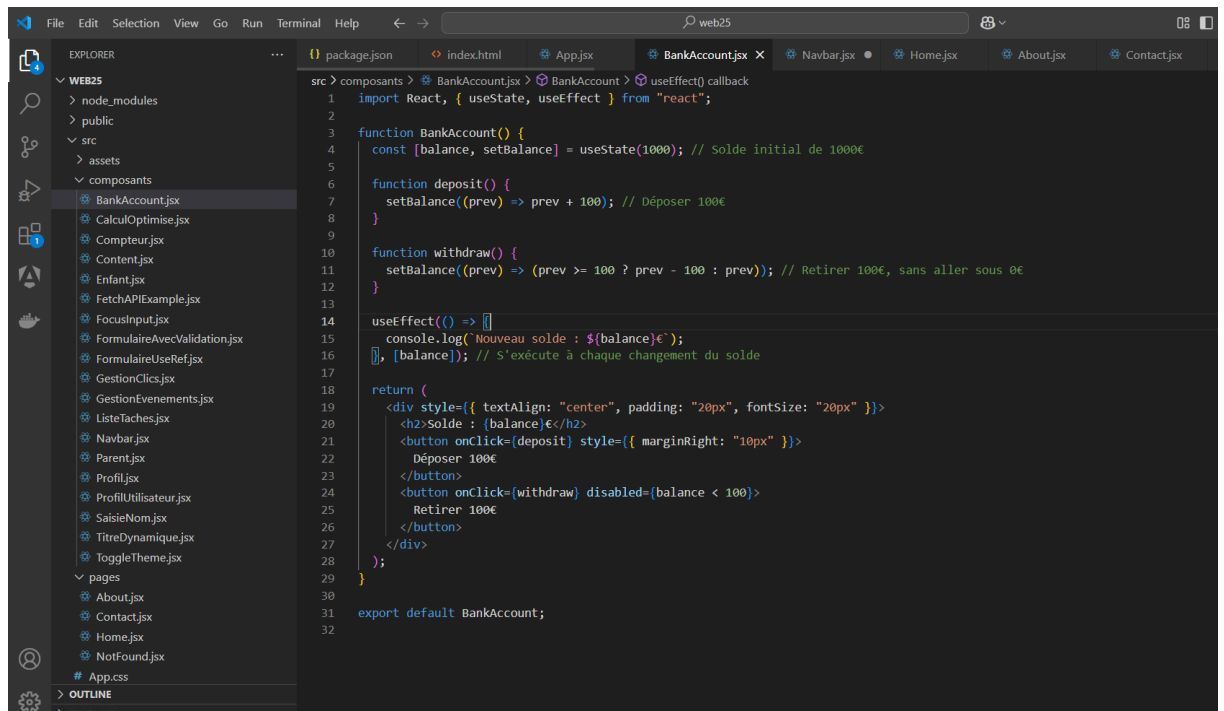
      </button>

    </div>

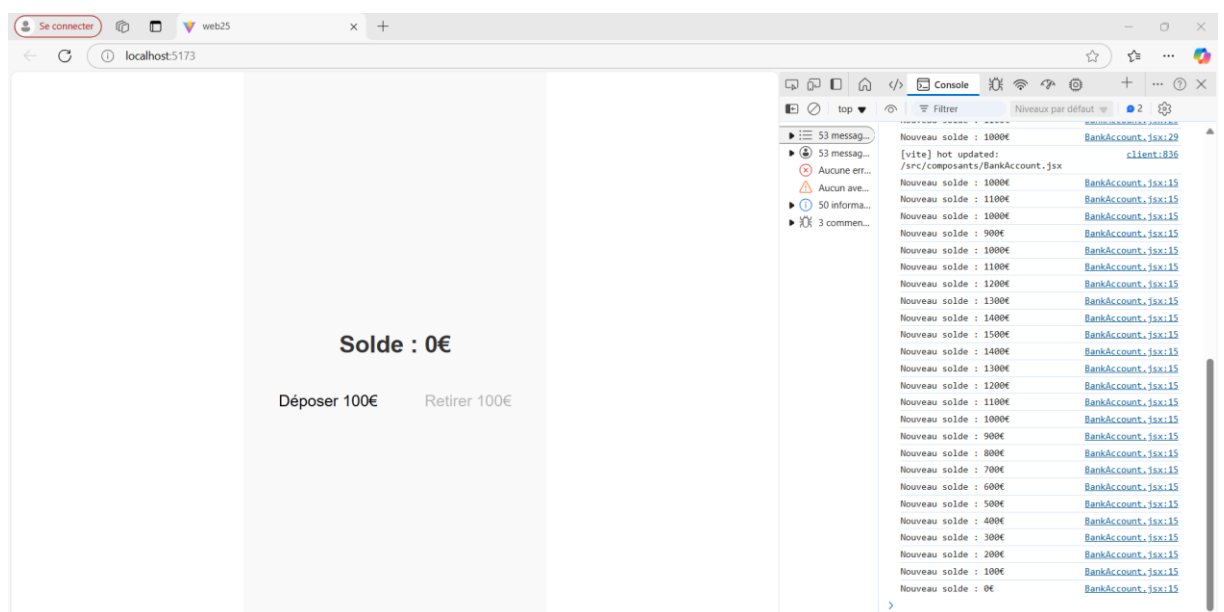
  )
}
```

```
);  
  
}
```

```
export default BankAccount;
```



```
src > composants > BankAccount.jsx > BankAccount > useEffect() callback  
1  import React, { useState, useEffect } from "react";  
2  
3  function BankAccount() {  
4    const [balance, setBalance] = useState(1000); // Solde initial de 1000€  
5  
6    function deposit() {  
7      setBalance((prev) => prev + 100); // Déposer 100€  
8    }  
9  
10   function withdraw() {  
11     setBalance((prev) => (prev >= 100 ? prev - 100 : prev)); // Retirer 100€, sans aller sous 0€  
12   }  
13  
14   useEffect(() => {  
15     console.log("Nouveau solde : ${balance}€");  
16     [, [balance]]; // S'exécute à chaque changement du solde  
17   }, [balance]);  
18  
19   return (  
20     <div style={{ textAlign: "center", padding: "20px", fontSize: "20px" }}>  
21       <h2>Solde : {balance}</h2>  
22       <button onClick={deposit} style={{ marginRight: "10px" }}>  
23         Déposer 100€  
24       </button>  
25       <button onClick={withdraw} disabled={balance < 100}>  
26         Retirer 100€  
27       </button>  
28     </div>  
29   );  
30 }  
31  
32 export default BankAccount;
```



3. Hook useRef : Références et Accès Direct au DOM

useRef permet de :

- Accéder directement à un élément du DOM (par exemple, un champ de formulaire),
- Stocker une valeur persistante entre deux re-renders sans déclencher un re-render.

Syntaxe de base :

```
const maRef = useRef(valeurInitiale);
```

Exemple : Focus sur un champ de texte

```
import { useRef } from "react";
```

```
function FocusInput() {
```

```
  const inputRef = useRef(null);
```

```
  const focusInput = () => {
```

```
    inputRef.current.focus(); // Accès direct à l'élément DOM
```

```
  };
```

```
  return (
```

```
    <div>
```

```
      <input ref={inputRef} type="text" placeholder="Entrez quelque chose" />
```

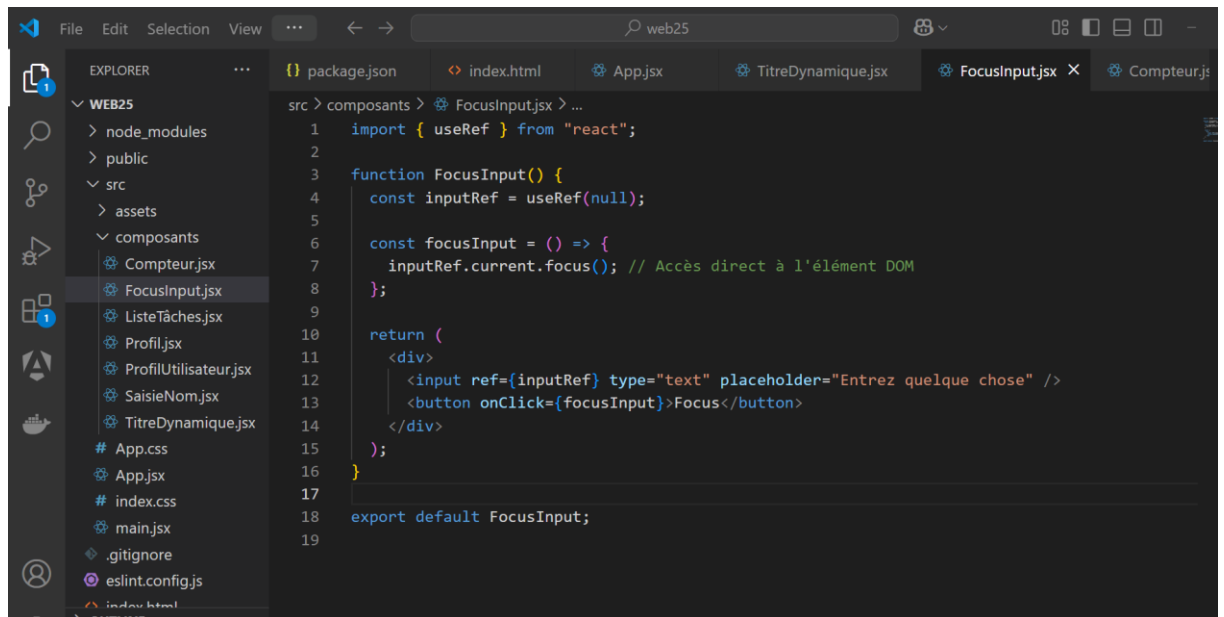
```
      <button onClick={focusInput}>Focus</button>
```

```
    </div>
```

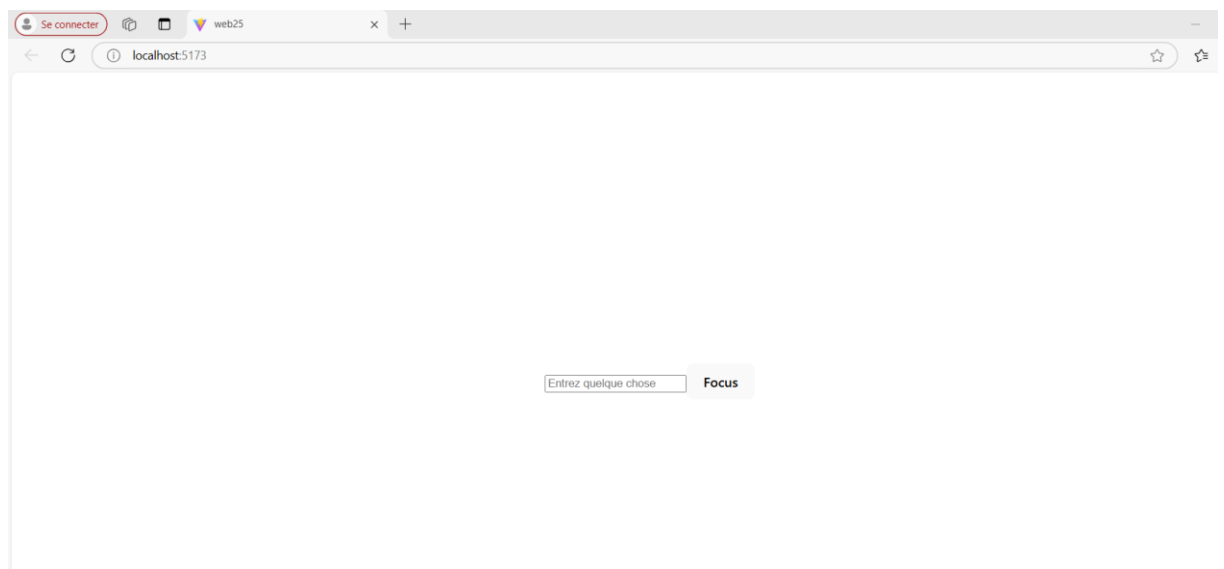


```
);  
}
```

```
export default FocusInput;
```



```
src > composants > FocusInput.jsx > ...  
1  import { useRef } from "react";  
2  
3  function FocusInput() {  
4    const inputRef = useRef(null);  
5  
6    const focusInput = () => {  
7      inputRef.current.focus(); // Accès direct à l'élément DOM  
8    };  
9  
10   return (  
11     <div>  
12       <input ref={inputRef} type="text" placeholder="Entrez quelque chose" />  
13       <button onClick={focusInput}>Focus</button>  
14     </div>  
15   );  
16 }  
17  
18 export default FocusInput;  
19
```



Explications :

1. **useRef initialise une référence** avec `null`.
2. Le champ de texte est lié à cette référence via `ref={inputRef}`.

3. Lors du clic, `inputRef.current.focus()` met le focus sur l'input.

➔ **Composant bancaire** utilisant `useRef`, qui permet de stocker une valeur sans déclencher un re-render lorsqu'elle change.

Nous allons utiliser `useRef` pour stocker l'historique du dernier solde avant la dernière modification.

```
import React, { useState, useEffect, useRef } from "react";
```

```
function BankAccount() {
```

```
  const [balance, setBalance] = useState(1000); // Solde initial de 1000€
```

```
  const previousBalance = useRef(1000); // Stocke le dernier solde sans provoquer de re-render
```

```
  function deposit() {
```

```
    setBalance((prev) => prev + 100); // Déposer 100€
```

```
  }
```

```
  function withdraw() {
```

```
    setBalance((prev) => (prev >= 100 ? prev - 100 : prev)); // Retirer 100€, sans aller sous 0€
```

```
  }
```

```
  useEffect(() => {
```

```
    console.log(`Ancien solde : ${previousBalance.current}€ → Nouveau solde :  
    ${balance}€`);
```

```
    previousBalance.current = balance; // Met à jour la valeur du ref après chaque changement
```

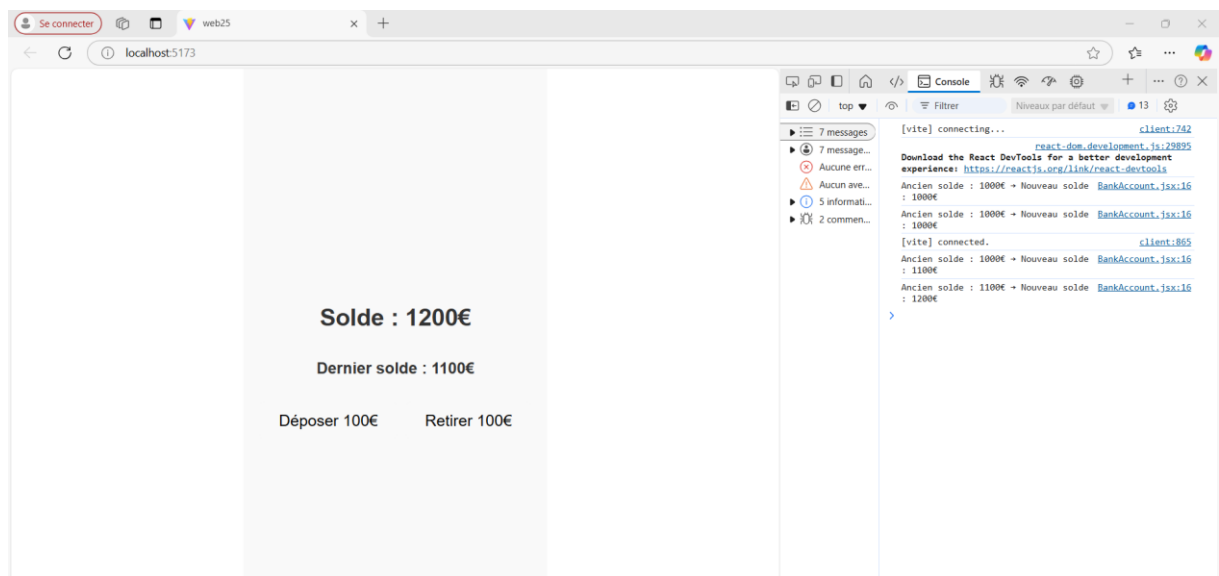
```
  }, [balance]); // S'exécute quand balance change
```

```
return (  
  <div style={{ textAlign: "center", padding: "20px", fontSize: "20px" }}>  
    <h2>Solde : {balance}€</h2>  
    <h4>Dernier solde : {previousBalance.current}€</h4>  
    <button onClick={deposit} style={{ marginRight: "10px" }}>  
      Déposer 100€  
    </button>  
    <button onClick={withdraw} disabled={balance < 100}>  
      Retirer 100€  
    </button>  
  </div>  
);  
}  
  
export default BankAccount;
```

```

src > composants > BankAccount.jsx > BankAccount
1  import React, { useState, useEffect, useRef } from "react";
2
3
4  function BankAccount() {
5    const [balance, setBalance] = useState(1000); // Solde initial de 1000€
6    const previousBalance = useRef(1000); // Stocke le dernier solde sans provoquer de re-render
7
8    function deposit() {
9      setBalance((prev) => prev + 100); // Déposer 100€
10   }
11
12   function withdraw() {
13     setBalance((prev) => (prev >= 100 ? prev - 100 : prev)); // Retirer 100€, sans aller sous 0€
14   }
15
16   useEffect(() => {
17     console.log("Ancien solde : ${previousBalance.current}€ → Nouveau solde : ${balance}€");
18     previousBalance.current = balance; // Met à jour la valeur du ref après chaque changement
19   }, [balance]); // S'exécute quand balance change
20
21   return (
22     <div style={{ textAlign: "center", padding: "20px", fontSize: "20px" }}>
23       <h2>Solde : {balance}</h2>
24       <h4>Dernier solde : {previousBalance.current}</h4>
25       <button onClick={deposit} style={{ marginRight: "10px" }}>
26         Déposer 100€
27       </button>
28       <button onClick={withdraw} disabled={balance < 100}>
29         Retirer 100€
30       </button>
31     </div>
32   );
33 }
34 export default BankAccount;
35
36

```



Explication :

1. **useState(1000)** : Gère l'état du solde bancaire.
2. **useRef(1000)** : Stocke l'ancien solde sans provoquer de re-render.
3. **useEffect** :
 - Affiche l'ancien et le nouveau solde dans la console.
 - Met à jour `previousBalance.current` avec la nouvelle valeur du solde.

3. Hook useMemo : Optimisation des Performances

useMemo est utilisé pour **mémoriser le résultat d'un calcul coûteux** et éviter de le recalculer inutilement.

Syntaxe de base :

```
const valeurMémoiree = useMemo(() => {  
  
  // Calcul complexe  
  
  return résultat;  
  
}, [dépendances]);
```

Exemple : Calcul Optimisé

```
import { useState, useMemo } from "react";  
  
function CalculOptimise() {  
  
  const [nombre, setNombre] = useState(0);  
  
  const [autreValeur, setAutreValeur] = useState(0);  
  
  // Calcul coûteux uniquement si `nombre` change  
  
  const facteur = useMemo(() => {  
  
    console.log("Calcul en cours...");  
  
    return nombre * 2;  
  
  }, [nombre]);  
  
  return (  

```

```

<div>

  <h2>Nombre : {nombre}</h2>

  <h3>Facteur (optimisé) : {facteur}</h3>

  <button onClick={() => setNombre(nombre + 1)}>Incrémenter Nombre</button>

  <button onClick={() => setAutreValeur(autreValeur + 1)}>

    Incrémenter Autre Valeur

  </button>

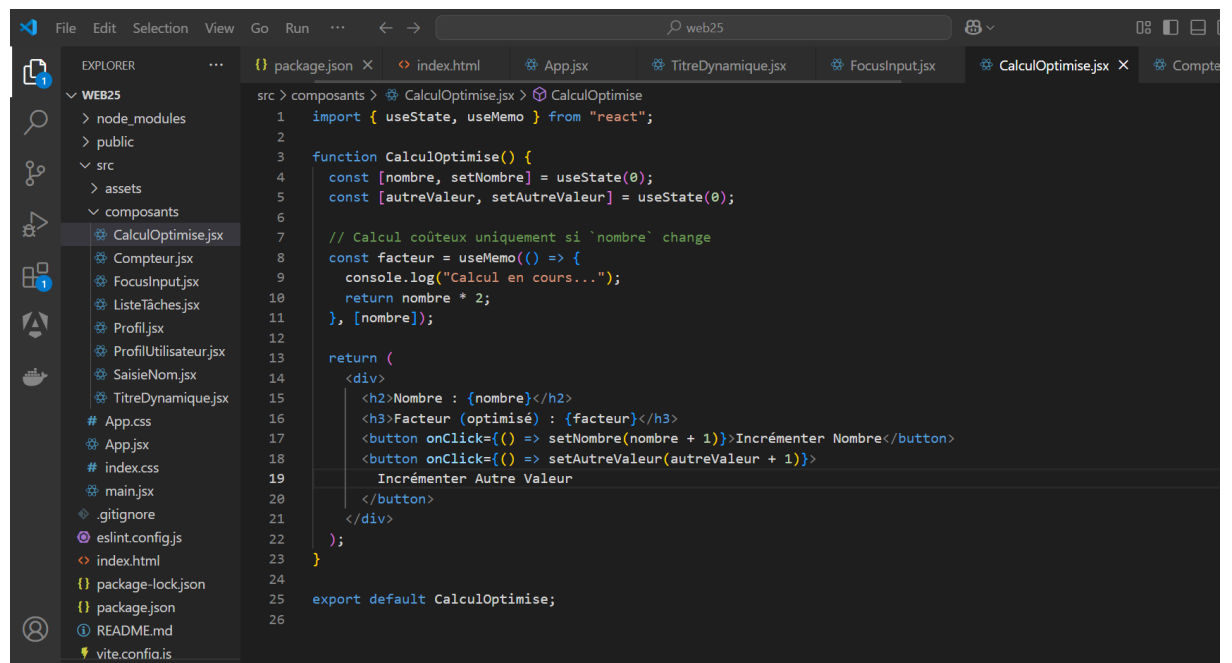
</div>

);

}

export default CalculOptimise;

```



Explications :

- Sans **useMemo**, le calcul de `nombre * 2` serait effectué à chaque re-render.

- Avec `useMemo`, le calcul ne se refait que si `nombre` change.

Résumé des Hooks Avancés

Hook	Utilisation	Quand l'utiliser ?
<code>useEffect</code>	Gérer les effets de bord (API, abonnement, timer)	À chaque changement de state ou prop
<code>useRef</code>	Accès direct au DOM ou stocker une valeur persistante	Lorsque vous avez besoin d'une référence
<code>useMemo</code>	Mémoriser le résultat d'un calcul	Lorsque le calcul est coûteux

Composant bancaire utilisant `useMemo`. L'objectif ici est d'optimiser certains calculs, comme le calcul du solde total ou d'autres valeurs dérivées qui n'ont besoin d'être recalculées que lorsque certaines dépendances changent.

Dans cet exemple, on utilise `useMemo` pour mémoriser une valeur dérivée comme **le solde restant après un retrait possible**. Cela permet d'optimiser les calculs en évitant des recalculs inutiles à chaque rendu du composant.

```
import React, { useState, useEffect, useMemo } from "react";

function BankAccount() {

  const [balance, setBalance] = useState(1000); // Solde initial de 1000€

  function deposit() {

    setBalance((prev) => prev + 100); // Déposer 100€

  }

  function withdraw() {

    setBalance((prev) => (prev >= 100 ? prev - 100 : prev)); // Retirer 100€, sans aller sous 0€

  }

}
```

```

// Utilisation de useMemo pour calculer le solde restant après retrait

const remainingBalance = useMemo(() => {

  return balance - 100; // Retrait de 100€, mémorisé pour éviter un recalcul à chaque rendu

}, [balance]);

useEffect(() => {

  console.log(`Solde actuel : ${balance}€`);

}, [balance]); // S'exécute à chaque changement du solde

return (

  <div style={{ textAlign: "center", padding: "20px", fontSize: "20px" }}>

    <h2> Solde : {balance}€</h2>

    <h4> Solde après retrait de 100€ : {remainingBalance}€</h4>

    <button onClick={deposit} style={{ marginRight: "10px" }}>

      Déposer 100€

    </button>

    <button onClick={withdraw} disabled={balance < 100}>

      Retirer 100€

    </button>

  </div>

);

}

export default BankAccount;

```


Se connecter

web25

localhost:5173

Solde : 1000€

Solde après retrait de 100€ : 900€

Déposer 100€ Retirer 100€

Console

43 messages

43 messages

Aucune erreur

Aucune avertissement

41 informations

2 commentaires

Solde actuel : 700€

Solde actuel : 600€

Solde actuel : 500€

Solde actuel : 600€

Solde actuel : 700€

Solde actuel : 800€

Solde actuel : 900€

Solde actuel : 1000€

Solde actuel : 1100€

Solde actuel : 1200€

Solde actuel : 1300€

Solde actuel : 1400€

Solde actuel : 1500€

Solde actuel : 1600€

Solde actuel : 1700€

Solde actuel : 1800€

Solde actuel : 1900€

Solde actuel : 2000€

Solde actuel : 1900€

Solde actuel : 1800€

Solde actuel : 1700€

Solde actuel : 1600€

Solde actuel : 1500€

Solde actuel : 1400€

Solde actuel : 1300€

Solde actuel : 1200€

Solde actuel : 1100€

Solde actuel : 1000€

Optimisation avec `useCallback`

`useCallback` permet de **mémoriser une fonction** pour éviter qu'elle ne soit recréée à chaque re-render. C'est utile pour empêcher des composants enfants de se re-render inutilement.

Exemple : Bouton avec une fonction optimisée

```
import React, { useState, useCallback } from "react";
```

// Composant principal

```
function GestionClics() {
```

```
  const [compteur, setCompteur] = useState(0);
```

```
  // Fonction pour incrémenter le compteur
```

```
  const incrementer = () => {
```

```
    setCompteur((prevCompteur) => prevCompteur + 1);
```

```
  };
```

```
  // Fonction pour afficher une alerte
```

```
  // Utilisation de useCallback pour éviter que la fonction soit recréée
```

```
  const afficherAlerte = useCallback(() => {
```

```
    alert("Bonjour ! Voici une alerte !");
```

```
  }, []);
```

```
  return (
```

```
    <div style={{ textAlign: "center", marginTop: "50px" }}>
```

```
      <h1>Compteur : {compteur}</h1>
```

```
    <Bouton onClick={incrémenter}>Incrémenter le Compteur</Bouton>

    <Bouton onClick={afficherAlerte}>Afficher une Alerte</Bouton>

  </div>

);
}
```

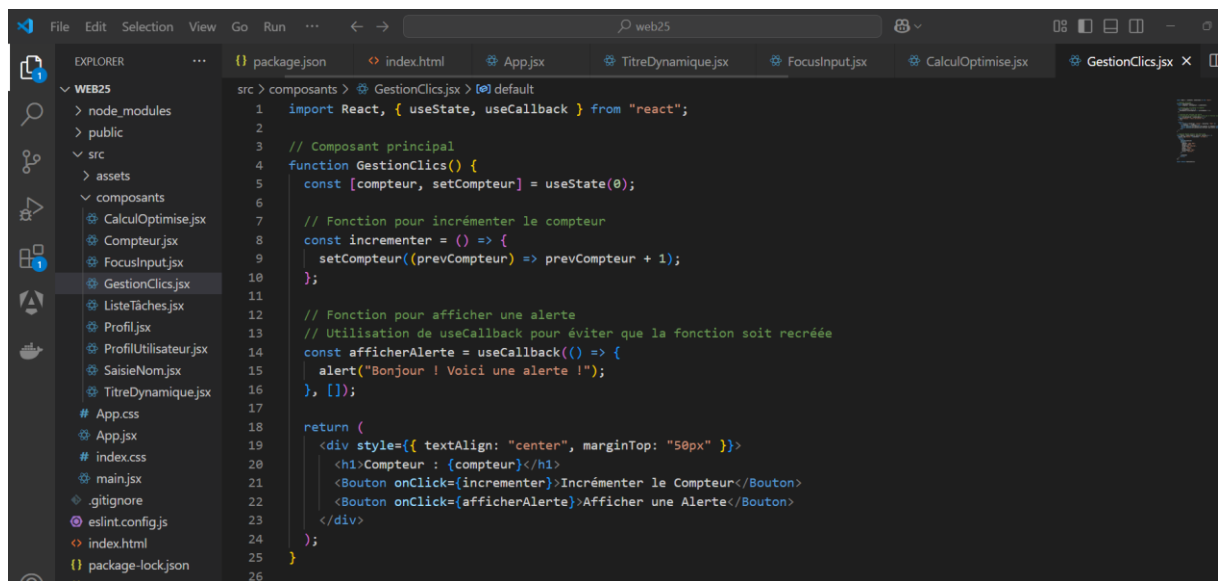
// Composant Bouton optimisé avec React.memo

```
const Bouton = React.memo(({ onClick, children }) => {
  console.log(`Bouton "${children}" re-rendu`);
  return (
    <button
      onClick={onClick}
      style={{
        padding: "10px 20px",
        margin: "10px",
        background: "#4caf50",
        color: "white",
        border: "none",
        borderRadius: "5px",
        cursor: "pointer",
      }}
    >
      {children}
    </button>
  )
});
```

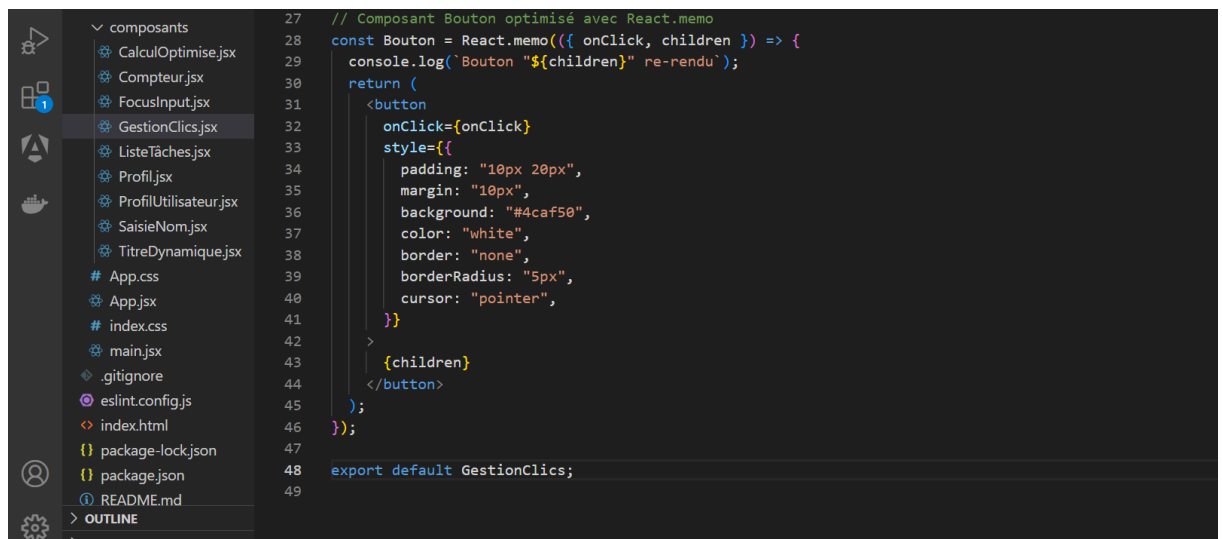
);

});

export default GestionClics;



```
1 import React, { useState, useCallback } from "react";
2
3 // Composant principal
4 function GestionClics() {
5   const [compteur, setCompteur] = useState(0);
6
7   // Fonction pour incrémenter le compteur
8   const incrementer = () => {
9     setCompteur((prevCompteur) => prevCompteur + 1);
10  };
11
12  // Fonction pour afficher une alerte
13  // Utilisation de useCallback pour éviter que la fonction soit recréée
14  const afficherAlerte = useCallback(() => {
15    alert("Bonjour ! Voici une alerte !");
16  }, []);
17
18  return (
19    <div style={{ textAlign: "center", marginTop: "50px" }}>
20      <h1>Compteur : {compteur}</h1>
21      <Bouton onClick={incrementer}>Incrémenter le Compteur</Bouton>
22      <Bouton onClick={afficherAlerte}>Afficher une Alerte</Bouton>
23    </div>
24  );
25 }
26
```



```
27 // Composant Bouton optimisé avec React.memo
28 const Bouton = React.memo(({ onClick, children }) => {
29   console.log(`Bouton "${children}" re-rendu`);
30   return (
31     <button
32       onClick={onClick}
33       style={{
34         padding: "10px 20px",
35         margin: "10px",
36         background: "#4caf50",
37         color: "white",
38         border: "none",
39         borderRadius: "5px",
40         cursor: "pointer",
41       }}
42     >
43       {children}
44     </button>
45   );
46 });
47
48 export default GestionClics;
49
```

- **Sans `useCallback`** → Une **nouvelle fonction** est créée à chaque re-rendu.
- **Avec `useCallback`** → La fonction est **mémorisée** et **réutilisée** si rien ne change dans les dépendances.

Les **dépendances** dans `useCallback` sont les **valeurs** sur lesquelles la fonction s'appuie pour fonctionner.

Elles sont placées dans les **crochets** `[]` et indiquent à **React** :

"Si ces valeurs changent, recrée la fonction. Sinon, garde l'ancienne."

```
import React, { useState, useCallback } from "react";
```

```
function BankAccount() {
```

```
  const [balance, setBalance] = useState(1000);
```

```
  const [bonus, setBonus] = useState(100); // Prime qui change
```

```
  // La fonction dépend de "bonus"
```

```
  const deposit = useCallback(() => {
```

```
    setBalance((prev) => prev + 500 + bonus);
```

```
  }, [bonus]); // "bonus" est une dépendance
```

```
  return (
```

```
    <div style={{ textAlign: "center", padding: "20px" }}>
```

```
      <h2>Solde : {balance} €</h2>
```

```
      <h4>Bonus actuel : {bonus} €</h4>
```

```
      <button onClick={deposit}>Déposer 500€ + Bonus</button>
```

`<br /><br />`

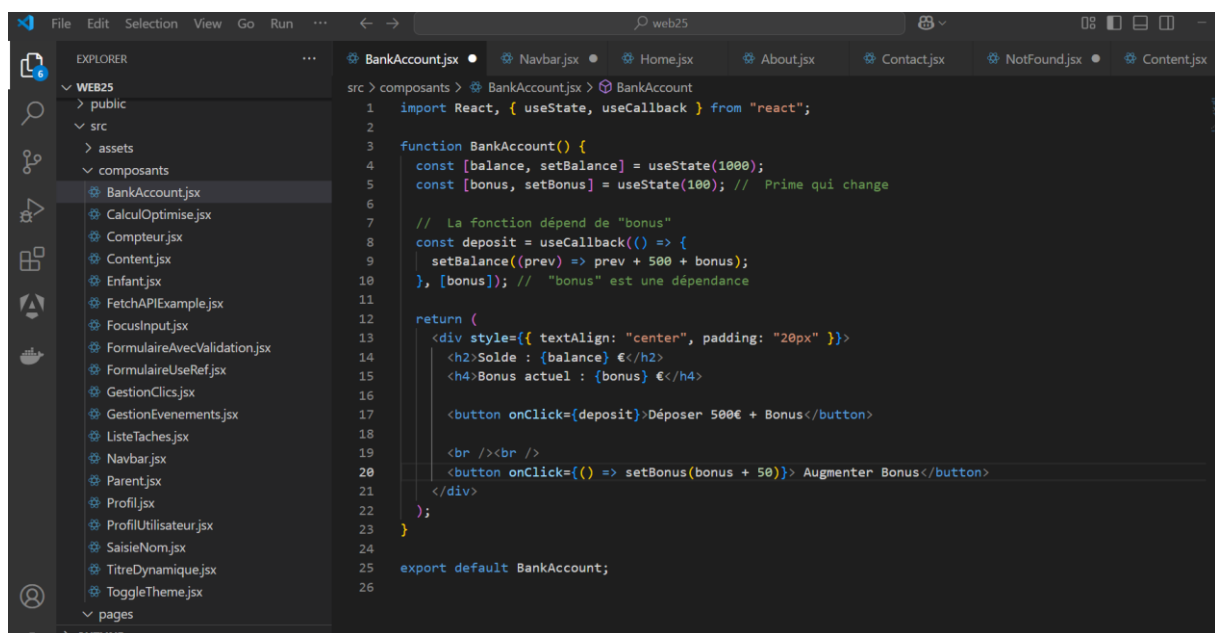
`<button onClick={() => setBonus(bonus + 50)}> Augmenter Bonus</button>`

`</div>`

`);`

`}`

`export default BankAccount;`



Gestion des événements en React

En React, la gestion des événements est similaire à JavaScript standard, mais avec quelques différences notables :

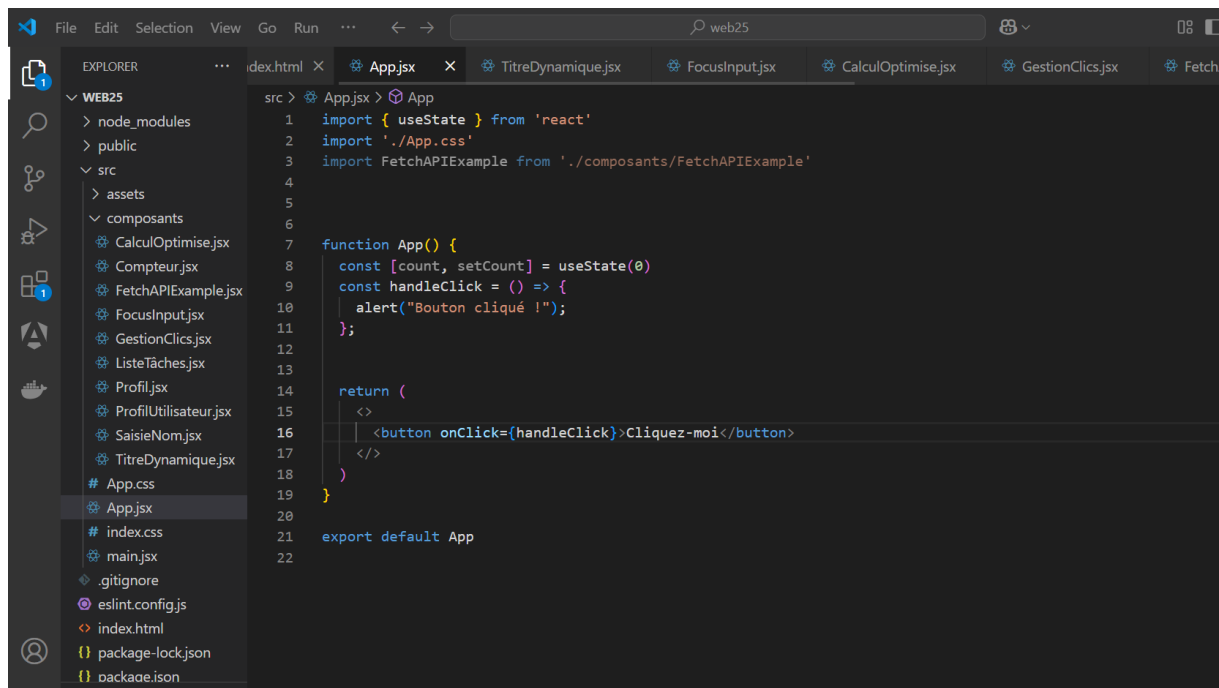
1. Les événements sont nommés en **camelCase** (ex : `onClick`, `onChange`).

2. Au lieu d'utiliser des chaînes de caractères comme gestionnaires d'événements (comme en HTML), vous passez une **fonction JavaScript**.

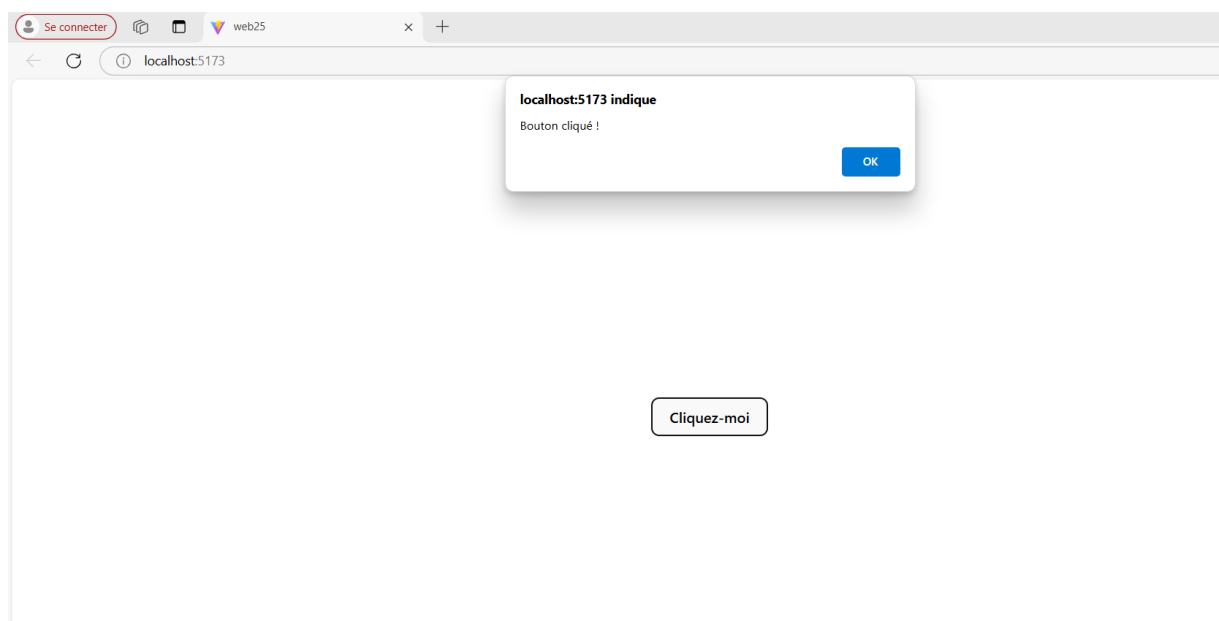
Syntaxe de base

Voici un exemple de gestion d'un clic de bouton :

```
function App() {  
  const handleClick = () => {  
    alert("Bouton cliqué !");  
  };  
  
  return (  
    <button onClick={handleClick}>Cliquez-moi</button>  
  );  
}
```



```
1 import { useState } from 'react'
2 import './App.css'
3 import FetchAPIExample from './composants/FetchAPIExample'
4
5
6
7 function App() {
8   const [count, setCount] = useState(0)
9   const handleClick = () => {
10     alert("Bouton cliqué !");
11   };
12
13   return (
14     <>
15     <button onClick={handleClick}>Cliquez-moi</button>
16     </>
17   )
18 }
19
20
21 export default App
22
```



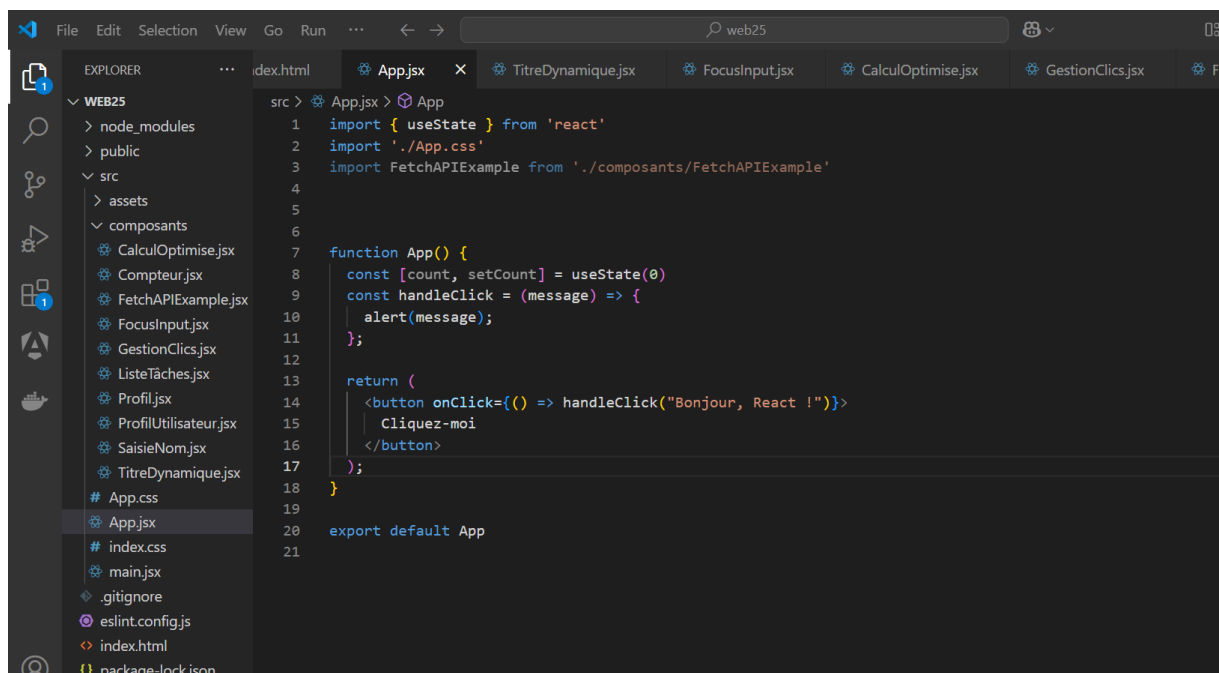
Différences entre HTML et React

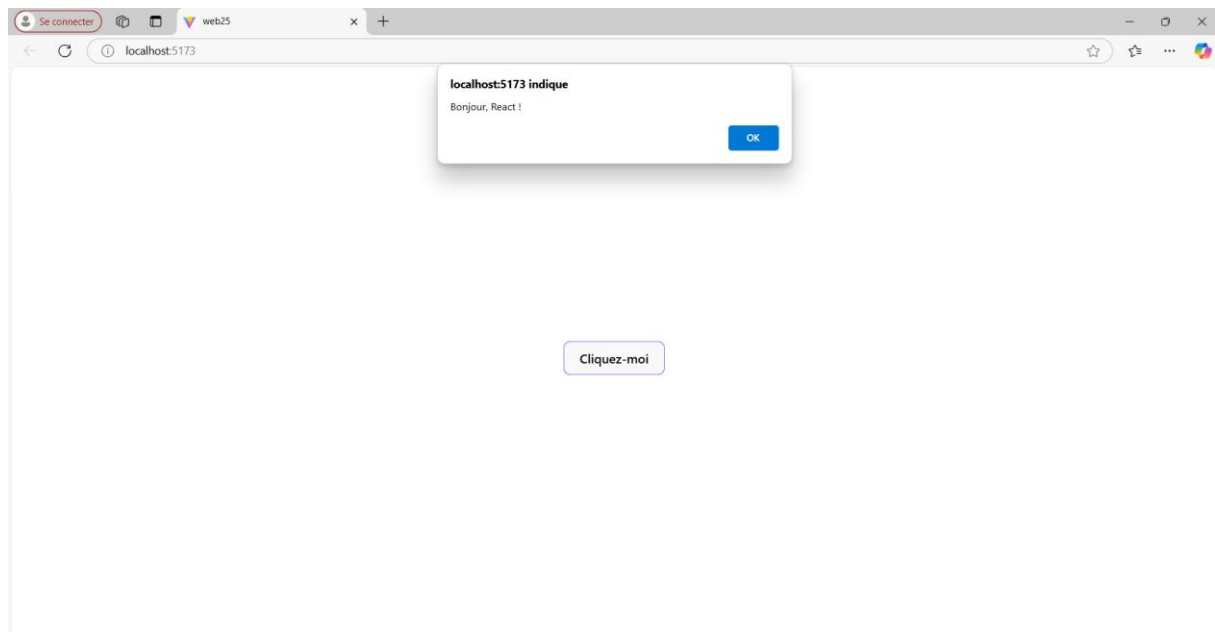
HTML traditionnel	React
<code><button onclick="handleClick()"></code>	<code><button onClick={handleClick}></code>
Les gestionnaires d'événements sont des chaînes de caractères	Les gestionnaires d'événements sont des fonctions JavaScript

Gestionnaire d'événements avec paramètres

Si vous devez passer des paramètres à votre fonction, utilisez une fonction fléchée ou `.bind()` :

```
function App() {  
  
  const handleClick = (message) => {  
  
    alert(message);  
  
  };  
  
  return (  
  
    <button onClick={() => handleClick("Bonjour, React !")}>  
  
      Cliquez-moi  
  
    </button>  
  
  );  
}
```





Liste des événements courants en React

Voici quelques événements React couramment utilisés :

1. **Événements souris :**
 - `onClick`
 - `onDoubleClick`
 - `onMouseEnter`
 - `onMouseLeave`
 - `onMouseMove`
2. **Événements clavier :**
 - `onKeyDown`
 - `onKeyPress` (déprécié, utilisez `onKeyDown`)
 - `onKeyUp`
3. **Événements de formulaire :**
 - `onChange` (pour les champs de formulaire)
 - `onSubmit` (pour les formulaires)
 - `onFocus`
 - `onBlur`
4. **Événements de fenêtre :**
 - `onScroll`
 - `onResize`

Exemple : Gestion des événements courants

Cet exemple montre comment gérer des événements courants, comme un clic et la saisie dans un champ de formulaire :

```
import React, { useState } from "react";

function GestionEvenements() {

  const [inputValue, setInputValue] = useState("");

  const handleClick = () => {

    alert("Bouton cliqué !");

  };

  const handleChange = (event) => {

    setInputValue(event.target.value); // Mise à jour de l'état avec la valeur de l'input

  };

  const handleSubmit = (event) => {

    event.preventDefault(); // Empêche le rechargement de la page

    alert(`Formulaire soumis avec : ${inputValue}`);

  };

  return (

    <div style={{ padding: "20px" }}>

      <h1>Gestion des événements en React</h1>

      { /* Gestion du clic */ }

      <button onClick={handleClick} style={{ marginBottom: "10px" }}>

        Cliquez-moi

      </button>

    </div>

  );
}
```

```
</button>
```

```
{/* Gestion du formulaire */}
```

```
<form onSubmit={handleSubmit}>
```

```
  <input
```

```
    type="text"
```

```
    value={inputValue}
```

```
    onChange={handleChange}
```

```
    placeholder="Tapez quelque chose"
```

```
    style={{ marginRight: "10px" }}
```

```
  />
```

```
  <button type="submit">Soumettre</button>
```

```
</form>
```

```
</div>
```

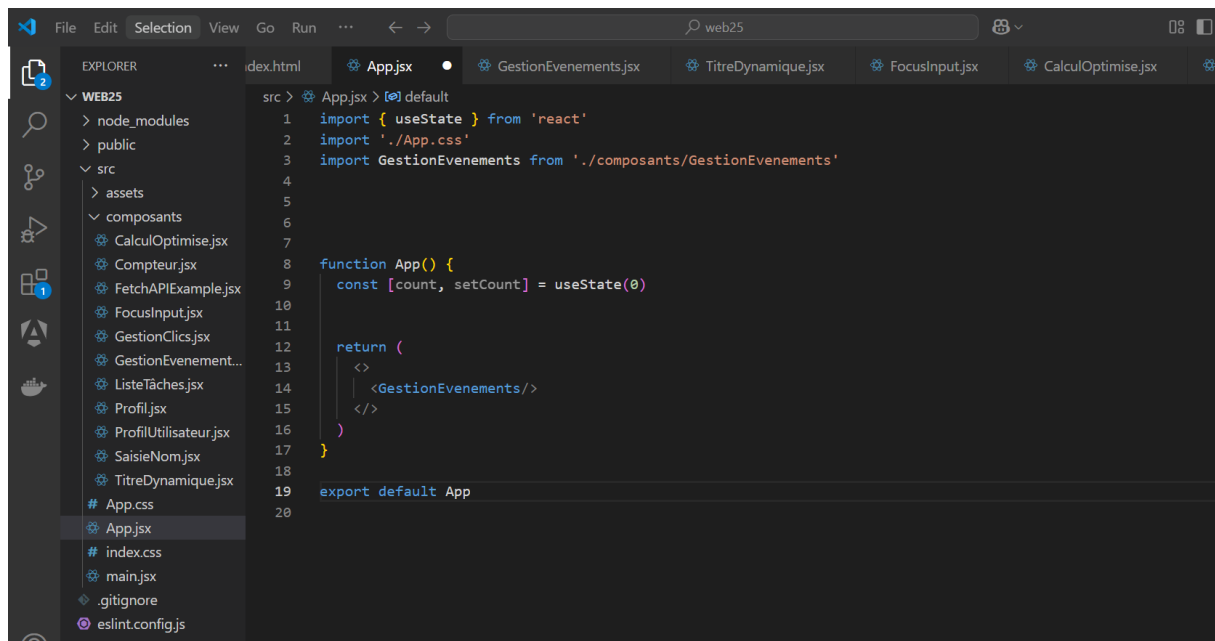
```
);
```

```
}
```

```
export default GestionEvenements;
```

```
1 import React, { useState } from "react";
2
3 function GestionEvenements() {
4   const [inputValue, setInputValue] = useState("");
5
6   const handleClick = () => {
7     alert("Bouton cliqué !");
8   };
9
10  const handleChange = (event) => {
11    setInputValue(event.target.value); // Mise à jour de l'état avec la valeur de l'input
12  };
13
14  const handleSubmit = (event) => {
15    event.preventDefault(); // Empêche le rechargement de la page
16    alert('Formulaire soumis avec : ${inputValue}');
17  };
18
19  return (
20    <div style={{ padding: "20px" }}>
21      <h1>Gestion des événements en React</h1>
22
23      /* Gestion du clic */
24      <button onClick={handleClick} style={{ marginBottom: "10px" }}>
25        Cliquez-moi
26      </button>
```

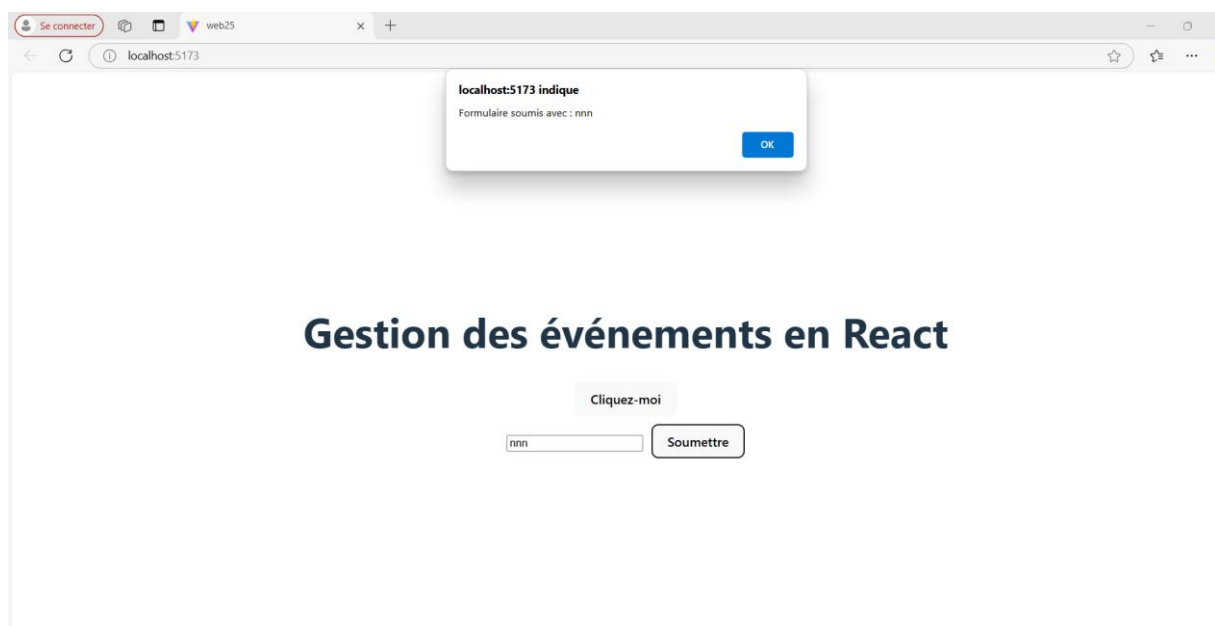
```
27
28    </button>
29    <form onSubmit={handleSubmit}>
30      <input
31        type="text"
32        value={inputValue}
33        onChange={handleChange}
34        placeholder="Tapez quelque chose"
35        style={{ marginRight: "10px" }}
36      />
37      <button type="submit">Soumettre</button>
38    </form>
39  </div>
40  );
41 }
42
43 export default GestionEvenements;
44
```



```
src > App.jsx > default
1 import { useState } from 'react'
2 import './App.css'
3 import GestionEvenements from './composants/GestionEvenements'
4
5
6
7
8 function App() {
9   const [count, setCount] = useState(0)
10
11
12   return (
13     <>
14       <GestionEvenements/>
15     </>
16   )
17 }
18
19 export default App
20
```

Explications :

1. **onClick :**
 - Attache un gestionnaire pour détecter les clics sur le bouton.
2. **onChange :**
 - Écoute les changements de valeur dans un champ de saisie (input).
 - Utilisation de `event.target.value` pour récupérer la nouvelle valeur.
3. **onSubmit :**
 - Intercepte la soumission du formulaire.
 - Utilisation de `event.preventDefault()` pour empêcher le rechargement de la page par défaut.



❖ Création et validation de formulaires

Concepts clés :

1. Gestion de l'état du formulaire : Avec `useState` pour suivre les valeurs des champs.
2. Validation des champs : Vérification des données saisies.
3. Gestion de l'événement `onSubmit` : Empêcher le rechargement de la page avec `event.preventDefault()`

Formulaire avec validation simple

```
import React, { useState } from "react";
```

```
function FormulaireAvecValidation() {  
  
  // États pour les champs du formulaire  
  
  const [formData, setFormData] = useState({  
  
    nom: "",  
  
    email: "",  
  
    motDePasse: "",  
  
  });
```

```
  // État pour les erreurs  
  
  const [erreurs, setErreurs] = useState({});
```

```
  // Fonction de gestion des changements dans les champs
```

```
  const handleChange = (event) => {  
  
    const { name, value } = event.target;  
  
    setFormData((prev) => ({  
  
      ...prev,  
  
      [name]: value, // Mise à jour du champ spécifique
```

```

    }));

};

// Fonction de validation des données

const valider = () => {

    const nouvellesErreurs = {};

    if (!formData.nom.trim()) {

        nouvellesErreurs.nom = "Le nom est requis.";

    }

    if (!formData.email.trim()) {

        nouvellesErreurs.email = "L'email est requis.";

    } else if (!/^S+@\S+\.\S+/.test(formData.email)) {

        nouvellesErreurs.email = "L'email est invalide.";

    }

    if (!formData.motDePasse) {

        nouvellesErreurs.motDePasse = "Le mot de passe est requis.";

    } else if (formData.motDePasse.length < 6) {

        nouvellesErreurs.motDePasse = "Le mot de passe doit contenir au moins 6 caractères.";

    }

    return nouvellesErreurs;

};

```



```

// Gestionnaire de soumission du formulaire

const handleSubmit = (event) => {

  event.preventDefault(); // Empêche le rechargement de la page

  const validationErreurs = valider();

  if (Object.keys(validationErreurs).length === 0) {

    console.log("Formulaire soumis avec succès :", formData);

    alert("Formulaire soumis avec succès !");

    setFormData({ nom: "", email: "", motDePasse: "" }); // Réinitialisation du
formulaire

  } else {

    setErreurs(validationErreurs); // Stocke les erreurs pour l'affichage

  }

};

return (

  <div style={{ padding: "20px", maxWidth: "400px", margin: "auto" }}>

    <h1>Formulaire avec Validation</h1>

    <form onSubmit={handleSubmit}>

      { /* Champ Nom */ }

      <div style={{ marginBottom: "15px" }}>

        <label>

          Nom :

          <input

            type="text"

```

```

        name="nom"

        value={formData.nom}

        onChange={handleChange}

        style={{ display: "block", width: "100%", padding: "8px" }}

    />

</label>

    {erreurs.nom && <p style={{ color: "red", fontSize: "12px"
}}>{erreurs.nom}</p>}

</div>

{ /* Champ Email */}

<div style={{ marginBottom: "15px" }}>

    <label>

        Email :

        <input

            type="email"

            name="email"

            value={formData.email}

            onChange={handleChange}

            style={{ display: "block", width: "100%", padding: "8px" }}

        />

    </label>

    {erreurs.email && <p style={{ color: "red", fontSize: "12px"
}}>{erreurs.email}</p>}

</div>

```

```
    { /* Champ Mot de passe */ }

    <div style={{ marginBottom: "15px" }}>

      <label>

        Mot de passe :

        <input

          type="password"

          name="motDePasse"

          value={formData.motDePasse}

          onChange={handleChange}

          style={{ display: "block", width: "100%", padding: "8px" }}

        />

      </label>

      {erreurs.motDePasse && <p style={{ color: "red", fontSize: "12px" }}>{erreurs.motDePasse}</p>}

    </div>
```

```
    { /* Bouton Soumettre */ }

    <button

      type="submit"

      style={{

        padding: "10px 20px",

        background: "blue",

        color: "white",

        border: "none",

        borderRadius: "5px",

        cursor: "pointer",
```

```

    }}
  >
  Soumettre
</button>
</form>
</div>

);
}

```

export default FormulaireAvecValidation;

```

src > composants > FormulaireAvecValidation.jsx > FormulaireAvecValidation
1  import React, { useState } from "react";
2
3  function FormulaireAvecValidation() {
4    // États pour les champs du formulaire
5    const [formData, setFormData] = useState({
6      nom: "",
7      email: "",
8      motDePasse: "",
9    });
10
11    // État pour les erreurs
12    const [erreurs, setErreurs] = useState({});
13
14    // Fonction de gestion des changements dans les champs
15    const handleChange = (event) => {
16      const { name, value } = event.target;
17      setFormData((prev) => ({
18        ...prev,
19        [name]: value, // Mise à jour du champ spécifique
20      }));
21    };
22
23

```

```

24    // Fonction de validation des données
25    const valider = () => {
26      const nouvellesErreurs = {};
27
28      if (!formData.nom.trim()) {
29        nouvellesErreurs.nom = "Le nom est requis.";
30      }
31
32      if (!formData.email.trim()) {
33        nouvellesErreurs.email = "L'email est requis.";
34      } else if (!/^[a-zA-Z0-9+@-]+\.[a-zA-Z]{2,}$/.test(formData.email)) {
35        nouvellesErreurs.email = "L'email est invalide.";
36      }
37
38      if (!formData.motDePasse) {
39        nouvellesErreurs.motDePasse = "Le mot de passe est requis.";
40      } else if (formData.motDePasse.length < 6) {
41        nouvellesErreurs.motDePasse = "Le mot de passe doit contenir au moins 6 caractères.";
42      }
43
44      return nouvellesErreurs;
45    };
46
47

```

```
46 // Gestionnaire de soumission du formulaire
47 const handleSubmit = (event) => {
48   event.preventDefault(); // Empêche le rechargement de la page
49   const validationErreurs = valider();
50
51   if (Object.keys(validationErreurs).length === 0) {
52     console.log("Formulaire soumis avec succès :", formData);
53     alert("Formulaire soumis avec succès !");
54     setFormData({ nom: "", email: "", motDePasse: "" }); // Réinitialisation du formulaire
55   } else {
56     setErreurs(validationErreurs); // Stocke les erreurs pour l'affichage
57   }
58 };
```

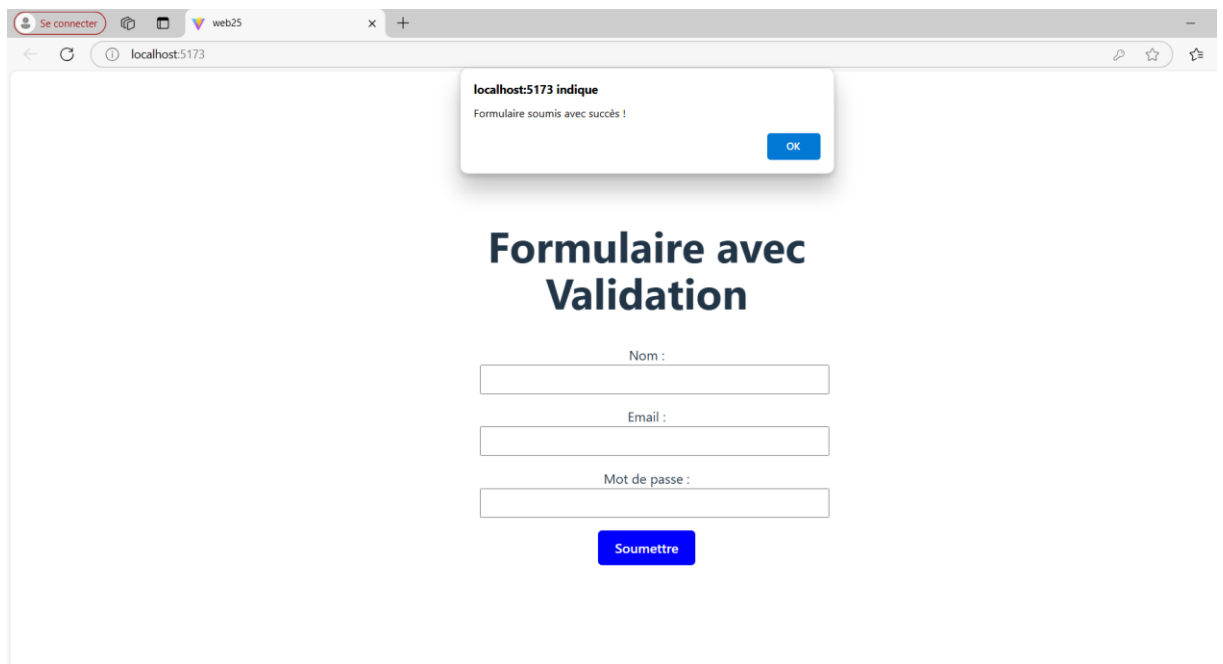
```
60 return (
61   <div style={{ padding: "20px", maxWidth: "400px", margin: "auto" }}>
62     <h1>Formulaire avec Validation</h1>
63     <form onSubmit={handleSubmit}>
64       <div style={{ margin: "10px 0" }}>
65         <div style={{ margin: "15px 0" }}>
66           <label>
67             Nom :
68             <input
69               type="text"
70               name="nom"
71               value={formData.nom}
72               onChange={handleChange}
73               style={{ display: "block", width: "100%", padding: "8px" }}
74             />
75           </label>
76           {erreurs.nom && <p style={{ color: "red", fontSize: "12px" }}>{erreurs.nom}</p>}
77         </div>
78       </div>
```

```
79 <div style={{ margin: "10px 0" }}>
80   <div style={{ margin: "15px 0" }}>
81     <label>
82       Email :
83       <input
84         type="email"
85         name="email"
86         value={formData.email}
87         onChange={handleChange}
88         style={{ display: "block", width: "100%", padding: "8px" }}
89       />
90     </label>
91     {erreurs.email && <p style={{ color: "red", fontSize: "12px" }}>{erreurs.email}</p>}
92   </div>
93 </div>
```

```
94 <div style={{ margin: "10px 0" }}>
95   <div style={{ margin: "15px 0" }}>
96     <label>
97       Mot de passe :
98       <input
99         type="password"
100         name="motDePasse"
101         value={formData.motDePasse}
102         onChange={handleChange}
103         style={{ display: "block", width: "100%", padding: "8px" }}
104       />
105     </label>
106     {erreurs.motDePasse && <p style={{ color: "red", fontSize: "12px" }}>{erreurs.motDePasse}</p>}
107   </div>
108 </div>
```



```
109  { /* Bouton Soumettre */
110
111  <button
112    type="submit"
113    style={{
114      padding: "10px 20px",
115      background: "blue",
116      color: "white",
117      border: "none",
118      borderRadius: "5px",
119      cursor: "pointer",
120    }}
121  >
122    Soumettre
123  </button>
124 </form>
125 </div>
126 );
127 }
128 export default FormulaireAvecValidation;
129
```



Explications détaillées :

1. Structure des champs :

- Chaque champ de formulaire (nom, email, mot de passe) est géré avec un état spécifique dans `formData`.
- `handleChange` met à jour la valeur du champ correspondant.

2. Validation :

- La fonction `valider` vérifie les champs un par un :
 - Champ vide.
 - Format de l'email.
 - Longueur minimale du mot de passe.
- Si une erreur est trouvée, elle est ajoutée à l'objet `nouvellesErreurs`.

3. Gestion de la soumission :

- `handleSubmit` empêche le comportement par défaut du formulaire (rechargement de la page).
- Si aucune erreur n'est trouvée, les données sont affichées dans la console, et une alerte indique que le formulaire a été soumis avec succès.

4. Affichage des erreurs :

- Si un champ est invalide, un message d'erreur est affiché sous le champ concerné.

◆ Gestion des formulaires avec `useRef`

◆ Avec `useRef` : Gestion non contrôlée

Avec `useRef`, on suit l'approche d'un **composant non contrôlé**. Ici, React n'intervient pas directement dans la gestion des valeurs des champs. Les valeurs des champs sont récupérées directement via une **référence** (et non via un état React).

```
import React, { useRef } from "react";
```

```
function FormulaireUseRef() {
```

```
  // Références pour les champs
```

```
  const nomRef = useRef();
```

```
  const emailRef = useRef();
```

```
  const motDePasseRef = useRef();
```

```
  // Gestionnaire de soumission
```

```
  const handleSubmit = (event) => {
```

```
    event.preventDefault(); // Empêche le rechargement de la page
```

```
  // Récupération des valeurs directement via les références
```

```
  const nom = nomRef.current.value;
```

```
  const email = emailRef.current.value;
```

```
const motDePasse = motDePasseRef.current.value;
```

```
console.log("Données soumises :", { nom, email, motDePasse });  
};
```

```
return (
```

```
<form onSubmit={handleSubmit}>
```

```
<div>
```

```
<label>
```

```
Nom :
```

```
<input type="text" ref={nomRef} />
```

```
</label>
```

```
</div>
```

```
<div>
```

```
<label>
```

```
Email :
```

```
<input type="email" ref={emailRef} />
```

```
</label>
```

```
</div>
```

```
<div>
```

```
<label>
```

```
Mot de Passe :
```

```
<input type="password" ref={motDePasseRef} />
```

```
</label>
```

```
</div>
```



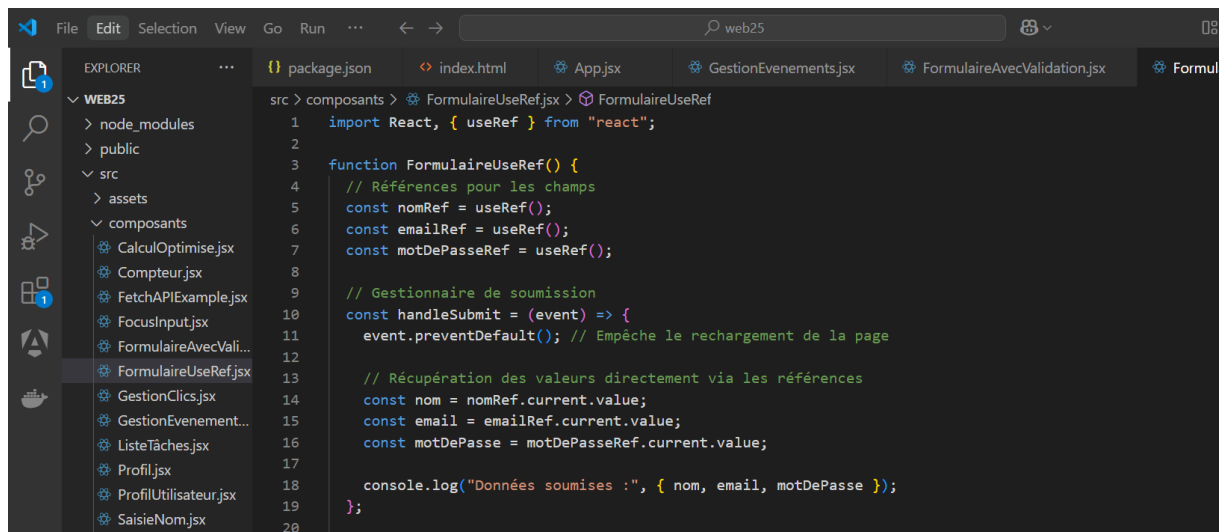
```
<button type="submit">Soumettre</button>
```

```
</form>
```

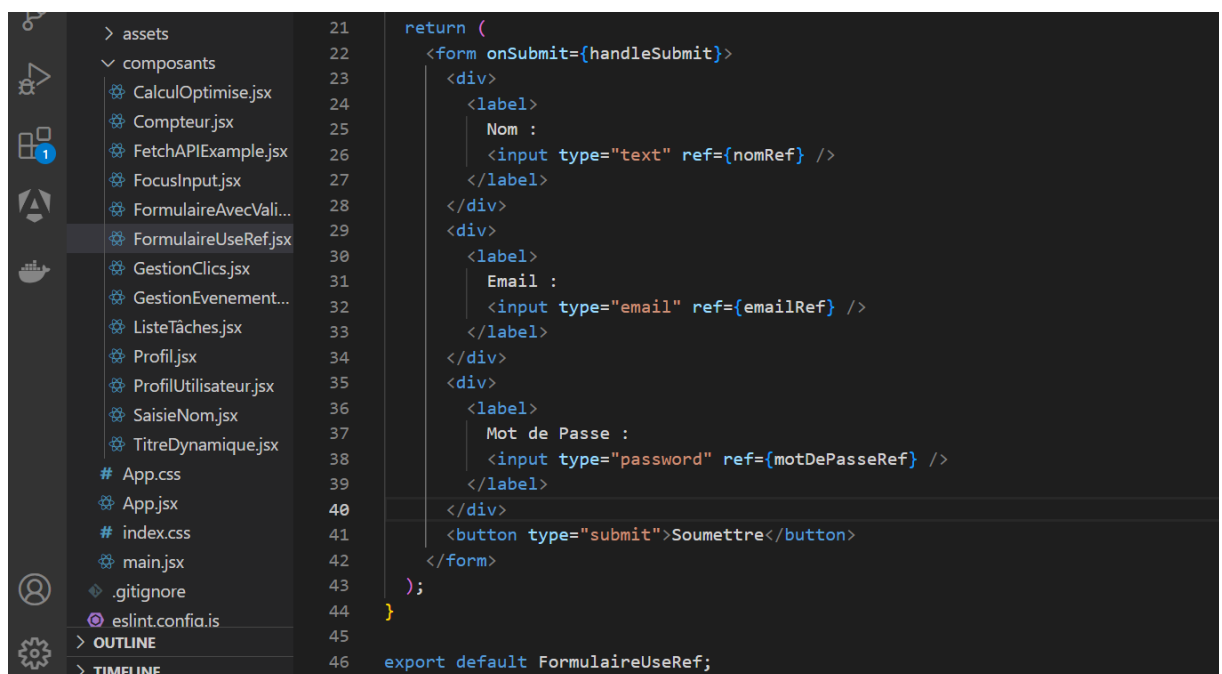
```
);
```

```
}
```

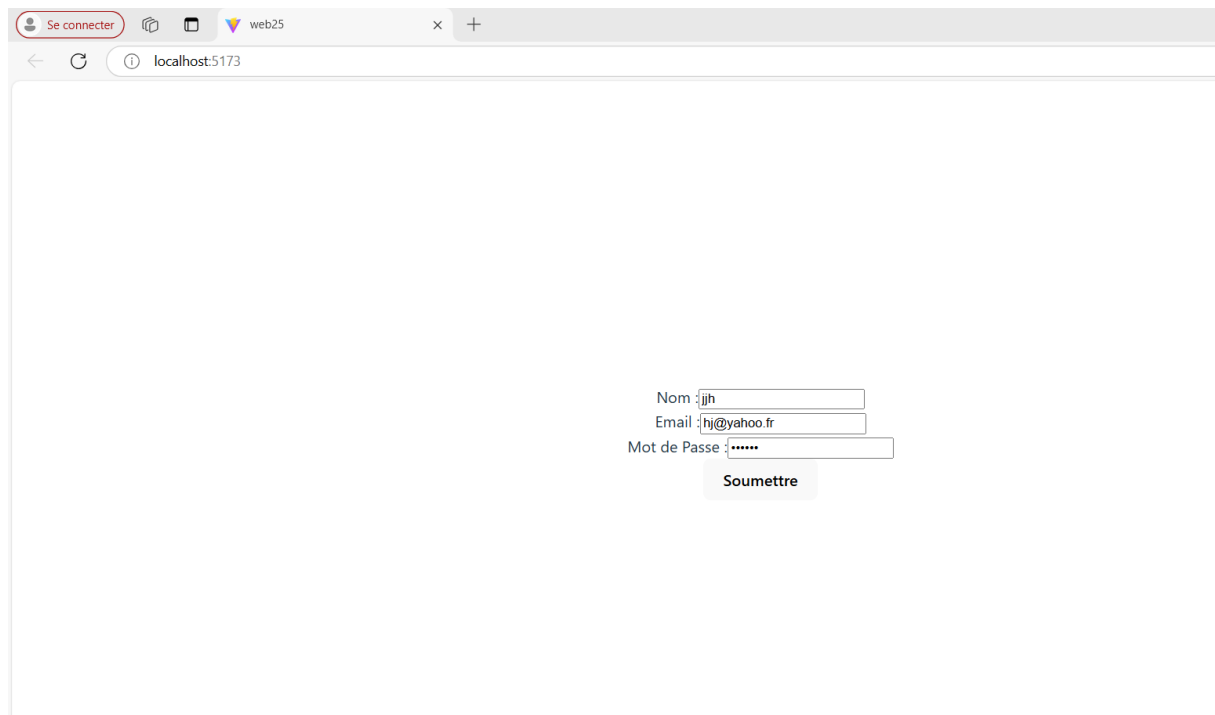
```
export default FormulaireUseRef;
```



```
1  import React, { useRef } from "react";
2
3  function FormulaireUseRef() {
4    // Références pour les champs
5    const nomRef = useRef();
6    const emailRef = useRef();
7    const motDePasseRef = useRef();
8
9    // Gestionnaire de soumission
10   const handleSubmit = (event) => {
11     event.preventDefault(); // Empêche le rechargement de la page
12
13     // Récupération des valeurs directement via les références
14     const nom = nomRef.current.value;
15     const email = emailRef.current.value;
16     const motDePasse = motDePasseRef.current.value;
17
18     console.log("Données soumises :", { nom, email, motDePasse });
19   };
20 }
```



```
21
22   <form onSubmit={handleSubmit}>
23     <div>
24       <label>
25         Nom :
26         <input type="text" ref={nomRef} />
27       </label>
28     </div>
29     <div>
30       <label>
31         Email :
32         <input type="email" ref={emailRef} />
33       </label>
34     </div>
35     <div>
36       <label>
37         Mot de Passe :
38         <input type="password" ref={motDePasseRef} />
39       </label>
40     </div>
41     <button type="submit">Soumettre</button>
42   </form>
43 );
44 }
45
46 export default FormulaireUseRef;
```



- **useRef pour chaque champ :**

- Une référence est attachée à chaque champ via l'attribut `ref`.

- **Récupération des valeurs :**

- Lors de la soumission, les valeurs sont récupérées via `nomRef.current.value`, sans passer par un état React.

- ◆ **Comparaison entre `useState` et `useRef`**

Aspect	<code>useState</code>	<code>useRef</code>
Approche	Composant contrôlé	Composant non contrôlé
Gestion des valeurs	Gérée par React via l'état	Gérée directement dans le DOM
Mise à jour en direct	Oui, à chaque modification	Non, récupération manuelle
Utilisation recommandée	Formulaires interactifs (validation, prévisualisation)	Cas simples ou intégration externe
Exemple	Mise à jour de <code>formData</code>	Accès via <code>ref.current.value</code>

◆ Chapitre 5 : Gestion Globale de l'État et Navigation et Routing

◆ Contexte et useContext

Le hook `useContext` est une manière d'éviter de passer des props manuellement à travers plusieurs niveaux de composants. Il permet de partager des données (comme un thème, un utilisateur, ou une langue) entre plusieurs composants.

Exemple avec `useContext` : Gestion d'un thème

```
import React, { createContext, useContext, useState } from "react";
```

```
// Création du contexte
```

```
const ThemeContext = createContext();
```

```
function App() {
```

```
  const [theme, setTheme] = useState("clair");
```

```
  return (
```

```
    <ThemeContext.Provider value={{ theme, setTheme }}>
```

```
      <div style={{ padding: "20px" }}>
```

```
        <h1>Application avec useContext</h1>
```

```
        <ToggleTheme />
```

```
        <Content />
```

```
      </div>
```

```
    </ThemeContext.Provider>
```

```
  );
```

```
}
```

```
// Composant pour basculer le thème

function ToggleTheme() {

  const { theme, setTheme } = useContext(ThemeContext);

  return (

    <button

      onClick={() => setTheme(theme === "clair" ? "sombre" : "clair")}

      style={{

        padding: "10px",

        margin: "10px 0",

        background: theme === "clair" ? "black" : "white",

        color: theme === "clair" ? "white" : "black",

        border: "none",

      }}

    >

      Passer en thème {theme === "clair" ? "sombre" : "clair"}

    </button>

  );

}
```

```
// Composant pour afficher du contenu

function Content() {

  const { theme } = useContext(ThemeContext);
```

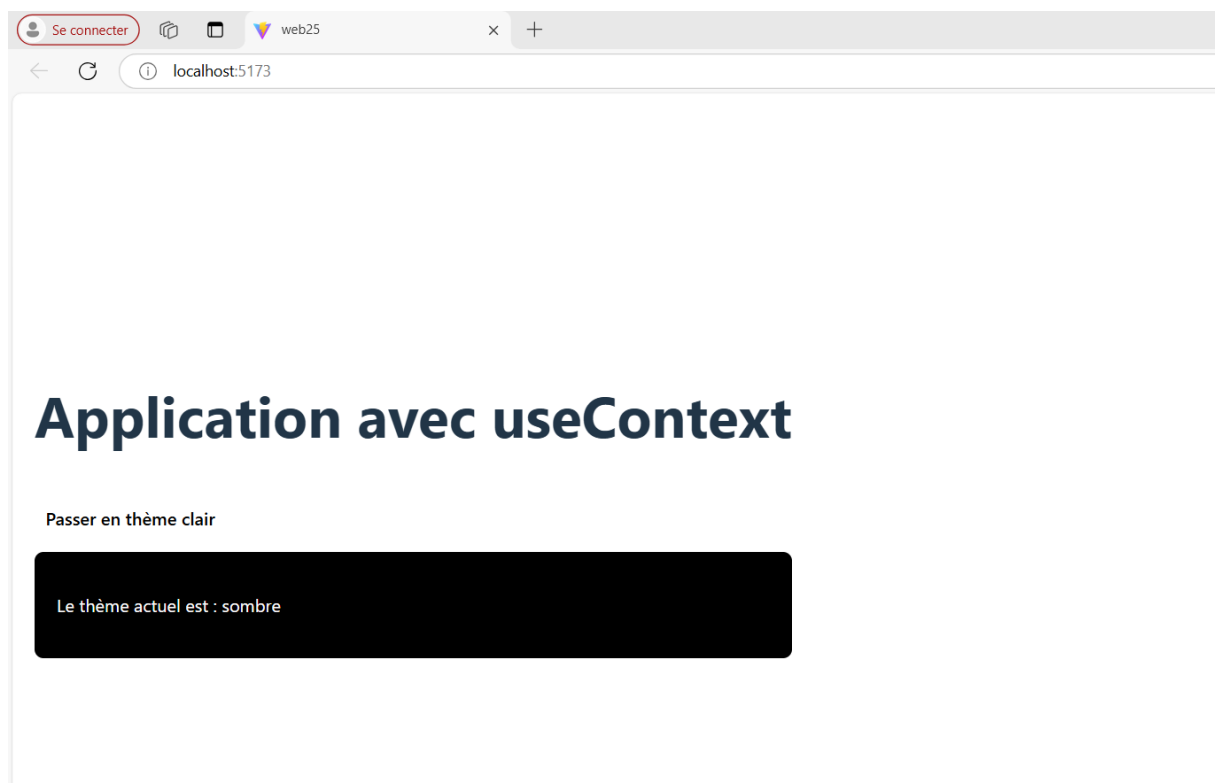
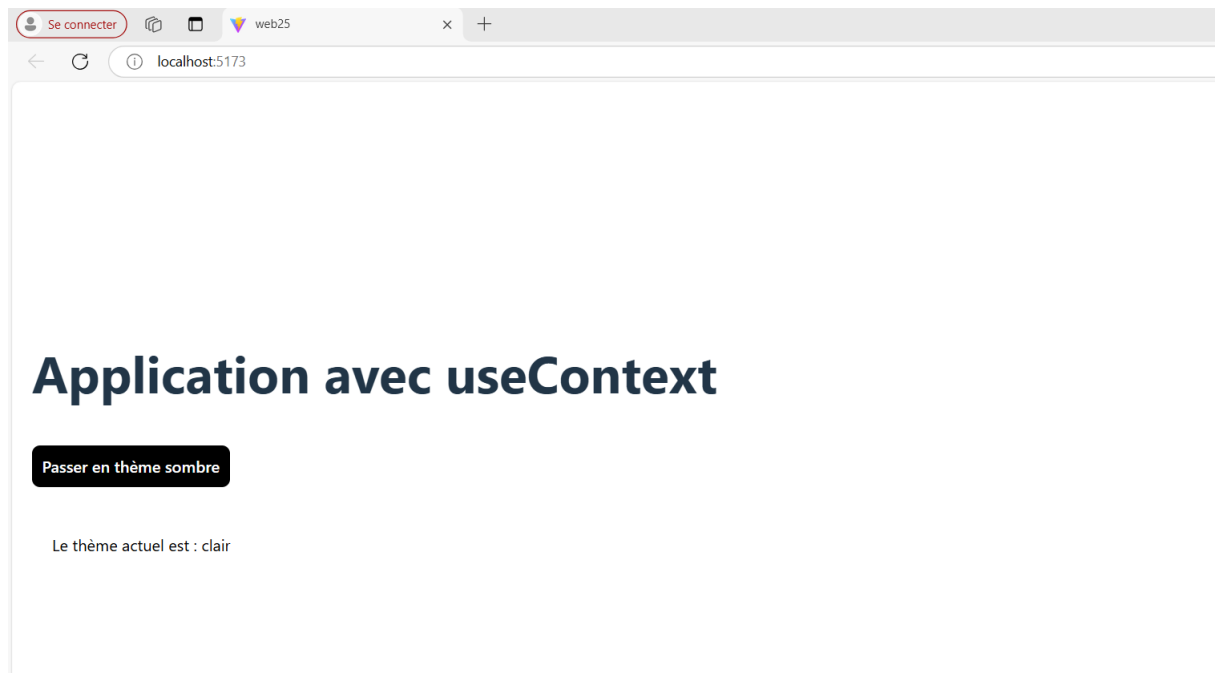
```
return (  
  <div  
    style={{  
      background: theme === "clair" ? "white" : "black",  
      color: theme === "clair" ? "black" : "white",  
      padding: "20px",  
      borderRadius: "8px",  
    }}  
  >  
    <p>Le thème actuel est : {theme}</p>  
  </div>  
);  
}  
  
export default App;
```

```
File Edit Selection View Go Run ... web25
EXPLORER
  WEB25
    node_modules
    public
    src
      assets
      composants
        CalculOptimise.jsx
        Compteur.jsx
        Content.jsx
        Enfant.jsx
        FetchAPIExample.jsx
        FocusInput.jsx
        FormulaireAvecValidation.jsx
        FormulaireUseRef.jsx
        GestionClics.jsx
        GestionEvenements.jsx
        ListeTaches.jsx
  App.jsx X Content.jsx ToggleTheme.jsx Enfant.jsx Parent.jsx GestionEvenements.jsx FormulaireAvecValidation.jsx

src > App.jsx > App
1 import React, { createContext, useContext, useState } from "react";
2
3 // Création du contexte
4 const ThemeContext = createContext();
5
6 function App() {
7   const [theme, setTheme] = useState("clair");
8
9   return (
10     <ThemeContext.Provider value={{ theme, setTheme }}>
11       <div style={{ padding: "20px" }}>
12         <h1>Application avec useContext</h1>
13         <ToggleTheme />
14         <Content />
15       </div>
16     </ThemeContext.Provider>
17   );
18 }
19
```

```
20 // Composant pour basculer le thème
21 function ToggleTheme() {
22   const { theme, setTheme } = useContext(ThemeContext);
23
24   return (
25     <button
26       onClick={() => setTheme(theme === "clair" ? "sombre" : "clair")}
27       style={{
28         padding: "10px",
29         margin: "10px 0",
30         background: theme === "clair" ? "black" : "white",
31         color: theme === "clair" ? "white" : "black",
32         border: "none",
33       }}
34     >
35       Passer en thème {theme === "clair" ? "sombre" : "clair"}
36     </button>
37   );
38 }
39
```

```
40 // Composant pour afficher du contenu
41 function Content() {
42   const { theme } = useContext(ThemeContext);
43
44   return (
45     <div
46       style={{
47         background: theme === "clair" ? "white" : "black",
48         color: theme === "clair" ? "black" : "white",
49         padding: "20px",
50         borderRadius: "8px",
51       }}
52     >
53       <p>Le thème actuel est : {theme}</p>
54     </div>
55   );
56 }
57
58 export default App;
59
```



◆ React Router : Création de routes

React Router est une bibliothèque standard pour gérer la navigation dans une application React. Elle permet de créer des routes dans une application à page unique (SPA) et de gérer la navigation entre différentes pages sans recharger la page.

Installation de React Router

Pour commencer à utiliser React Router dans ton projet, tu dois d'abord l'installer avec npm :

npm install react-router-dom

```
found 0 vulnerabilities
D:\CFITECH\React\cours react 2025\exercices\web25>npm install react-router-dom
```



◆ Création de Routes avec React Router

Dans React, une "route" est une association entre une URL et un composant. Pour déclarer des routes, tu utilises les composants suivants de React Router :

- **BrowserRouter** : Permet de gérer l'historique et les redirections dans une application.
- **Route** : Définit un chemin (URL) et le composant qui sera affiché lorsque ce chemin est atteint.
- **Link** : Permet de créer des liens de navigation entre les différentes pages.

Exemple de base avec des routes

Structure du projet :

Voici comment nous allons organiser les fichiers pour séparer les composants :

src/

```
|
|
|— App.jsx           // Composant principal
|— composants/
|   |— Navbar.jsx    // Menu de navigation
|
```



```
|— pages/
| |— Home.jsx      // Page d'accueil
| |— About.jsx     // Page À propos
| |— Contact.jsx   // Page Contact
| |— NotFound.jsx  // Page 404
```

1. Navbar.jsx (Menu de navigation)

Le composant pour afficher un menu de navigation simple avec des liens.

```
import React from "react";
```

```
import { Link } from "react-router-dom";
```

```
function Navbar() {
```

```
  return (
```

```
    <nav style={{ padding: "10px", backgroundColor: "#f0f0f0" }}>
```

```
      <ul style={{ display: "flex", gap: "10px", listStyleType: "none" }}>
```

```
        <li>
```

```
          <Link to="/">Accueil</Link>
```

```
        </li>
```

```
        <li>
```

```
          <Link to="/about">À propos</Link>
```

```
        </li>
```

```
        <li>
```

```
          <Link to="/contact">Contact</Link>
```

```
        </li>
```

```
      </ul>
```

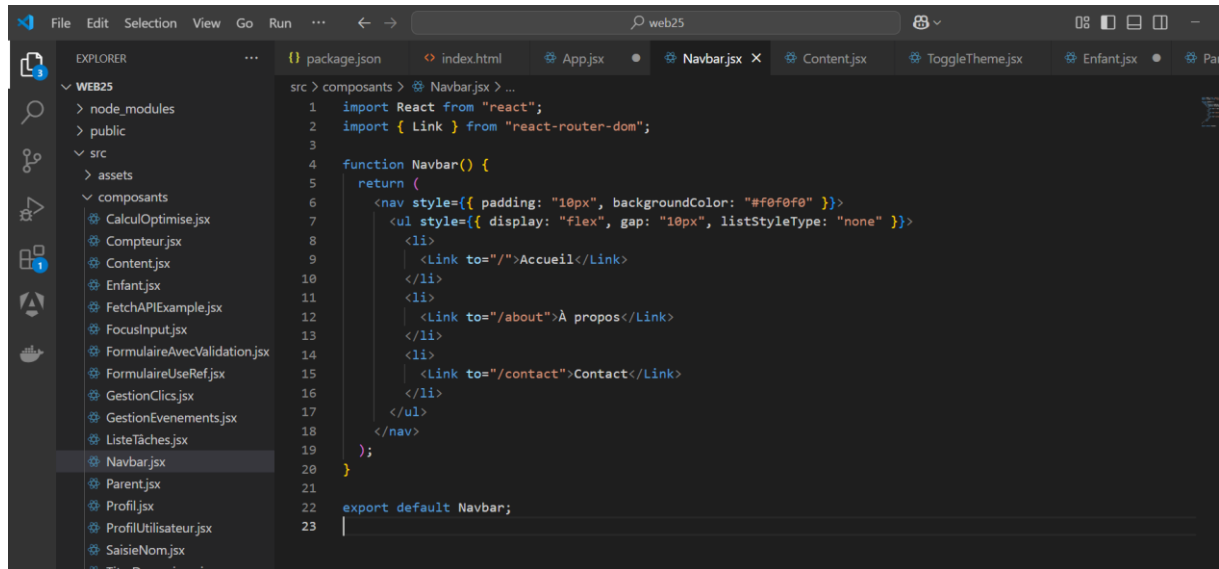
```

    </nav>

  );
}

```

export default Navbar;



2. Home.jsx (Page d'accueil)

Le composant pour la page d'accueil.

```
import React from "react";
```

```
function Home() {
```

```
  return (
```

```
    <div>
```

```
      <h1>Page d'accueil</h1>
```

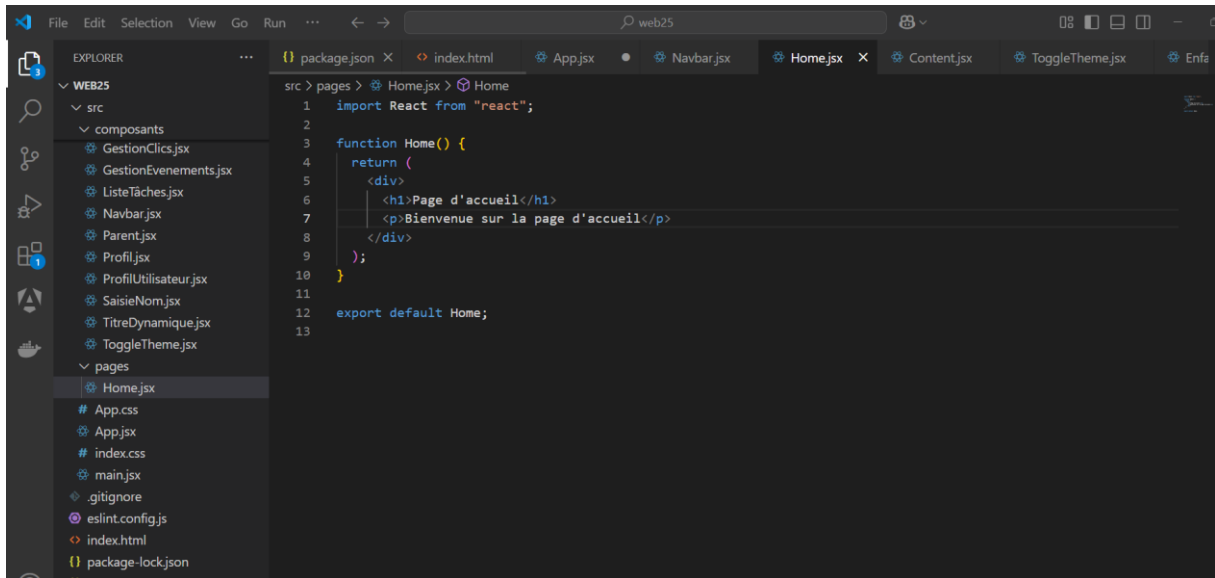
```
      <p>Bienvenue sur la page d'accueil</p>
```

```
    </div>
```

```
  );
```

```
}
```

export default Home;



3. About.jsx (Page À propos)

Le composant pour la page "À propos".

```
import React from "react";
```

```
function About() {
```

```
  return (
```

```
    <div>
```

```
      <h1>À propos</h1>
```

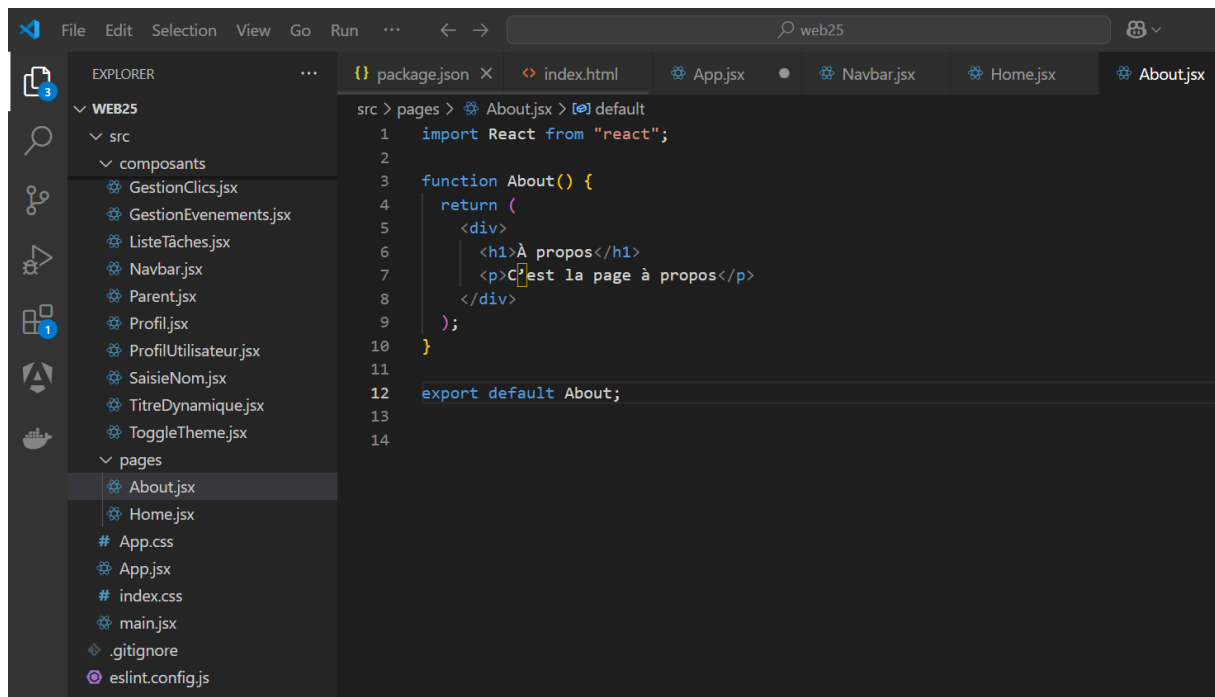
```
      <p>C'est la page à propos</p>
```

```
    </div>
```

```
  );
```

```
}
```

export default About;



4. Contact.jsx (Page Contact)

Le composant pour la page "Contact".

```
import React from "react";
```

```
function Contact() {
```

```
  return (
```

```
    <div>
```

```
      <h1>Contact</h1>
```

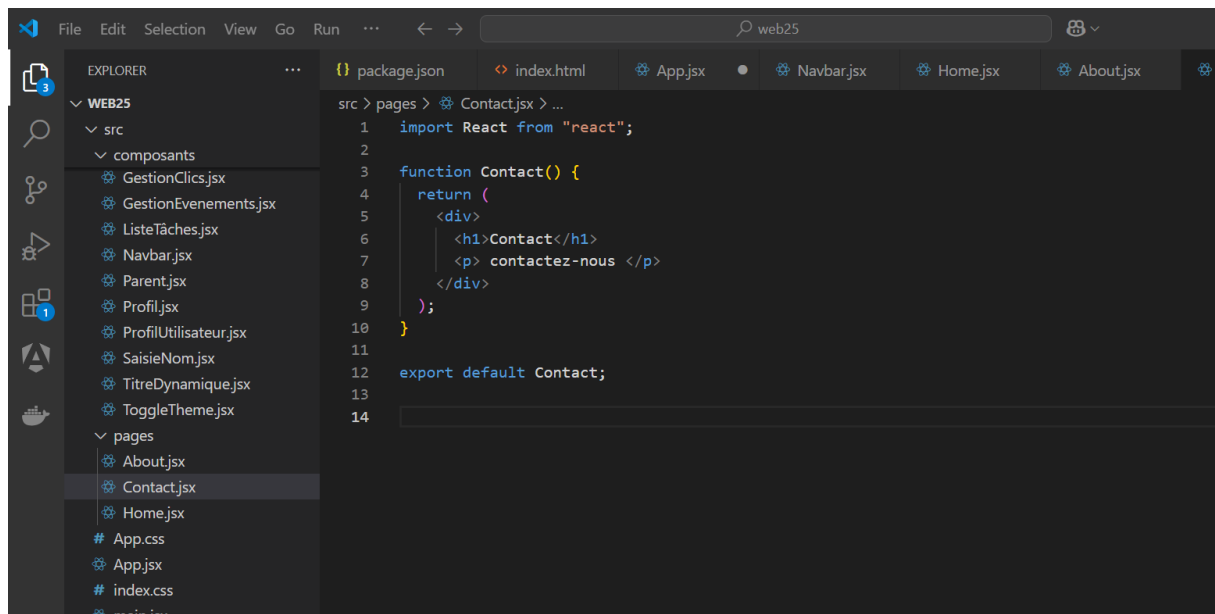
```
      <p> contactez-nous </p>
```

```
    </div>
```

```
  );
```

```
}
```

export default Contact;



5. NotFound.jsx (Page 404)

Le composant pour gérer les erreurs 404 (page non trouvée).

```
import React from "react";
```

```
function NotFound() {
```

```
  return (  
    <div>
```

```
      <h1>404</h1>
```

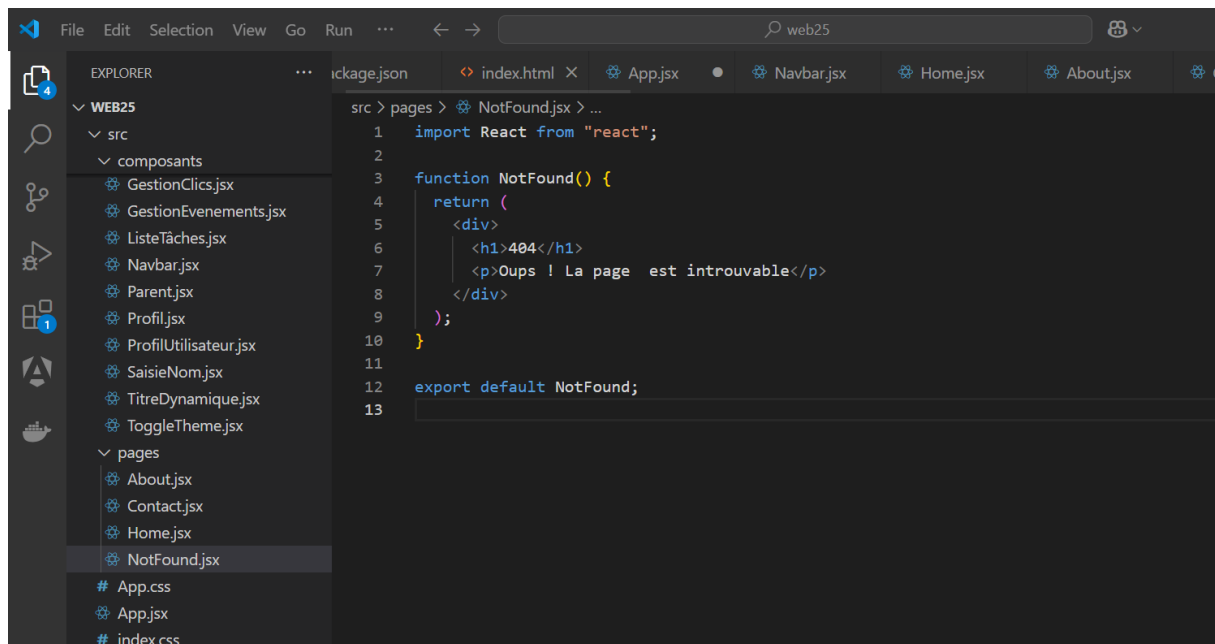
```
      <p>Oups ! La page est introuvable</p>
```

```
    </div>
```

```
  );
```

```
}
```

export default NotFound;



6. App.jsx (Composant principal)

Le composant principal qui intègre tout, y compris React Router.

```
import React from "react";
```

```
import { BrowserRouter as Router, Route, Routes } from "react-router-dom";
```

```
// Import des composants
```

```
import Navbar from "../composants/Navbar";
```

```
import Home from "../pages/Home";
```

```
import About from "../pages/About";
```

```
import Contact from "../pages/Contact";
```

```
import NotFound from "../pages/NotFound";
```

```
function App() {
```

```
  return (
```

```
<Router>
```

```
<div>
```

```
  { /* Menu de navigation */ }
```

```
<Navbar />
```

```
  { /* Gestion des routes */ }
```

```
<Routes>
```

```
  <Route path="/" element={ <Home /> } />
```

```
  <Route path="/about" element={ <About /> } />
```

```
  <Route path="/contact" element={ <Contact /> } />
```

```
  <Route path="*" element={ <NotFound /> } /> { /* Page 404 */ }
```

```
</Routes>
```

```
</div>
```

```
</Router>
```

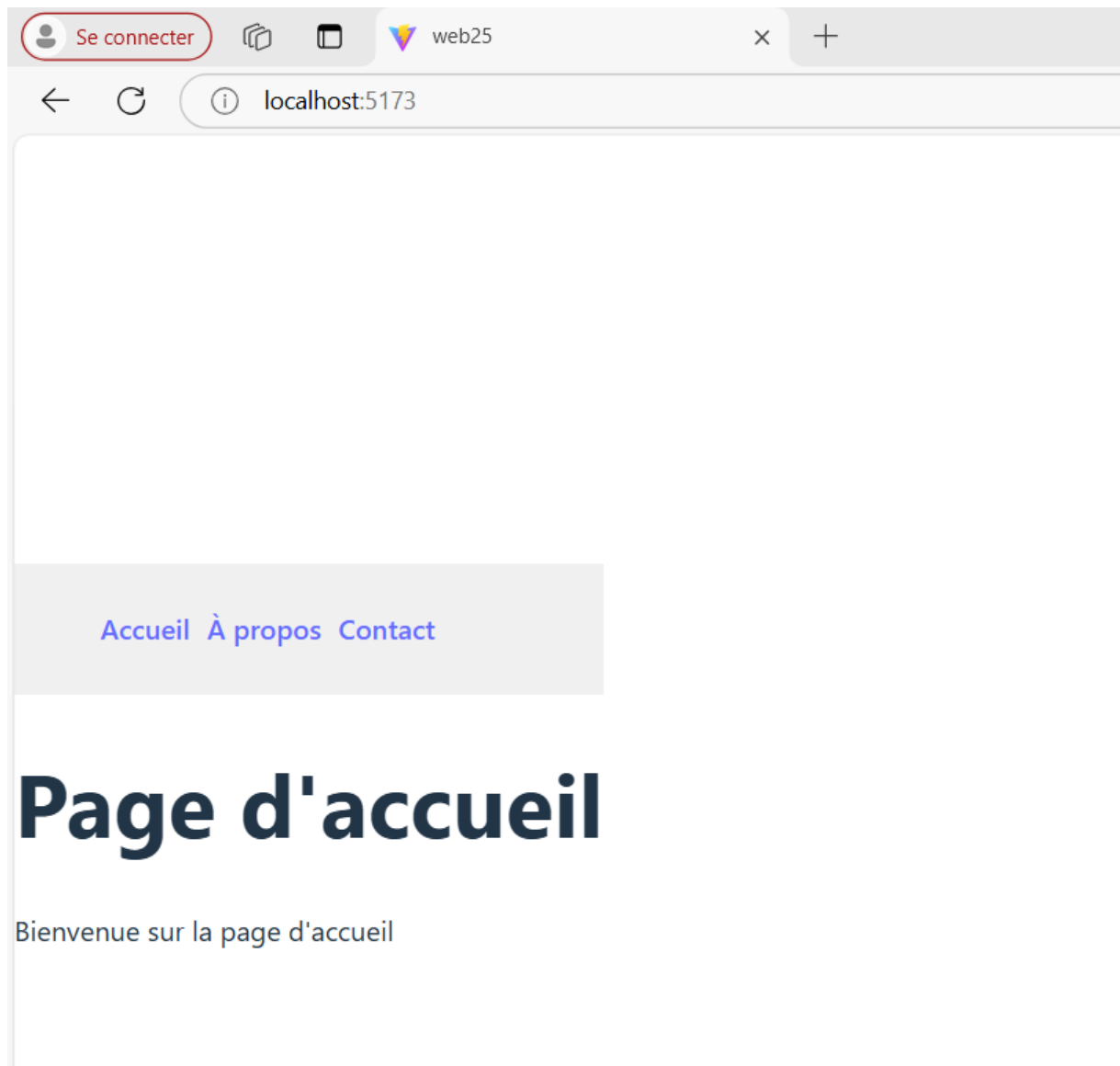
```
);
```

```
}
```

```
export default App;
```

```
src > App.jsx > ...
1  import React from "react";
2  import { BrowserRouter as Router, Route, Routes } from "react-router-dom";
3  // Import des composants
4  import Navbar from "../composants/Navbar";
5  import Home from "../pages/Home";
6  import About from "../pages/About";
7  import Contact from "../pages/Contact";
8  import NotFound from "../pages/NotFound";
9
10 function App() {
11   return (
12     <Router>
13       <div>
14         {/* Menu de navigation */}
15         <Navbar />
16
17         {/* Gestion des routes */}
18         <Routes>
19           <Route path="/" element={<Home />} />
20           <Route path="/about" element={<About />} />
21           <Route path="/contact" element={<Contact />} />
22           <Route path="*" element={<NotFound />} /> {/* Page 404 */}
23         </Routes>
24       </div>
25     </Router>
26   );
27 }
28
29 export default App;
```

```
src > App.jsx > App
1  import React from "react";
2  import { BrowserRouter as Router, Route, Routes } from "react-router-dom";
3
4  // Import des composants
5  import Navbar from "../composants/Navbar";
6  import Home from "../pages/Home";
7  import About from "../pages/About";
8  import Contact from "../pages/Contact";
9  // import NotFound from "../pages/NotFound";
10
11 function App() {
12   return (
13     <Router>
14       <div>
15         {/* Menu de navigation */}
16         <Navbar />
17
18         {/* Gestion des routes */}
19         <Routes>
20           <Route path="/" element={<Home />} />
21           <Route path="/about" element={<About />} />
22           <Route path="/contact" element={<Contact />} />
23           {/* <Route path="*" element={<NotFound />} /> Page 404 */}
24         </Routes>
25       </div>
26     </Router>
27   );
28 }
29
30 export default App;
```

App.css qui va centrer ta page et améliorer l'affichage du menu de navigation

```
/* Réinitialisation de base */
body, html {
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
  background-color: #f9f9f9;
  color: #333;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: 100vh;
}

/*Conteneur principal centré */
.container {
  text-align: center;
  width: 80%;
  max-width: 800px;
  background: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0px 0px 10px rgba(0, 0, 0, 0.1);
}

/*Navigation */
nav {
  background-color: #0078d4;
  padding: 15px 0;
  width: 100%;
  position: fixed;
  top: 0;
  left: 0;
  display: flex;
  justify-content: center;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

nav ul {
  list-style: none;
  display: flex;
  padding: 0;
  margin: 0;
}

nav ul li {
  margin: 0 15px;
```

```
}

nav ul li a {
  color: black;
  text-decoration: none;
  font-weight: bold;
  padding: 10px 15px;
  transition: background 0.3s ease;
  border-radius: 5px;
}

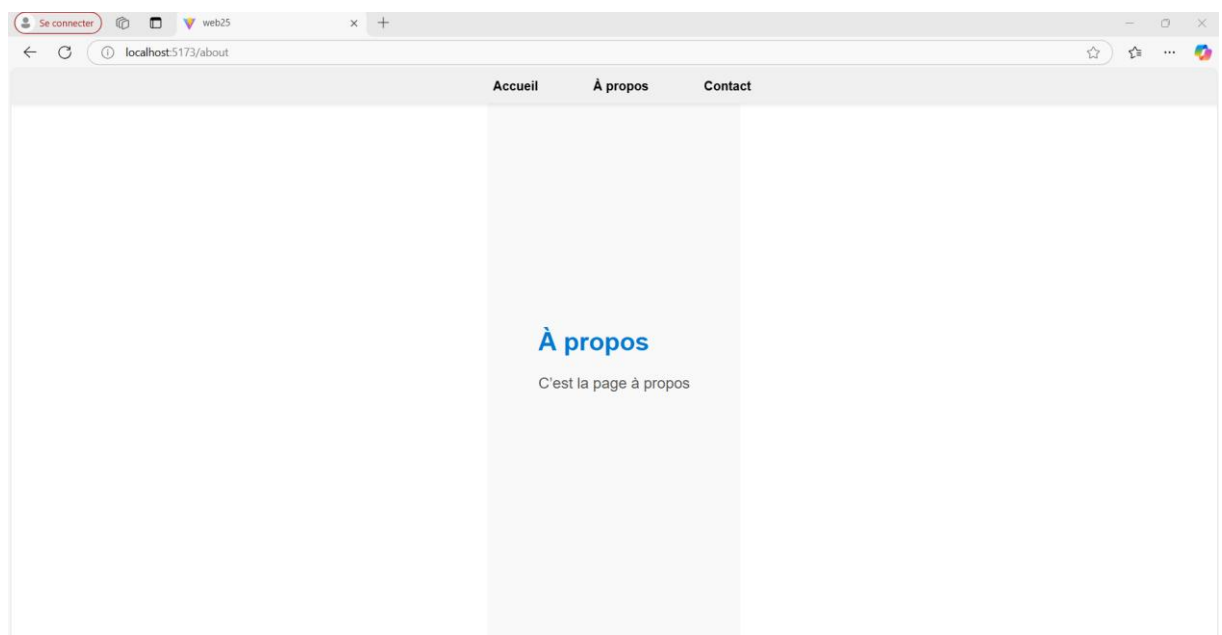
nav ul li a:hover {
  background: rgba(255, 255, 255, 0.2);
}

/*Ajustements pour éviter que le contenu soit caché sous la navbar */
.content {
  margin-top: 80px;
  text-align: center;
}

/*Centrage du texte */
h1 {
  color: #0078d4;
  font-size: 2rem;
}

p {
  font-size: 1.2rem;
  color: #555;
}
```

```
1 import React from "react";
2 import { BrowserRouter as Router, Route, Routes } from "react-router-dom";
3 import "../App.css";
4
5 // Import des composants
6 import Navbar from "../composants/Navbar";
7 import Home from "../pages/Home";
8 import About from "../pages/About";
9 import Contact from "../pages/Contact";
10 // import NotFound from "../pages/NotFound";
11
12 function App() {
13   return (
14     <Router>
15       <div>
16         { /* Menu de navigation */ }
17         <Navbar />
18
19         { /* Gestion des routes */ }
20         <Routes>
21           <Route path="/" element={<Home />} />
22           <Route path="/about" element={<About />} />
23           <Route path="/contact" element={<Contact />} />
24           { /* <Route path="*" element={<NotFound />} /> Page 404 */ }
25         </Routes>
26       </div>
27     </Router>
28   );
29 }
```



Explications du code :

1. Navbar.jsx :

- Ce composant affiche un menu de navigation avec des liens **React Router** grâce à **Link**.
- Les liens mènent vers différentes pages de l'application : /, /about, et /contact.

2. **Pages :**

- Chaque page est un composant React séparé (comme **Home.js**, **About.js**, etc.) pour une meilleure organisation et réutilisation du code.

3. **App.jsx :**

- Il utilise **React Router** pour définir les routes et associer chaque URL à un composant correspondant.
- La route ***** capture toutes les URL non définies et affiche la page **404** (NotFound).